SYSTEMATIC LITERATURE REVIEW · FINAL MANUSCRIPT

Large Language Models for Hardware-Aware Test Generation in Embedded C: A Systematic Review of Peripheral Dependencies, Interrupts, and Execution Validation

Authors: [author names to be supplied]
Affiliations: [affiliations to be supplied]
Corresponding author: [name, email, postal address to be supplied]

105 records screened | 46 included → 28 studies / 29 reports | 9 research questions | 15 figures · 5 tables · 36 references

PRISMA 2020 format — not a compliance claim | No quality assessment applied

Author, affiliation, funding, conflict-of-interest and ORCID fields are placeholders: none appears in the supplied project records, so none is invented here.

Abstract

Embedded C correctness depends on the hardware the code drives — registers, interrupts, DMA and timing — which software-level test harnesses never exercise. This systematic review synthesises how large language models are used for hardware-aware test generation in embedded C across nine research questions, joining two literatures that have not met: non-LLM firmware re-hosting and fuzzing, and LLM-driven test generation. Reported in PRISMA 2020 format without a compliance claim, the review is unregistered and documents no bibliographic database search; 105 registered records yielded 59 exclusions and 46 inclusions, collapsing to 28 unique studies (29 reports), of which 13 LLM-construct studies denominates every LLM-population statement. Each question was answered from its own verified dataset (N = 4–8; the derived gap analysis, N = 6); no quality-assessment instrument was applied and quantitative meta-analysis is impossible on the source evidence. Test types are bimodal — unit 2/4, HIL/system-level 2/4, integration and regression 0/4 — and only 1/4 emits C test code. Zero-shot prompting (6/7), retrieval-augmented generation (5/7) and agentic workflows (4/7) dominate, but eleven hardware-specification categories are each 0/8 as model input. Dependency handling is delegated, not generated (5/6; behaviourally faithful doubles 0 of 27 configurations); LLM-driven interrupt/timing test generation is 0/6; execution is emulator-bound (6/6; real hardware primary 0/6); loops close universally (5/5) on software signals; and real hardware, HIL, fault seeding, fault exposure and human evaluation are each 0/4, as are common benchmark, baseline, quantified metric and evaluation environment. Fourteen of the 28 pairwise study-set overlaps are zero: no study is simultaneously LLM-based, test-generating, embedded-C-source-level and hardware-aware. Author-acknowledged gaps are strongest for hardware-free validation (5/6), narrow advisory LLM capability (5/6) and peripheral dependence (4/6). Every zero is bounded to the verified dataset and licenses no prevalence claim; the structure indicates a pre-paradigm intersection, not a failing literature.

Keywords

large language models · embedded software testing · automated test generation · hardware-aware testing · peripheral dependencies · systematic literature review

1. Introduction

Embedded software — the C and C++ code that runs on microcontrollers and firmware and interacts directly with sensors, actuators, buses, device registers, interrupts and direct-memory-access (DMA) channels — is foundational to safety-critical and resource-constrained systems, from automotive electronic control units to industrial controllers. Testing such code is harder than testing general-purpose software because correctness is entangled with the hardware it drives: a peripheral-dependent routine may compile and pass an isolated unit test yet misbehave once it touches a device register, an interrupt line or a DMA buffer, and asynchronous interrupts and timing constraints can alter behaviour in ways that a purely software-level test harness never exercises. These are not incidental edge cases but the core difficulty that embedded testing must confront, and they are exactly the properties that make hardware awareness central rather than peripheral to the problem.

Because that entanglement is so fundamental, a body of systems and security research has made the hardware abstraction itself a research object in order to test embedded code without complete physical hardware. Firmware re-hosting and fuzzing frameworks model peripheral interfaces and memory-mapped I/O explicitly — P2IM through automatic peripheral-interface modelling [13], HALucinator through abstraction-layer emulation [14], Fuzzware through precise MMIO modelling [15], and DICE through automatic emulation of DMA input channels [16] — while a parallel line of work treats interrupts and asynchronous events as first-order obstacles, modelling interrupt service routines and interrupt-driven state changes to make firmware behaviour observable (IEmu [10], AIM [12], AidFuzzer [9]). This literature establishes the hardware-modelling problems concretely, but it is largely infrastructure research that predates, and is independent of, the recent rise of large language models (LLMs).

LLMs have meanwhile become a mainstream engine for automated test generation and code synthesis across software engineering, as documented in broad surveys of software testing with LLMs [21]. Within the embedded domain specifically, the studies reviewed here show LLMs being applied to concrete testing tasks rather than generic code generation: retrieval-augmented generation of unit tests for industrial embedded C (EmbC-Test [1]); a structured multi-agent workflow producing unit tests with library-aware doubles and iterative repair for firmware (openSIL [2]); requirement-driven functional-test generation for ECUs executed on hardware-in-the-loop benches (AutoHIL [3]); execution-path-aware prompting for embedded C/C++ test-case generation (Tran [4]); continual pre-training or retrieval enhancement aimed at low-level embedded code (H2LooP [6], Smarter-not-Bigger [5]); and agentic embedded-IoT development with compile/flash feedback (EmbedGenius [7]). Related work sits adjacent to test generation rather than squarely within it — agent-driven validation and patching of LLM-generated embedded firmware (Abtahi & Azim [8]), comparative evaluation of LLMs on C test-case generation (Guzu et al. [19]), and benchmarking or characterisation studies that assess many models on embedded development and debugging tasks (EmbedAgent [20], Englhardt et al. [11]). Of these, Guzu et al. [19] and the broad survey of software testing with LLMs [21] are **not** members of the reviewed corpus: [19] does not appear in the 105-record screening register at all, having entered the project through a separate dataset, and [21] enters only as one of the six RQ9 evidence studies. Neither is counted toward the 28 included studies, the 46 records, the 29 reports or any cohort denominator, and both are listed separately in Part B of the References. A recurring mechanism across the test-generation efforts is an iterative generate–execute–refine loop, mirroring general test-generation systems that close the loop using compiler errors, coverage, runtime traces or execution results (CoverUp [17], Panta [18], LLMLOOP [22]). Across all of these, it is important not to conflate *producing* a test artefact with demonstrating that the artefact compiles, executes, or is behaviourally correct on hardware; the studies differ sharply on how much of that chain they actually report.

What has received comparatively little attention, on the evidence of the studies this review covers, is a synthesis that treats the two strands above together — LLM-driven test generation *and* the embedded-C-specific dimensions that make it distinctive: peripheral and hardware-configuration dependencies, hardware/software context supplied to the model, interrupt and asynchronous/timing behaviour, the execution and validation environment, feedback loops, and evaluation. The reviewed corpus divides these concerns along the very lines that general software-testing surveys and firmware re-hosting infrastructure leave unaddressed in combination: the broad testing-with-LLMs survey carries essentially no embedded-hardware dimension [21], the re-hosting work is not LLM-based [9][13][16], and the emerging LLM test-generation studies each engage only part of the peripheral / interrupt / execution-validation picture [1][2][3][8]. Whether these dimensions are converging into a coherent, hardware-aware capability for embedded-C test generation, or are being pursued by largely separate groups, is precisely the kind of question that an integrated, evidence-traceable review is positioned to examine. This review's contribution is therefore one of scope and analytical framing rather than a claim of bibliographic priority.

Synthesising both the *context provided to the model* and the *environment where generated tests are compiled, executed or validated* matters because the two jointly bound how much hardware-awareness a result can carry. On the context side, whether an LLM is given datasheet, register or peripheral specifications, and interrupt or timing constraints — versus what is merely discussed in the paper — bears directly on whether a generated test can encode hardware-specific behaviour. On the validation side, the execution setting (emulator, virtualised real-time system, hardware-in-the-loop bench or physical device) determines which hardware behaviours a test can evidence at all; producing an artefact and demonstrating that it passes on real hardware are different levels of evidence, and conflating them would overstate a result's hardware validity. Examining these dimensions as named, separable concerns — rather than folding them into a generic "code quality" account — is therefore central to judging how hardware-aware LLM test generation for embedded C actually is.

To examine this systematically, the review addresses nine research questions, using the approved wording recorded in the project. RQ1–RQ8 are treated as primary empirical dimensions, each answered from its own verified dataset; RQ9 synthesises the evidence and explicitly reported limitations into research gaps and future directions, and is reported with its own distinct denominators.

- **RQ1.** What types of tests for embedded C software (unit, integration, hardware-in-the-loop, regression) are generated or supported by LLM-based approaches?
- **RQ2.** Which LLMs and which prompting or adaptation strategies (zero/few-shot, RAG, fine-tuning, agentic workflows) are used for embedded test generation?
- **RQ3.** What hardware/software context (datasheets, register maps, HAL/driver code, specifications) is provided to LLMs for test generation?
- **RQ4.** How do LLM-based approaches handle hardware and peripheral dependencies (mocks, stubs, fakes, simulators)?
- **RQ5.** How do existing approaches address interrupts, asynchronous behavior, and timing constraints?
- **RQ6.** In what environments (emulators, simulators, virtual platforms, real hardware, HIL) are generated tests executed and validated?
- **RQ7.** What feedback loops (compiler errors, coverage, runtime traces, execution results) are used to refine generated tests?
- **RQ8.** What metrics and validation methods (compilability, pass rate, coverage, mutation score, fault detection, human evaluation) are used to evaluate generated tests?
- **RQ9.** What limitations and research gaps remain in hardware-aware LLM-based test generation for embedded C?

The contributions of this review are the following. First, it provides a structured synthesis of LLM-based test-generation approaches relevant to embedded C, organised around the nine questions above. Second, it analyses, jointly and as separable dimensions, context provision, peripheral-dependency handling, interrupt and asynchronous/timing behaviour, execution environments, feedback loops, and evaluation — rather than reporting them in isolation. Third, it explicitly separates study-level LLM evidence from contextual non-LLM firmware infrastructure and from general software-engineering comparators, using a documented record/report/study counting rule so that LLM-population findings are not computed over non-LLM studies. Fourth, it delivers an evidence-traceable analysis of limitations, underexplored areas, reporting gaps and future directions (RQ9) that keeps method-use absences, limitation-discussion silences and reporting-completeness issues distinct, and states each gap only at the strength the underlying counts license. Fifth, it reports the screening and search-history evidence transparently, including the candid limitation that the project's original search documentation is incomplete and that eligibility was adjudicated against criteria finalised retrospectively. Detailed findings — the verified counts, the two-cluster structure of the corpus, and the gap matrix — are reserved for the Results section and are not claimed here.

The remainder of the paper is organised as follows. Section 2 fixes the vocabulary and the counting conventions on which every count in this review depends. Section 3 (Methodology) documents the review protocol, information sources, screening and eligibility decisions, the record/report/study counting rules, and the limits of the available search log. Section 4 (Results) reports study selection and characteristics, the verified findings for RQ1–RQ8 and the evidence-derived limitations for RQ9, supported by figures and tables generated exclusively from audited counts. Section 5 (Discussion) interprets the cross-dimension structure of the corpus and its construct-validity constraints. Section 6 develops the research gaps and future directions from the verified gap matrix, Section 7 sets out the threats to the validity of this review, and Section 8 concludes. References follow, with the search protocol and reporting-

gap disclosure, the eligibility criteria and decision defaults, the included-study characteristics, the metadata conflict register, the unresolved items requiring author decision, and the supplementary-materials inventory given as Appendices A to F.

---

2. Background and Related Concepts

This section defines the vocabulary and the counting conventions on which every count in Section 4 depends. It introduces no new evidence, no new studies and no new citations beyond those already listed in the References; its purpose is to make the review's classification lens explicit before results are reported against it, because — as Section 7 documents — several of this review's zero counts are sensitive to the definitions given here.

2.1 Why hardware awareness is a distinct problem in embedded C testing

Embedded C code is not self-contained. It reads and writes memory-mapped device registers, configures and services interrupt lines, moves data through direct-memory-access (DMA) channels, and observes or produces signals on buses and pins under timing constraints that the compiler does not model. Two consequences follow for testing, and both recur throughout the results reported in Section 4.

First, **correctness is entangled with the substrate**. A routine that compiles and passes an isolated unit test may still misbehave once it touches a peripheral register, an interrupt vector or a DMA descriptor, because the behaviour under test is defined jointly by the C source and by hardware semantics that the source only references. A test that never instantiates those semantics can pass without exercising them.

Second, **the cost of exercising real hardware is high**, which has produced a substantial literature on testing firmware without complete physical devices. That literature treats the hardware abstraction itself as the research object: peripheral interfaces and memory-mapped I/O are modelled explicitly so that firmware can be executed off-target, and interrupt service routines and interrupt-driven state changes are modelled so that asynchronous firmware behaviour becomes observable at all. Within this review's corpus, that work forms the *Contextual non-LLM infrastructure* cohort (§2.4) — P2IM [13], HALucinator [14], Fuzzware [15], DICE [16], IEmu [10], AIM [12] and AidFuzzer [9] — and it is, by construction, not LLM-based.

Large language models (LLMs) have meanwhile become a mainstream engine for automated test generation and code synthesis across software engineering [21], and a body of work now applies them to embedded tasks specifically: retrieval-augmented generation of unit tests for industrial embedded C (EmbC-Test [1]), multi-agent unit-test generation with library-aware doubles and iterative repair for firmware (openSIL [2]), requirement-driven functional-test generation for electronic control units executed on hardware-in-the-loop benches (AutoHIL [3]), execution-path-aware prompting for embedded C/C++ test-case generation [4], low-level-code adaptation through continual pre-training or retrieval enhancement [5][6], and agentic embedded-IoT development with compile/flash feedback (EmbedGenius [7]).

These two bodies of work address complementary halves of the same problem, and the central analytical observation of this review — developed in §5.1 and §4.9.4 — is that within the verified corpus they do not meet inside any single study.

2.2 Constructs examined by this review, and how each is operationalised

Each research question corresponds to a named construct. Because the primary studies use these terms inconsistently (documented in §7.3), the operationalisation below is stated once and applied uniformly.

Test types (RQ1). The four canonical levels named in the review question: *unit*, *integration*, *hardware-in-the-loop (HIL)* and *regression*. A further distinction is preserved between a test type an approach *generates* and one it merely *supports*; in this corpus every reported type is of the generated kind. Output language (C, Python, JSON or another artefact form) is recorded separately, because a test expressed in Python or JSON is not a C test artefact even when its target is C code.

Models and adaptation strategies (RQ2). *Zero-shot* and *few-shot* prompting, *retrieval-augmented generation (RAG)*, *fine-tuning* (recorded separately for the generation model and for any auxiliary component such as an embedding model), and *agentic workflows* — multi-step pipelines in which a model plans, invokes tools and revises. Strategies co-occur within a study, so counts overlap by design and do not sum to the denominator.

Hardware and software context supplied to the model (RQ3). The critical distinction here is between material *provided to the model* and material merely *discussed in the paper*. Only the former counts. Within it, this review separates *software context* (source files, repository structure, build logs, dependency lists, existing tests) from *hardware context* (datasheets, register maps, microcontroller or processor specifications, clock and memory-map configuration, board and electrical-interface documentation, sensor/actuator and communication-protocol specifications, interrupt, timer and DMA specifications, and HAL/driver/BSP/RTOS artefacts). A further distinction separates *inference-time* context from *training-time* consolidation: a corpus of datasheets absorbed into weights during pre-training or continual pre-training is not context supplied at generation time, and is not counted as such.

Peripheral and hardware-dependency handling (RQ4). Handling divides into two families that this review keeps apart. The *provision* family creates substitutes — test doubles such as mocks, stubs and fakes, or generated peripheral models, device drivers, HAL layers, wrappers and link-time replacements. The *replacement* family swaps the execution substrate — an emulator, simulator or physical target replaces the absent hardware. A *hybrid* strategy uses both. Note that the term "mock" is used with two different senses in this literature: as a unit-test double, and as a verb denoting system-level state simulation; the two are not merged. A test double is counted as *behaviourally faithful* only where register, timing, interrupt or protocol semantics are modelled, and faithfulness is treated as a separate question from whether faithfulness was ever *measured*.

Interrupts, asynchronous behaviour and timing (RQ5). Three levels are distinguished: the *presence* of interrupts in the target; the *modelling* of interrupt behaviour (interrupt service routine handling, interrupt injection, nested-interrupt and priority semantics, interrupt-controller simulation); and the *testing* of temporal properties (worst-case execution time, jitter, deadline validation, race conditions, delay duration, periodic execution, timeout as a functional constraint). Time *control* — a fake or mock clock, a deterministic simulation-time mechanism, explicit clock-domain modelling — is recorded separately from time *inheritance*, in which virtual time exists only as a side effect of running an emulator. Concurrency counts are reported alongside, because an absence of operating-system-level scheduling semantics is not an absence of concurrency handling.

Execution and validation environments (RQ6). Five environment classes are recognised: *emulator*, *simulator* (strict sense), *virtual platform*, *hardware-in-the-loop* rig and *real hardware*. Two further distinctions matter. A real target may serve as the *primary* execution environment or only in a *secondary* role such as confirming a bug found off-target or providing a baseline for comparison; the two are counted separately. And the emulator/simulator boundary is a classification decision, not an empirical fact: the same tooling may be coded on either side depending on the lens applied, which is why this review reports the conflict rather than resolving it (§7.3).

Feedback loops (RQ7). A loop is *closed* when generated artefacts are refined using signals returned from processing them, and *one-shot* when they are not. Feedback sources are recorded individually: compiler errors, compiler warnings, coverage, runtime traces, static-analysis findings, execution results, test failures, hardware or peripheral feedback, mutation or fault information, human feedback, and model self-critique. Evaluation-only measurement is not a refinement input and is not counted as a loop. Loop *frequency* is never treated as evidence of loop *effectiveness*.

Evaluation metrics and validation methods (RQ8). Validation methods (execution, compilation, coverage analysis, mutation testing, simulation or emulation, benchmark comparison, fault seeding, fault exposure, human evaluation, expert review, qualitative and static analysis) are recorded separately from headline metric names (coverage, mutation score, pass rate, pass@k), because a shared metric *name* does not imply a shared definition or denominator. Comparability is itself a measured property: whether studies share a benchmark, a baseline, a quantified metric or an evaluation environment.

2.3 Counting units: records, reports and studies

This review distinguishes three counting layers, and conflating them is the single most common source of error in the underlying project records. A **record** is one row of the screening register; a study entered for several research questions appears as several records. A **report** is one published or deposited document; one study may have several reports, as when a conference version and a

preprint both exist. A **study** is one unit of scientific work. The verified values for this corpus are 46 records, 29 reports and 28 studies, and each is used only where its own layer is meant (§3.6).

A related distinction runs through every count in Section 4: a **study** is not a **configuration**. Several research questions are extracted at configuration level — an experimental setting, a model variant or a prompt condition within one study — and configuration-level counts (51 for RQ2, 32 for RQ3, 27 for RQ4, 24 for RQ5, 11 for RQ6, 25 for RQ7, 46 for RQ8) are never presented as study prevalences.

2.4 Cohorts and why they are demarcated rather than merged

Not every included study belongs to the population about which the review's headline question is asked. Eligibility adjudication therefore *demarcated* four cohorts rather than excluding studies whose LLM relevance failed, so that no verified evidence was discarded while no non-LLM study could inflate an LLM statistic. The **LLM-construct** cohort (13 studies) is the population for every LLM-population statement, so its denominator is 13 and never 28. The **Contextual non-LLM infrastructure** cohort (8 studies) contributes environment and mechanism framing for RQ4–RQ6 only. The **general-SE comparator** cohort (4 studies) calibrates RQ7/RQ8 mechanisms but is not counted as an embedded-C finding. The **grey-zone** cohort (3 studies, one of whose identity is unresolved in the records) is retained verbatim with flags. Cohort definitions, membership and counting rules are given in full in Table 3 (§3.6).

Each research question additionally reports over its own verified dataset, whose study membership and denominator are registered in Table 4 (§4.0). Percentages are therefore valid only inside their own denominator and are never pooled, averaged or summed across research questions.

2.5 Evidence-status vocabulary

Three vocabularies are used throughout Section 4 and are defined here once.

Reporting status. "Not reported" (the source is silent), "Unclear" (the source addresses the point but does not settle it) and "No" (the source states a negative) are kept distinct and are never collapsed into one another. In particular, a silence is never converted into a negative.

Two kinds of zero. A **type I zero** answers "did any verified study *use* method X?" with no. It supports an *underexplored-within-this-dataset* claim. A **type II zero** answers "did any study *discuss* Y as a limitation?" with no. It is silence about a topic, not absence of the topic, and it licenses no gap claim whatsoever. The same construct can be a type I zero in one dataset and a type II zero in another — HIL, HAL and RTOS behaviour are exactly such cases — and the two readings are never substituted for one another (Table 5, §4.9.3).

Evidence grades for gap statements. EA = an author-acknowledged limitation confirmed by the verification record; EA-q = acknowledged but downgraded during verification; SI = synthesis-inferred from adjacent evidence; IFS = inferred from silence; NE = not established (type II); NR = outside the study's stated scope. Single-study items are marked as such and are not inflated into field-level gaps.

2.6 What this section does not do

This section defines terms; it does not survey the field and does not substitute for the systematic evidence reported in Section 4. No claim about the prevalence, effectiveness or maturity of any technique is made here, and none of the counts above is a finding. Section 3 documents how the corpus was assembled and what its documentation limits are; Section 4 reports the verified counts; Sections 5–7 interpret them and bound them.

---

3. Methodology

3.0 Reporting framework and provenance of this section (retrospective-reconstruction disclosure)

This review is reported following the **format** of PRISMA 2020 (Page et al., 2021) and, for search reporting, the PRISMA-S extension guidance; official checklist and flow-diagram resources were consulted (URLs recorded in the project prompt library). Flow diagrams were generated in template 2 (databases/registers + other methods), the only PRISMA 2020 template structurally able to hold records whose discovery route was not a database search.

We state explicitly: reporting in PRISMA 2020 format does not constitute a claim of PRISMA 2020 compliance, and the completeness of the flow diagram demonstrates reporting transparency only — it does not establish the soundness of the search, screening, or synthesis. The project records themselves forbid that claim.

Most process documentation in this project was produced **retrospectively**. The reviews' eligible-record datasets were compiled first (RQ-by-RQ workbooks and per-record verification documents dated 2026-10-08 to 2026-10-09); the screening register, eligibility criteria codes, exclusion reason codes, PRISMA count audit, and this methodology were built afterwards, on 2026-10-12, by auditing the already-existing materials rather than prospectively logging a search as it happened. The exclusion-reason codes in the screening register are themselves labelled "PROPOSED code" and, for several records, "**retroactive code**" — the register's own wording. Throughout this section, reconstructed values are marked as such and distinguished from directly recorded ones.

No protocol registration exists. The project contains no registration record; the words "protocol registration" appear only in drafting instructions that prohibit inventing one. We therefore report this review as unregistered.

3.1 Review protocol and research questions

The review addresses nine questions. RQ1–RQ8 are answered from eight independently compiled, record-level datasets; RQ9 is a **derived** question synthesised over the RQ1–RQ8 corpora (it introduces no new studies):

- **RQ1.** What types of tests for embedded C software (unit, integration, hardware-in-the-loop, regression) are generated or supported by LLM-based approaches?
- **RQ2.** Which LLMs and which prompting or adaptation strategies (zero-shot, few-shot, RAG, fine-tuning, agentic workflows) are used for embedded test generation?
- **RQ3.** What hardware/software context (datasheets, register maps, HAL/driver code, specifications) is provided to LLMs for test generation?
- **RQ4.** How do LLM-based approaches handle hardware and peripheral dependencies (mocks, stubs, fakes, simulators)?
- **RQ5.** How do existing approaches address interrupts, asynchronous behavior, and timing constraints?
- **RQ6.** In what environments (emulators, simulators, virtual platforms, real hardware, HIL) are generated tests executed and validated?
- **RQ7.** What feedback loops (compiler errors, coverage, runtime traces, execution results) are used to refine generated tests?
- **RQ8.** What metrics and validation methods (compilability, pass rate, coverage, mutation score, fault detection, human evaluation) are used to evaluate generated tests?
- **RQ9.** What limitations and research gaps remain in hardware-aware LLM-based test generation for embedded C?

The documented workflow sequence (prompt library, 30 numbered task prompts) ran: project inventory and source-data audit → search-history audit → criteria and screening register → PRISMA count calculation and consistency audit → PRISMA diagram →

quality-assessment decision → per-RQ extraction verification (RQ1–RQ9) → per-RQ syntheses (RQ1–RQ8) → cross-RQ synthesis → RQ9 gap synthesis → figures → methodology/report drafting. The RQ analyses themselves predate this audit chain; the audit reconstructed and verified their provenance without re-running any RQ analysis The project record states this verbatim as "未重启项目、未重做任何 RQ 分析、未引入外部文献" — "the project was not restarted, no RQ analysis was redone, and no external literature was introduced" (PRISMA final-status record, 2026-10-12).

3.2 Information sources

The project records document **exactly one identified-source route with retained evidence**:

- **DeepSeek (AI conversational/search tool) export, 2026-10-12** — file `deepseek_csv_20261012_f6f391.txt` : 59 records, 5 columns (Category, Title, Year, Venue/DOI, Reason for Exclusion). This export contains **excluded records only**; no query string, no result count, and no per-database provenance were retained. 59 of the 105 register rows carry this route verbatim in their "Discovery route (evidence)" column.
- **46 retained (included) records: discovery route NOT RECORDED.** The register's own qualifier for every one of these rows: "local PDF presence = custody only, not route." File-name patterns (`rq_1 find N.pdf` , `rq_2 find_N.pdf`) hint at a discovery process, but the 11 corresponding "find" files are extraction/verification documents (each opens with "PAPER IDENTIFICATION"), not search-result artefacts, and carry no source or "found via" fields.

Bibliographic databases: no search is documented. A full project scan found no `.ris` , `.bib` , or database export of any kind; the strings Scopus / Web of Science / IEEE Xplore / ACM Digital Library / DBLP / PubMed occur **zero times across 105 register rows × 22 columns**. IEEE (×17) and ACM (×14) strings occurring in the venue column denote **publication venues of the papers, not searched platforms**; a paper published in a venue indexed by a database does not evidence that the database was searched. We therefore do **not** claim any systematic database search.

Registers: not applicable. No trial or preprint register is in scope for software-engineering literature; register-related keywords return zero hits across 21 project texts. The PRISMA "registers" box is reported as NOT APPLICABLE rather than 0.

Identification total: unknown. The number 105 is the count of records **registered and adjudicated**, not records identified; the lower bound of the true identification total is 105, the upper bound is unknown. We report no "N records identified" figure.

3.3 Search strategy and search strings

No executed search string exists in the project records for any source, including the DeepSeek route (query string, search fields, and filters all explicitly NOT RECORDED). Candidate strings for Scopus, Web of Science, IEEE Xplore, ACM DL, and arXiv exist in the search-history audit only as labelled **PROPOSED QUERY** entries with unfilled result templates whose placeholder reads "[执行后填写]", i.e. "[to be completed after execution]"; there is no record that they were ever executed, so we do not present them as our search. Consequently:

- PRISMA-S-style per-database query reporting is **not possible** for this review as documented; this is a reproducibility limitation of the underlying process, not of this write-up.
- Filters (date range, language, publication type): **no applied filter is recorded.** Any claim such as "no date restriction was applied" would itself be unsupported; we simply do not state one way or the other.
- The review's evidence base spans works with recorded publication years **2020–2026**; 17 of 46 retained records carry no usable year in the source, which was **not** an eligibility criterion.

Supplementary searches to remedy the missing search log were **proposed** in the audit report (action list, 2026-10-12) but the project contains no execution record for them. If the authors did execute such searches, they must be added here with dates, strings, and result counts **(not recoverable from the project records; Appendix E.)**; if they did not, this section stays exactly as written.

3.4 Inclusion criteria

Criteria were formulated as **PROPOSED** criteria during the 2026-10-12 audit — explicitly *not* described as the historical criteria used to assemble the original datasets — and were then applied to the already-registered records. The register's full-text-decision field preserves this status verbatim on all 46 included rows ("not yet re-screened against the PROPOSED criteria — decision PENDING author approval"), and the final determinations were executed under **author-accepted decision defaults DP-1A–DP-6A, dated 2026-10-12** (the accepted default is recorded in the column header itself).

Criterion codes referenced by the records themselves (with their operative role):

- **IC-1 — publication type / grey literature:** applied to S04, a master's thesis; grey literature is admissible with the DP-5A restriction (descriptive-only synthesis).
- **IC-5 — LLM-construct eligibility** (the study itself uses an LLM for test generation or test support): every included LLM-construct record carries the tag "LLM-construct candidate (IC-5 met by this record)". Failures of IC-5 were **demarcated rather than excluded** under DP-1A/DP-3A (see 3.6).
- **IC-6 — directness of testing relevance:** studies whose testing relevance is indirect (context mechanisms or model substrates) are grey-zone under DP-4A.

Full text available on file was the operative retrieval condition (no record was lost to retrieval failure; see 3.6).

The complete wording of criteria IC-1 through IC-6 is **not stored in the project records** — only these three codes appear in row-level annotations. Reconstructing the full criterion texts **(not recoverable from the project records; Appendix E.)**. Likewise, of the accepted defaults DP-1A–DP-6A, only DP-1A, DP-3A, DP-4A and DP-5A are cited on records (8 + 6 + 4 + 1 times); the texts of DP-2A and DP-6A are not present in any file **(Appendix E, U11.)**.

3.5 Exclusion criteria

The 59 screened-out records carry substantive scope reasons only; **zero** exclusions were attributed to retrieval failure (paywall/unobtainable phrasing returns zero hits). Exclusion codes EC-1–EC-6 are retrospective mappings of the twelve recorded category labels; the register marks them "(PROPOSED code)" and, where the mapping was reconstructed after the fact, "(retroactive code)". The category→code mapping is 1:1 in the reverse direction (each category maps to exactly one code; no category splits), so the codes add no information beyond the original category labels:

Table 1. Exclusion reason codes, definitions as recorded in the screening register, and counts. The codes are retrospective mappings of the twelve recorded category labels and are labelled “(PROPOSED code)” or “(retroactive code)” in the register; the mapping is 1:1 in reverse, so the codes add no information beyond the category labels. Σ = 59.

Code	Definition (register wording)	n
EC-1	Software under test is general/web/mobile/desktop or non-embedded; no embedded-C or firmware artefact	30
EC-3	Target is consensus protocols / blockchain formal verification	9
EC-2	Object is RTL/HDL design verification (SystemVerilog/UVM), not embedded-software testing	5
EC-4	LLM is the system under test; no embedded test artefact produced	5
EC-5	Fuzzing method without embedded-firmware target (kernel / GUI / black-box library / microservice / cloud)	5
EC-6	Contribution is pedagogy/curriculum, not a testing method or tool	5
	Σ	**59** ✓

(Verified against the register: the EC-2 definition string reads "SystemVerilog/UVM" exactly as reproduced above.)

RQ1 and RQ2 have no exclusion records at all — their datasets (4 and 7 included studies) entered the project without a documented rejected-candidate pool, unlike RQ3–RQ8 whose candidates were screened against the above reasons. The audit explicitly flags that it "cannot confirm whether RQ1/RQ2 papers passed systematic screening." **(Appendix E, U12.)**

3.6 Screening and study selection

Deduplication and pre-screening removals: zero. All 105 registered rows were adjudicated; no row was ever deleted. Cross-RQ re-registration of the same study was handled by **annotation, not removal**: 28 rows are tagged into 10 "SAME-STUDY CLUSTER" groups (S02×2, S05×3, S06×3, S07×2, S08×2, S09×3, S10×2, S13×4, S17×4, S26×3), and 77 rows state "no duplicate detected within the included records by normalized-title clustering." The de-duplication method evidenced is therefore **normalised-title clustering within the included records**, implemented as annotation. The register's dedup statement is scoped "within the included records"; the 59 excluded rows carry no dedup statement. No automation/classification or screening tool was used (tool names Rayyan/ASyA/Abstrackr/Gatherer/Zotero/EndNote/Mendeley: zero hits; "removed by automation" phrasing: zero hits).

Screening stages. Title/abstract-stage decisions are recorded (59 EXCLUDED / 46 INCLUDED), but **the stage of adjudication was not recorded in the source** for any row; for the 59 excluded records the title/abstract stage is an *inference* (no full text on file), flagged as such in the register itself ("stage inferred, see Notes"). We report: 105 records screened, 59 excluded at (inferred) title/abstract stage, 46 advancing.

Reviewers. Every register row states "reviewer identity NOT RECORDED; screening date NOT RECORDED." Single- vs. multiple-reviewer arrangements, consensus, and any inter-rater agreement measurement are **undocumented**; we report no reviewer counts and no κ statistics (drafting instructions explicitly prohibit inventing them). **(Appendix E, U12; §7.2.)**

Full-text assessment and eligibility. 46 reports were sought and all 46 have documented full-text consultation (reconstructed, then verified at file level: 10 publication originals + 36 per-record verification documents containing page-level citations; 28/28 studies have full-text evidence; none missing). At full-text stage, **0 records were excluded**: 42 received FINAL INCLUDED and 4 were RETAINED with flags under DP-4A (grey zone). Study layer: 25 FINAL INCLUDED + 3 RETAINED = 28. This 0 is a statement about this register only; it supports **no inference** about criteria strictness.

Records → reports → studies (three-layer counting rules, applied throughout):

Table 2. The three counting layers applied throughout this review, with their verified values and the evidence for each. Conflating the layers is the most common source of error in the underlying project records: the report count is not the number of files on disk, and the study count is not the number of LLM studies.

Layer	Count	Rule and evidence
record	**46**	Register rows kept; includes cross-RQ re-registrations
report	**29**	28 studies each 1 report + S26 (LLMLOOP) conference-and-preprint version pair documented as one study with two reports; S06's two DOI spellings normalised to one report; NOT the 46 PDFs on disk, because 36 are derived verification documents (8 PDF-producer signature groups; 22/22 recorded original filenames absent from disk; a shared embedded-font-program MD5 links the derived set to a known verification document)
study	**28**	S01–S28 in the study-level determinations; 46 − 18 study-level collapses

Cohort demarcation (author-accepted defaults, 2026-10-12) and denominators:

Table 3. Cohort demarcation under the author-accepted decision defaults of 2026-10-12, with study and record counts, the basis for each, and the cohort's role in the synthesis. Cohorts are demarcated rather than merged so that no verified evidence is discarded while no non-LLM study can inflate an LLM statistic; LLM-population statements use N = 13, never 28.

Cohort	Studies	Records	Basis	Synthesis role
LLM-construct (incl. 1 grey-literature thesis, DP-5A descriptive-only)	**13** (S01–S07, S09, S10, S13, S14, S17, S28)	28	IC-5 met	Population for all LLM claims — **denominator 13, never 28**
Contextual non-LLM infrastructure	**8** (S15, S16, S18–S23)	8	DP-1A	Environment framing for RQ4–RQ6 only; excluded from LLM population statistics
Comparator general-SE	**4** (S24–S27)	6	DP-3A	Calibration for RQ7/RQ8; not counted as embedded-C findings
Grey zone (indirect relevance; incl. S12 whose identity is unresolved)	**3** (S08, S11, S12)	4	DP-4A	Evidence retained verbatim with flags
Σ	**28**	**46**		

Arithmetic audit: 16 reconciliation relations were computed from the logs; 15 close exactly; the one non-closing relation (46 assessed − 0 excluded ≠ 28 included) is a **unit mismatch by design** (records vs studies, the 18 being study-level collapses), not an error. Two further documented conflicts remain open: (i) the REC-002↔REC-011 "byte-identical" note is mis-aimed (REC-011 is a derived document; the true identical pair is the RQ_1/RQ_2 copies of EmbC-Test) — wording fix pending, counts unaffected; (ii) the cross-RQ synthesis self-reports "15" zero-overlap pairs where its own matrix yields 14 (14 strict zeros, 12 pairs of 1–3, 2 pairs ≥4 of 28 pairs) — **(not recoverable from the project records; Appendix E.)**.

PRISMA identification-stage boxes are reported with status labels ("0 records attributed", NOT APPLICABLE, "105 registered; 46 unattributed") rather than as retrieval yields; the diagram is fillable in format, and we say so precisely — full 12-box transparency while stating that two identification boxes rest on PARTIAL evidence.

3.7 Quality assessment

No quality assessment was applied. The project contains no scoring of any included study: no QA column exists in the register or the determinations table, and no scored rubric output exists anywhere in the project. What the records contain is a **draft rubric held as a proposal only** (dimensions QA1–QA7: objective stated; LLM/approach stated; embedded context stated; test procedure stated; execution/validation setup stated; metrics reported; reproducibility detail), generated under an explicit instruction that "if no approved QA criteria exist, propose a draft rubric and label it as a proposal rather than an already-used method." We report it as such and have not used it to include, exclude, weight, or qualify any study.

Two study-specific treatment facts substitute for quality gating where they matter: the grey-literature thesis (S04) is admitted with a descriptive-only restriction (DP-5A); and RQ8's own synthesis prohibits popularity claims because of its denominator (4 studies) and 9 recorded intra-dataset count conflicts (D-3) — a data-quality judgement embedded in the synthesis layer rather than in a QA instrument. If the authors ran any quality appraisal not reflected in these files, it must be documented here **(not recoverable from the project records; Appendix E.)**; otherwise the honest statement is that reporting completeness was handled per-RQ through "Not reported / Unclear / No" distinctions (below) rather than through a scored QA.

3.8 Data extraction

Extraction was organised **per RQ** rather than through one cross-RQ form. Each RQ dataset stores study-level rows plus **configuration-level records** (an experimental setting is not a study): RQ1 4 studies; RQ2 7 studies / 51 configuration rows; RQ3 8 / 32 configuration IDs; RQ4 6 / 27; RQ5 6 / 24 verified combinations; RQ6 6 / 11 environment configurations; RQ7 5 / 25 feedback-mechanism records; RQ8 4 studies / 5 documents / 46 metric-level records; RQ9 (derived) 6 studies / 79 extracted items → 4 rejected in verification → 75 retained.

Extraction categories follow the RQs: test types (RQ1); models and prompt strategies with explicit zero-shot / few-shot / RAG / fine-tuning / agentic distinctions (RQ2); hardware/software documentation *provided to the model* versus merely discussed (RQ3); dependency-handling artefacts with explicit mocks-vs-simulation-vs-physical boundaries (RQ4); interrupt handling versus generic

asynchronous behaviour versus timing constraints (RQ5); execution environment versus metric versus feedback loop (RQ6–RQ7, kept orthogonal); metrics/validation methods (RQ8); and author-acknowledged limitations with L-codes, verbatim quotes, page/section locators, and verification rulings (RQ9).

Verification was **record-level and adversarial**: every RQ dataset passed a strict verification pass whose artefacts are the 36 per-record "FINAL VERIFIED EXTRACTION / STRICT VERIFICATION AND SYNTHESIS" documents, each carrying Previous-Classification → Verified-Classification tables with evidence quotations, page/section locators, and reason-for-change. Standing extraction rules (recorded in the verification prompts): never silently change a verified value; never invent page numbers or quotations; keep "Not reported", "Unclear" and "No" distinct (never map a silence to a negative); do not classify background mentions as the study's own method; do not derive historical counts from files currently on disk.

Extraction provenance limitation (disclosed): for 18 of 28 studies (S11–S28) the publication original is not stored in the project; their extraction evidence is the page-cited verification documents rather than the originals themselves. The remaining 10 (S01–S10) are evidenced by publication originals (4 arXiv-family PDFs in RQ_1, the AutoHIL ACM PDF, the TUHH thesis, and 5 arXiv-family PDFs in RQ_2). Who performed the extraction, and whether extraction was performed independently more than once, is **not documented** — the register records "NOT RECORDED" rather than a second extractor **(not recoverable from the project records; Appendix E.)**.

Study-counting rules used everywhere: percentages are computed only within an RQ's own study-level denominator and only over that RQ's own cohort (13 for LLM-population statements; 6 for RQ5/RQ6 environment statements; 4 studies for RQ8 with the 5-document/4-study discrepancy self-disclosed in RQ8 §10 with both n-versions shown; 6 studies for RQ9, never its 75 instance count — ADFEmu contributes 13/75 = 17.3% of instances but counts once per category). Cross-RQ overlap counts are computed at study layer after identity matching that was itself documented as a synthesis act, not a source-report claim.

3.9 Data synthesis and analysis

Synthesis is **structured qualitative aggregation with exact counts; no meta-analysis and no effect-size pooling**, a conclusion the source reports reach independently: RQ8 records zero shared benchmarks, zero shared baselines, zero identical metrics, zero identical evaluation environments, and six-dimensional systematic heterogeneity, ruling out comparability; RQ1 records mutually incomparable execution accounting (four different "executable → compilable → executed → passed" bookkeeping regimes) and states that quantitative meta-analysis is impossible. No study reports a complete funnel.

The documented analysis sequence: (1) per-RQ syntheses with denominators declared in §1.1 of each report; (2) a cross-RQ synthesis operating on the 28-study set: a 28-pair overlap matrix (14 strict-zero, 12 low, 2 high pairs per the matrix; see open item in §3.6), a two-cluster structure (an LLM-test-generation cluster around S06/S09/S10/S17 and a firmware-verification infrastructure cluster S15–S23) whose cross-cluster relations route through RQ4's dependency-handling construct and the unresolved M2 bridge hypothesis (S13 ≟ "Abtahi & Azim", **(not recoverable from the project records; Appendix E.)** — if false, all RQ4×RQ5/RQ6/RQ7 cross-dimension links collapse to zero); (3) pattern synthesis (numbered patterns with evidence-type grading "single-dataset exhaustive / cross-dataset convergent"); (4) RQ9 gap analysis as an **evidence matrix** (G1–G16 with per-cell study attribution; U1–U6 uncertainty register; D-1…D-11 documented intra-dataset conflicts left unresolved rather than silently reconciled; V1–V8 validity items) — deliberately not frequency charts, because zero counts here license "not reported in this dataset", never absence from literature; (5) visualisation and PRISMA reporting generated exclusively from the audited counts.

Headline construct-validity constraint inherited by all findings: the verified corpus contains **no study that simultaneously satisfies** LLM-based + test-generation + embedded-C + hardware-in-the-loop validation; findings are therefore per-dimension claims about partial coverage, and no combination claim is treated as established.

3.10 Use of AI assistance

Documented, verifiable uses of AI tools in this project:

1. **Identification (recorded):** one DeepSeek AI-tool session on **2026-10-12** produced the 59-record excluded export used as the discovery route for those records (§3.2). The export retained no query and no result count. No other AI identification event is

recorded.

2. **Process documentation and auditing (recorded):** the 30-prompt working library structured the audit chain (dataset audit → PRISMA recalculation → consistency arithmetic → figure/PRISMA generation → section drafting) executed on 2026-10-12; per-record verification documents (created 2026-10-08/09, all 36 by the same "Print To PDF" generation chain) carry the verification tables described in §3.8. **Which AI tool(s), if any, generated or transcribed those verification documents is not recorded (not recoverable from the project records; Appendix E.)** — the files evidence structure and dates, not authorship.
3. **Not documented — and therefore not claimed:** AI involvement in eligibility screening decisions, in extraction beyond what item 2 evidences, or as an independent second reviewer. The register's reviewer column is NOT RECORDED for all rows (§3.6), which equally bars attributing screening to AI tools.
4. **Confusion guard:** AI product names (gpt-5, gemini-2.5-pro, deepseek-r1, qwen3-32B, etc.) appear 60 times across project texts, but 57 of those mentions are **models evaluated inside the reviewed studies** (e.g., AutoHIL's evaluation table) and 0 constitute discovery-route evidence. Reviewed-study AI usage and review-process AI usage are kept strictly separate in this manuscript.
5. The authors remain responsible for all content; every figure, count and eligibility statement in this manuscript is traceable to the project records listed in Appendix A.

---

4. Results

All counts, percentages and denominators in this section are taken verbatim from the verified RQ1–RQ9 extraction syntheses and from the screening register and study-level determination records; every figure cited here was generated exclusively from those audited values — Figures 2–15 by the audited figure pipeline, whose plates and source tables are listed in Appendix F.3, and Figure 1 by the PRISMA count audit, each of its boxes carrying its own evidence status, and prose, tables and figures agree. Percentages are valid only inside their own denominator (Table 4); categories overlap wherever a study can exhibit several properties, so sums are not expected to reach 100%. "Not reported", "Unclear" and "No" are kept distinct throughout: a zero count in this section means *not reported within the verified dataset*, and licenses no claim that the phenomenon is absent from the wider literature. Findings carry no causal explanation and no recommendation here; interpretation is reserved for the Discussion. Section 4.11 consolidates all data-quality notes and unresolved source conflicts in one place so that the subsections read cleanly.

4.0 Study selection and characteristics of the included corpus

Study selection is reported per PRISMA 2020; identification-stage quantities were not documented by the source process and are reported with their evidence status rather than estimated (Figure 1; Figure 15; Methodology §3.2–3.6). **Records screened: 105. Records excluded: 59** (56.2% of registered records; reasons in Figure 15: Non-embedded unit test gen 8, Blockchain/Smart contract 5, Education 5, Hardware verification 5, Mobile app development 5, Non-embedded fuzzing 5, Testing LLMs themselves 5, Web development 5, Blockchain consensus verification 4, Database/Text-to-SQL 4, Enterprise code review / IaC 4, Game development 4). **Records included: 46** (43.8%). Reports sought for retrieval: 46; not retrieved: 0; records excluded at full-text stage: 0. Duplicate records removed: 0 (cross-question re-loggings were annotated, not deleted). **46 records collapse to 28 unique studies** (18 study-level collapses) and correspond to **29 reports** (one documented conference-plus-preprint version pair).

Characteristics of the 28 studies (Figure 2; Figure 3): stated publication years at study level are 2020 (3), 2022 (1), 2023 (1), 2024 (2), 2025 (8), 2026 (4); **Not recorded (9), 32.1% of studies** — kept in a separate category, never imputed. Ten studies are held with publication originals; for 18 studies the extraction evidence is per-record verification documents rather than publisher PDFs (note N3, §4.11). Cohort composition (Figure 3): LLM-construct 13 (46.4%), Contextual non-LLM 8 (28.6%), General-SE comparator 4 (14.3%), Gray-zone 3 (10.7%). LLM-population statements in this section use **N = 13**, never 28. Eighteen of the 28 studies appear in exactly one RQ dataset; ten appear in multiple RQ datasets, which is what makes the overlap analysis of Figure 13 necessary. One

study (S04) is a master's thesis admitted under the grey-literature decision with a descriptive-only restriction; one grey-zone study (S12) has an identity unresolved in the records.

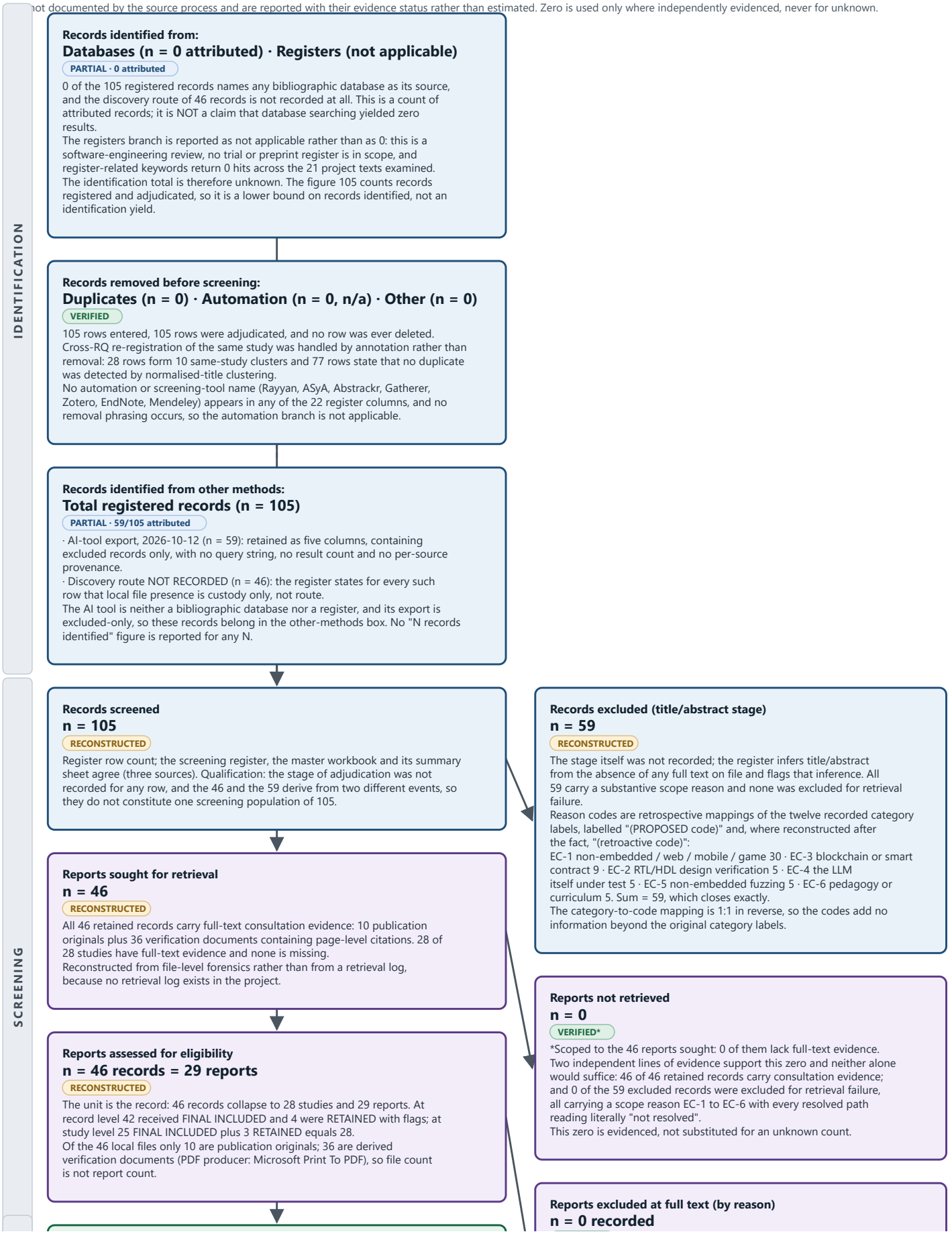
Table 4. Denominator registry and cohort composition of each verified dataset. (Source: Study-Level Final Determinations + Screening Register; matches the RQ final-answer headers. S02 is logged jointly for RQ1 and RQ2 and is counted in both.)

Dataset	N (studies)	Study IDs	LLM-construct	Contextual	Comparator	Gray-zone
RQ1	4	S01, S02, S03, S04	4	0	0	0
RQ2	7	S02, S05, S06, S07, S08, S09, S10	6	0	0	1
RQ3	8	S02, S05, S06, S07, S08, S09, S11, S12	5	0	0	3
RQ4	6	S05, S06, S09, S10, S13, S14	6	0	0	0
RQ5	6	S13, S15, S16, S17, S18, S19	2	4	0	0
RQ6	6	S13, S17, S20, S21, S22, S23	2	4	0	0
RQ7	5	S13, S17, S24, S25, S26	2	0	3	0
RQ8	4	S17, S26, S27, S28	2	0	2	0
RQ9 (derived)	6	ADFEmu (=S17); Abtahi & Azim (=S13); Asmita et al.; RQ9-P4 (survey); RQ9-P5 (HardSecBench); RQ9-P6 (P2IM lineage)	—	—	—	—

Sum of per-RQ study counts (4+7+8+6+6+6+5+4 = 46) equals the record count by construction; the union is 28 studies. RQ9's denominator is 6 studies at the *study* level; its limitation-instance counts (75) are never used as prevalence.

Figure 1. PRISMA 2020 flow diagram (template 2: databases/registers + other methods)

Reported in the FORMAT of PRISMA 2020; this is not a claim of PRISMA 2020 compliance, and the completeness of this diagram demonstrates reporting transparency only — it does not establish the soundness of the search, screening or synthesis. Counting units are kept distinct throughout: record (46) ≠ report (29) ≠ study (28). Identification-stage quantities not documented by the source process and are reported with their evidence status rather than estimated. Zero is used only where independently evidenced, never for unknown.



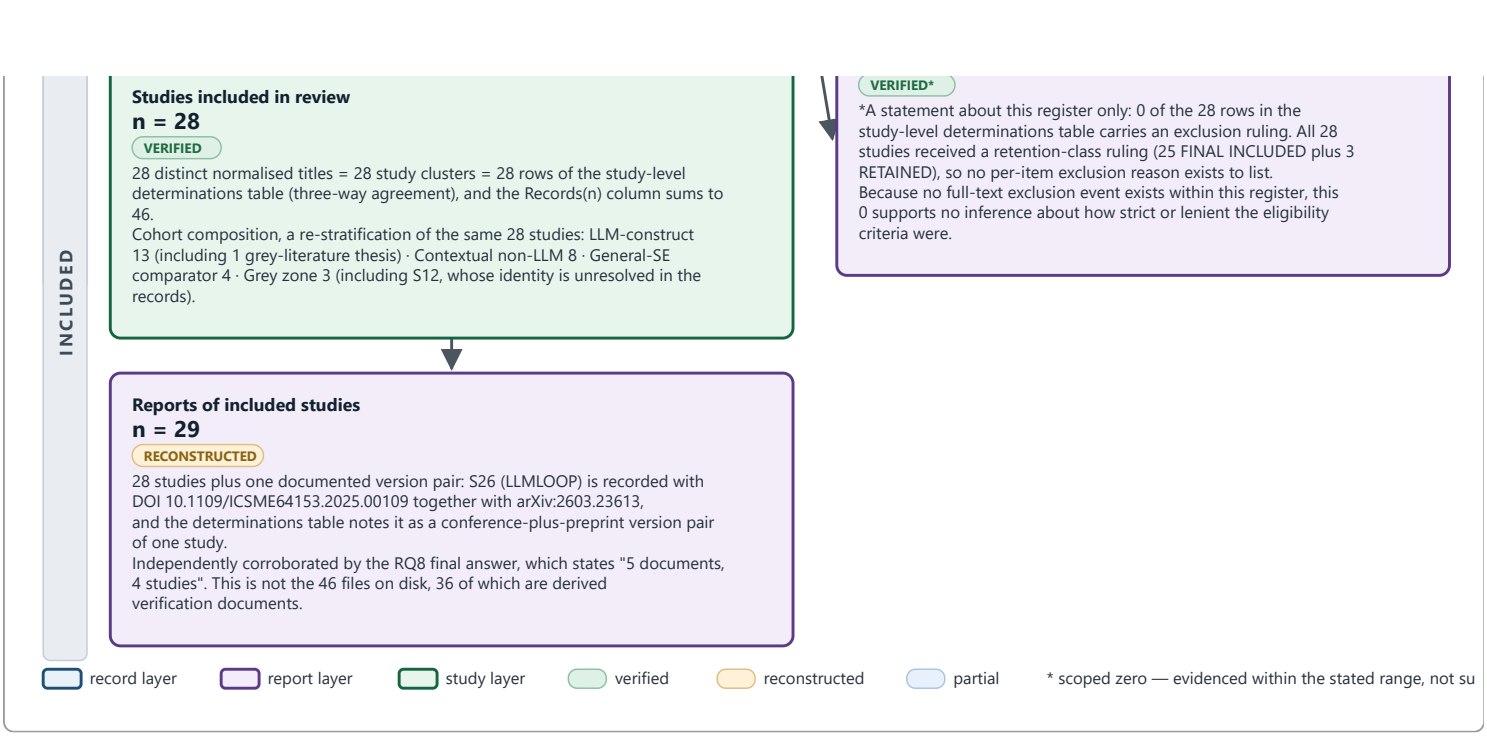


Figure 1. PRISMA 2020 flow diagram (template 2: databases/registers + other methods). Reported in the format of PRISMA 2020; this is not a claim of PRISMA 2020 compliance, and the completeness of the diagram demonstrates reporting transparency only. Each box carries its evidence status — verified, reconstructed or partial — and zero is used only where independently evidenced, never in place of an unknown count. The registers branch is reported as not applicable rather than as 0. Counting units are kept distinct throughout: 46 records ≠ 29 reports ≠ 28 studies. Identification-stage quantities were not documented by the source process. Source: PRISMA count audit, box values 12/12; see Appendix A.

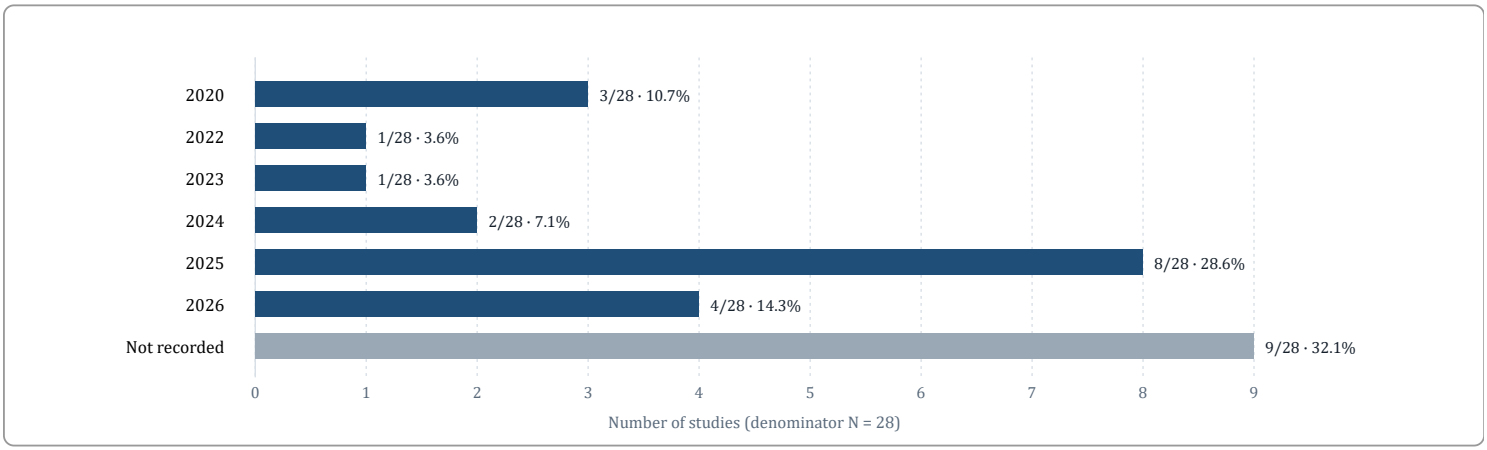


Figure 2. Publication years of the 28 included studies, at study level. A study is year-stated only if at least one of its records states a four-digit year (rule N8, §4.11), so a parenthetical mention such as "(references 2026)" is recorded as not stated. Missing years are never imputed: 9 of 28 studies (32.1%) have no recorded year and form their own category. S26 is plotted at its peer-reviewed venue year (2025) with its 2026 preprint member disclosed rather than dropped — the single adjudication applied. Source: `fig01_publication_year_study_level.csv` .

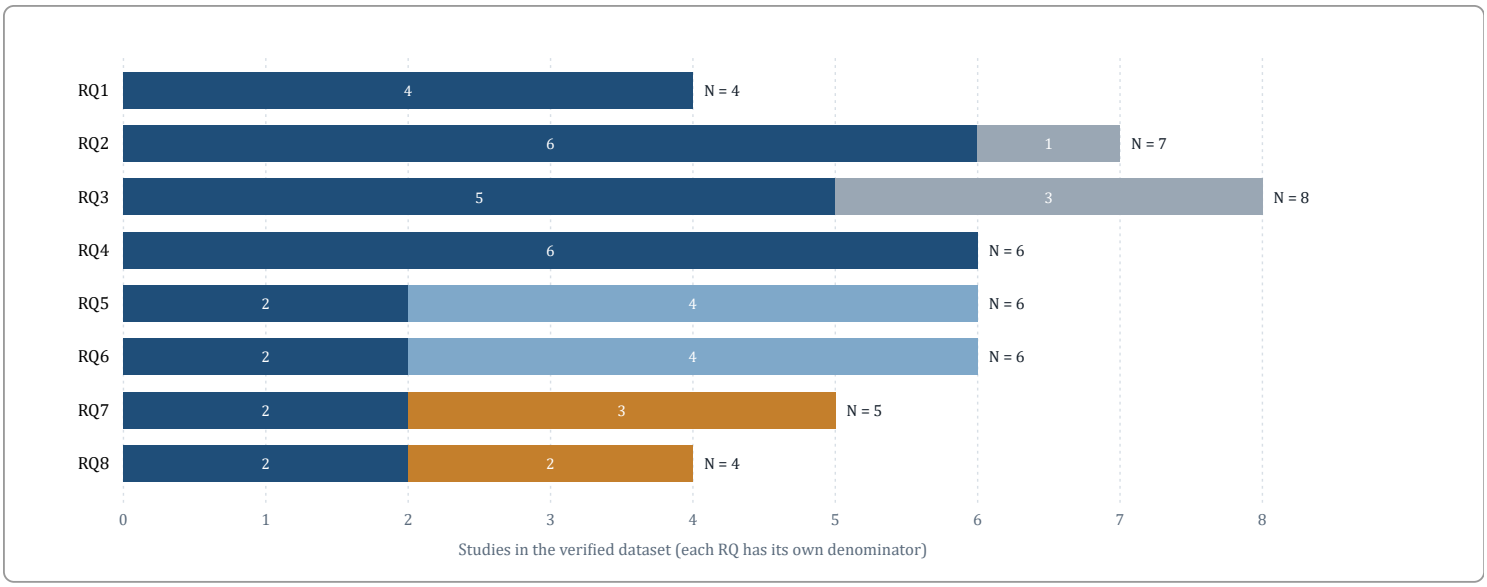


Figure 3. Scale and cohort composition of each verified RQ dataset. Bars give the study-level denominator per research question (RQ1 4, RQ2 7, RQ3 8, RQ4 6, RQ5 6, RQ6 6, RQ7 5, RQ8 4; RQ9 derived over 6 studies); segments give cohort membership within each dataset. S02 is logged jointly for RQ1 and RQ2 and is counted in both, which is why the approved 4/7 counts exceed a naive dataset split of 3/6. Every percentage in this manuscript is valid only inside the denominator shown here (Table 4). Source: `fig02_rq_dataset_scale.csv` .

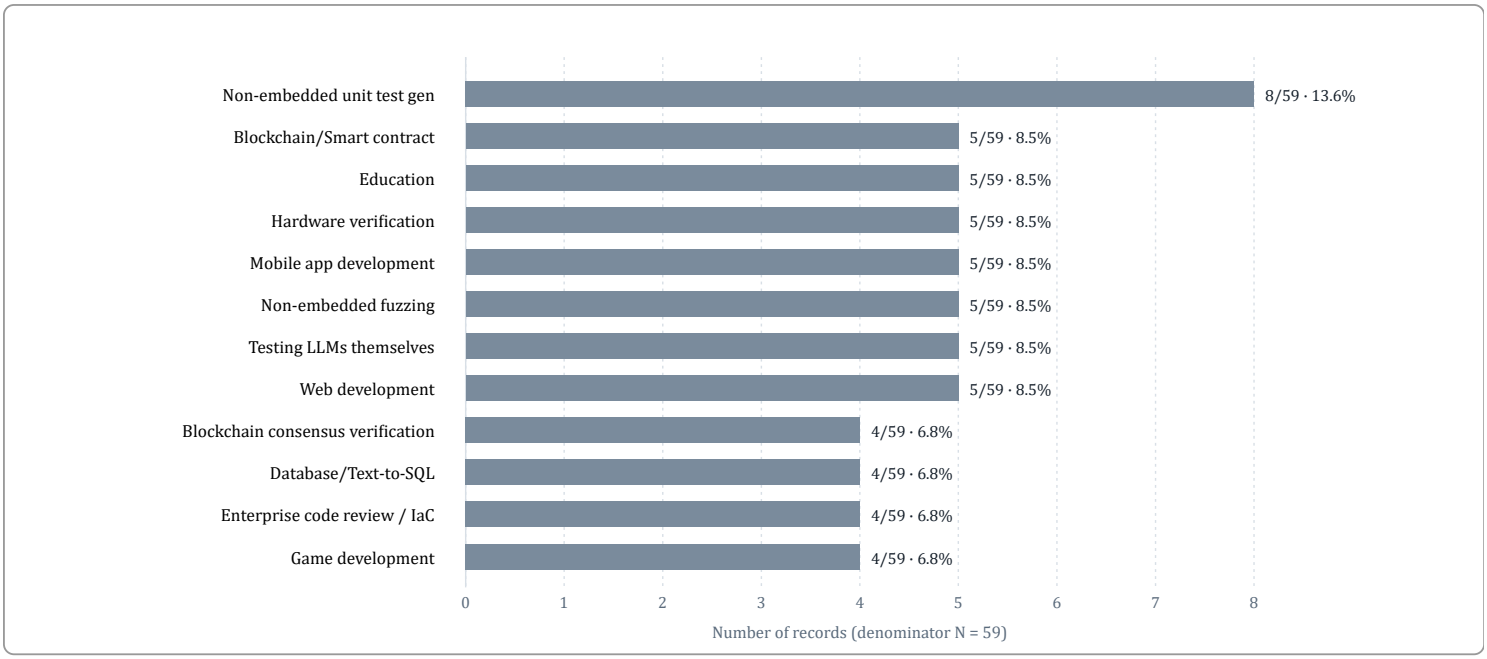


Figure 15. Screening-side exclusion reasons for the 59 excluded records, by the twelve recorded category labels. Provided as screening context only: these are categories of records excluded before eligibility, **not** PRISMA full-text exclusion reasons, and the full-text exclusion count is 0 (Figure 1). The categories map 1:1 in reverse onto codes EC-1 to EC-6 (Table 1), so they carry no information beyond the category labels. Σ = 59. Source: screening register category labels.

4.1 RQ1 — What types of tests for embedded C software (unit, integration, hardware-in-the-loop, regression) are generated or supported by LLM-based approaches?

Direct answer. In the RQ1 dataset (N = 4 studies; Figure 4), reported test types are strictly bimodal: **Unit testing 2/4 (50.0%)** and **HIL / system-level functional testing 2/4 (50.0%)**; **Integration testing 0/4** and **Regression testing 0/4**. Across all four canonical types, the "supported" and "generated-and-supported" cells are 0/4 — every reported type is of the *generated* kind. Only **1/4 (25.0%)** of the studies produces test code in C (study tran-mt25 = S04); the remaining three emit Python or JSON artefacts. All 4/4

studies report some execution of generated tests, but with four mutually incompatible execution-accounting regimes (executable-subset filtering; manual clean-up before counting; exemplary execution; aggregate validation rate), so cross-study test-type frequencies are descriptive only.

Exceptions and not-reported cases. Integration testing's 0/4 is an *evidence void*: two studies explicitly exclude it from scope, one is Unclear, one Not reported. Regression testing's 0/4 is *total silence*: three studies do not report it (two confirmed by full-text search of the term) and none states an explicit "No". **Overlaps.** A study may count in more than one test type; here the four studies split cleanly 2+2 with no study contributing to both peaks. **Limitations.** At N = 4, one study moves any percentage by 25 points; no proportional difference carries inferential meaning (source self-qualification). Attributing specific studies to the two 2/4 peaks is not supported record-by-record and is therefore not done here.

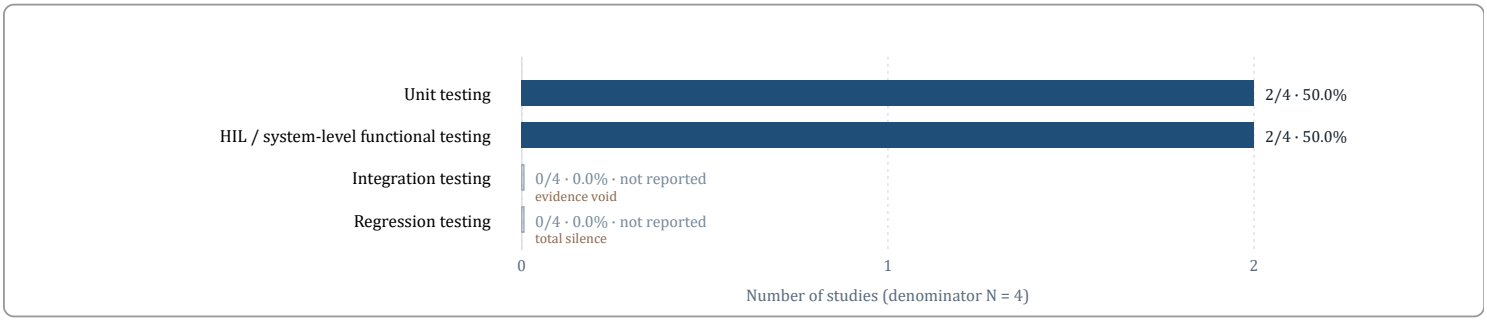


Figure 4. RQ1 — test types generated or supported (N = 4 studies). Unit testing 2/4 (50.0%) and HIL / system-level functional testing 2/4 (50.0%); integration 0/4 and regression 0/4. The two zeros differ in kind and are kept distinct: integration is an evidence void (two explicit scope exclusions, one Unclear, one Not reported) whereas regression is total silence (three Not reported, no explicit "No"). Only 1/4 studies emits C-language test code. Zero cells are hatched. At N = 4 one study moves any percentage by 25 points. Source: `fig03_rq1_counts.csv` .

4.2 RQ2 — Which LLMs and which prompting or adaptation strategies (zero/few-shot, RAG, fine-tuning, agentic workflows) are used for embedded test generation?

Direct answer. In the RQ2 dataset (N = 7 studies, S02 and S05–S10; 51 coded configuration rows; Figure 5), reported strategies are: **Zero-shot prompting 6/7 (85.7%)**; **RAG 5/7 (71.4%)**; **Agentic workflow 4/7 (57.1%)**; **Other prompting techniques 3/7 (42.9%)**; **Fine-tuning, any component 2/7 (28.6%)**, of which **fine-tuning, generation LLM only 1/7 (14.3%)**; **Few-shot 1/7 (14.3%)**; **Chain-of-thought (explicit technique) 0/7 configurations**. The LLM role is **test generation 2/7 (28.6%)**, **test support 1/7 (14.3%)**; **Multi-model studies 7/7 (100%)**. **No model is used by more than 2 studies**, which is why no model-popularity chart is reported (omitted by design).

Exceptions and not-reported cases. The Few-shot count is extractor-inferred — the underlying author text contains zero literal mentions of zero-/few-shot wording for the study concerned. Cross-dataset coding conflict (source conflict D-1, kept unmerged): RQ2 codes RAG at 5/7 while RQ3 codes RAG at 3/8; each value holds only in its own denominator. Reporting completeness: **LLM identity Not reported in ≥1 configuration 5/7 (71.4%)** and **Prompting Not reported in ≥1 configuration 5/7 (71.4%)**. **Overlaps.** Strategies co-occur within studies; percentages deliberately do not sum to 100. **Limitations.** At N = 7, one study moves any percentage by 14.3 points (source self-qualification). "Fine-tuned generation LLM + agentic = 0 studies" holds for that pairing only; an embedding-component fine-tune combined with RAG and agentic workflow appears in 1 study (S07).

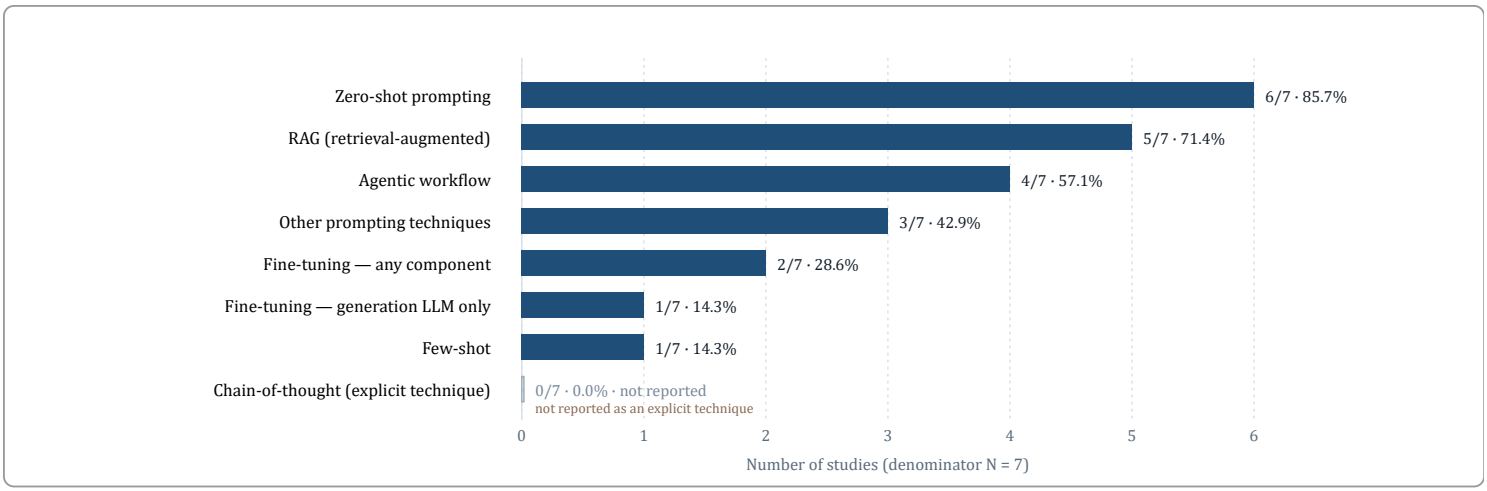


Figure 5. RQ2 — prompting and adaptation strategies (N = 7 studies, 51 configuration rows). Zero-shot 6/7 (85.7%), RAG 5/7 (71.4%), agentic workflow 4/7 (57.1%), other prompting techniques 3/7, fine-tuning of any component 2/7 with generation-LLM fine-tuning 1/7 (14.3%), few-shot 1/7, and explicit chain-of-thought in 0 configurations. Strategies co-occur within studies, so the bars overlap and deliberately do not sum to 100%. No model is used by more than two studies, which is why no model-popularity chart is reported. LLM identity and the prompting description are each unreported in at least one configuration for 5/7 studies (71.4%). Source: `fig04_rq2_counts.csv` .

4.3 RQ3 — What hardware/software context (datasheets, register maps, HAL/driver code, specifications) is provided to LLMs for test generation?

Direct answer. In the RQ3 dataset (N = 8 studies, S02, S05–S09, S11, S12; 32 configuration IDs; Figure 6), **Software context (code / repo artefacts) 8/8 (100%)** is universal. Embedded-hardware context is scarce and shallow: **Any embedded hardware context 5/8 (62.5%)**; **Execution feedback used as context 4/8 (50.0%)** (compiler errors, build/flash logs, coverage, HIL action results — which the source describes as substituting for hardware awareness); **Low-level hardware documentation 3/8 (37.5%)**; **Datasheet at inference time 2/8 (25.0%)**; **Peripheral specification 1/8 (12.5%)**, confirmed only for EmbedAgent (S06). The categories **Register map**, **MCU / processor specification**, **Clock / memory-map / timer / DMA / interrupt specs**, **Board / electrical-interface / sensor-actuator / protocol docs**, and **HAL / driver / BSP / RTOS at inference time** are each **0/8**; register-related strings appear in 2/8 studies only as code-symbol types or corpus categories, not as supplied context (S11, S08). Among the studies whose LLM role is genuinely test generation, hardware context is confirmed **0/3 (0%)**. Retrieval combinations: datasheet+RAG 0; register map+HAL 0; HAL+driver+peripheral specification 0.

Exceptions and not-reported cases. Three studies carry **no hardware-context record at all** despite working on code that contains hardware dependencies; the source therefore words its finding as "no evidence that context was provided", not "confirmed not provided" (self-qualification retained). **Overlaps.** Context categories co-occur across the 32 configurations. **Limitations.** At N = 8 one study = 12.5 points; five of the eight studies are code-generation, benchmark, traceability or pre-training works rather than test generation, which narrows the direct evidence base.

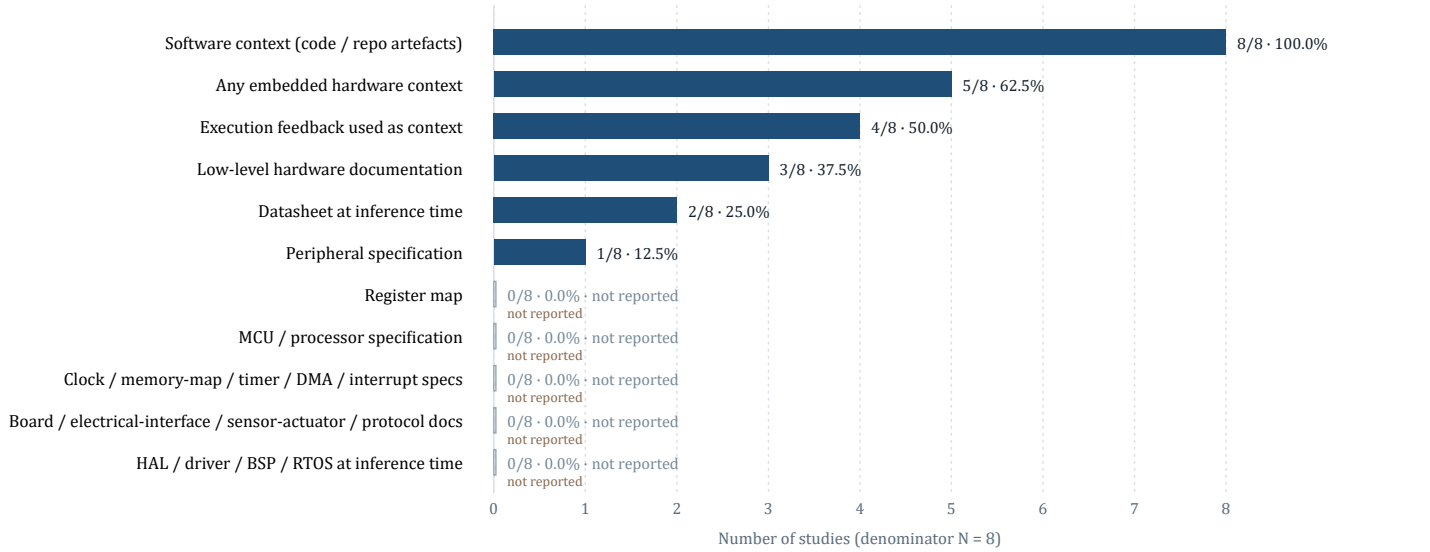


Figure 6. RQ3 — hardware and software context provided to the model (N = 8 studies, 32 configuration identifiers). Software context 8/8 (100%); any embedded-hardware context 5/8; execution feedback used as context 4/8; low-level hardware documentation 3/8; datasheet at inference time 2/8; peripheral specification 1/8. Register map, MCU/processor specification, clock, memory-map, timer, DMA and interrupt specifications, board and electrical-interface documentation, and inference-time HAL/driver/BSP/RTOS material are each 0/8. Only material *provided to the model* counts; material merely discussed in the paper does not. Source: `fig05_rq3_counts.csv` .

4.4 RQ4 — How do LLM-based approaches handle hardware and peripheral dependencies (mocks, stubs, fakes, simulators)?

Direct answer. In the RQ4 dataset (N = 6 studies, S05, S06, S09, S10, S13, S14; 27 configurations; Figure 7 and Figure 12), handling divides into two disjoint families — **Provision family (create substitutes / doubles) 4/6 (66.7%)** and **Replacement family (swap the execution substrate) 2/6 (33.3%)** — with **Hybrid approaches 0/6**. In **5/6 studies (83.3%)** the LLM produces **no dependency-handling artefact**; **19/27 configurations (70.4%)** instead have the LLM generate hardware-related *production* code. The artefact × status matrix (Figure 12) shows exactly one unambiguous "Yes" cell: LLM-generated tests + stubs (1 study; S05 openSIL, configurations C2–C4, host execution). Tests + mocks is coded **Partially / Not established** and deliberately carries no count. Every remaining artefact class — LLM-generated tests with simulators, emulators, HIL or real hardware; LLM-built simulator / emulator models, peripheral models, device drivers, HAL, wrappers / adapters / link-time replacements, configured execution environment; and **behaviourally faithful test doubles** with register / timing / interrupt / protocol semantics — is **No, 0 studies** (0.0% of the 27 configurations).

Exceptions and not-reported cases. Specific peripheral *types* are entirely unreported in 3/6 studies (S05, S13, S14); SPI, Ethernet, DMA, interrupt controllers, Flash/EEPROM, device drivers and HAL APIs are each 0/6 as assessed dependencies. One HIL label (S09/EmbedGenius) is ambiguous (source conflict D-9; flagged in Figure 7, not merged). **Overlaps.** The two families are mutually exclusive here (hybrid 0). **Limitations.** At N = 6 one study = 16.7 points; quantitative simulator/emulator fidelity assessment is 0/27, so "handled" never means "validated as faithful".

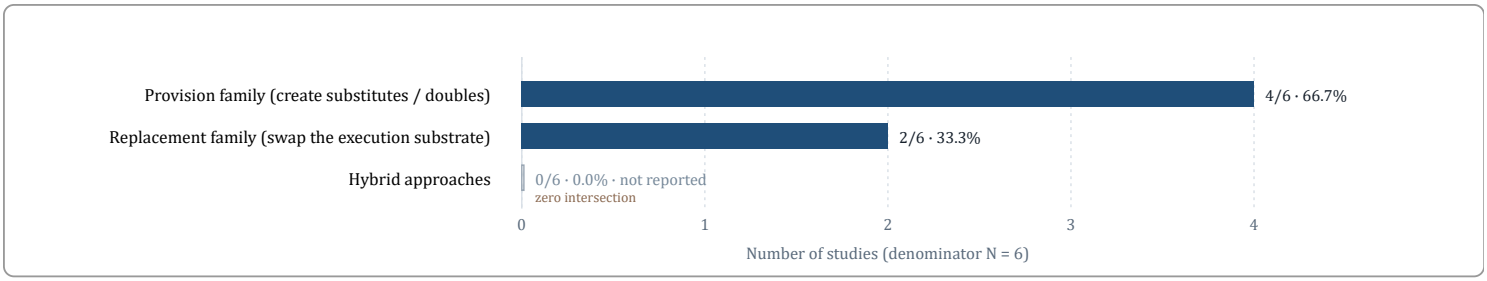


Figure 7. RQ4 — dependency-handling families (N = 6 studies, 27 configurations). The Provision family (create substitutes or doubles) covers 4/6 (66.7%) and the Replacement family (swap the execution substrate) 2/6 (33.3%); the two share no study, so Hybrid is 0/6. In 5/6 studies (83.3%) the LLM produces no dependency-handling artefact at all, while 19/27 configurations (70.4%) have it generate hardware-related *production* code. One HIL label (S09) is ambiguous (conflict D-9) and is flagged rather than merged. Source: `fig06_rq4_counts.csv` .

Figure 12. RQ4 — LLM-generated artefact × verification status (N = 6 studies, 27 configurations)

A matrix, not a bar chart: the cells carry categorical verification status, so a bar would misread "Partially / Not established" as a count. That cell deliberately carries no number. Quantitative fidelity assessment is 0/27, so "handled" never means "validated as faithful".

LLM-GENERATED ARTEFACT CLASS	VERIFICATION STATUS	STUDIES / SOURCE NOTE
tests + stubs	Yes	1 R4_p1 C2/C3/C4, host execution —
tests + mocks	Partially / Not established	— (no count)
tests + simulator	No	0
tests + emulator	No	0
tests + HIL	No	0
tests + real hardware	No	0
simulator / emulator models	No	0
peripheral models / device drivers / HAL	No	0
wrappers / adapters / link-time replacements	No	0
configured execution environment	No	0
behaviourally faithful test doubles (register / timing / interrupt / protocol)	No	0 0.0% of the 27 configurations

Read as: of 6 studies and 27 configurations, the only unambiguous instance of an LLM-generated dependency-handling artefact is tests with stubs (1 study, S05 openSIL, configurations C2–C4, host execution). Behaviourally faithful test doubles — register, timing, interrupt or protocol semantics — appear in 0 of 27 configurations (0.0%). Source: fig11_rq4_artefact_matrix.csv.

Figure 12. RQ4 — LLM-generated artefact × verification-status matrix (N = 6 studies, 27 configurations). Rendered as a matrix rather than a bar chart because the cells carry categorical verification status, not frequency; a bar chart would misread "Partially / Not established" as a count. Exactly one cell is an unambiguous Yes: LLM-generated tests with stubs (1 study, S05, configurations C2–C4, host execution). Tests with mocks is coded Partially / Not established and deliberately carries no count. Every remaining class — tests with simulators, emulators, HIL or real hardware; LLM-built simulator or emulator models, peripheral models, device drivers, HAL, wrappers, adapters or link-time replacements; configured execution environments; and behaviourally faithful test doubles with register, timing, interrupt or protocol semantics — is No, 0 studies (0.0% of the 27 configurations). Quantitative fidelity assessment is likewise 0/27, so "handled" never means "validated as faithful". Source: `fig11_rq4_artefact_matrix.csv` .

4.5 RQ5 — How do existing approaches address interrupts, asynchronous behavior, and timing constraints?

Direct answer. In the RQ5 dataset (N = 6 studies, S13 and S15–S19; 24 verified study×mechanism combinations; Figure 8), **Asynchronous behaviour 6/6 (100%)** and **Timeout-related behaviour 6/6 (100%)** are universal. **Interrupts present 5/6 (83.3%)**; **Timing constraints tested 5/6** and **Virtual time 5/6**; **ISR handling 4/6 (66.7%)**, **Interrupt injection 4/6**, **NVIC simulation 4/6**, **Temporal assertions 4/6**, **Delay handling 4/6**; **Background tasks 3/6**, **Concurrent 3/6**; **Concurrency (general) 2/6**, **Scheduling — explicit control / modelling 2/6**, **Deterministic scheduling (via reproducibility) 2/6**, and **LLM involvement in interrupt / async / timing work 2/6** (S13, S17). Real-time properties are rare and concentrated: **WCET 1/6 (16.7%), Jitter 1/6, Deadline validation 1/6, Race-condition testing 1/6** — all four contributed by the *same single study* (RQ5-P6), whose own interrupt handling is Not reported; plus **Lock / mutex 1/6, Semaphore-based 1/6, Threads 1/6**.

Zero block (type I, exhaustive over the 6 studies). 0/6 each for: **LLM used to generate interrupt / async / timing tests**; **Nested interrupts tested**; **Interrupt priorities tested**; **RTOS scheduler modelling**; **RTOS tasks treated as an async mechanism**; **Thread interleaving**; **Task switching**; **Message queues**; **Event queues**; **Fake / mock clock**; **Deterministic simulation-time mechanism**; **Clock behaviour / clock domains**; **Firmware timeout as a functional constraint**; **Delay-duration validation**; **Periodic execution as its own mechanism**; **HIL**. Per correction D-10 these zeros must be read together with the non-zero concurrency counts above (Concurrency 2/6; one scheduling 2/6 counts input-delivery scheduling, not OS scheduling). The source records that all four interrupt-modelling studies obtain the interrupt model **from the artefact under test, not from documentation**. **Overlaps.** Mechanism categories overlap; row totals exceed 24 because studies contribute multiple combinations. **Limitations.** Four of the six studies are non-LLM infrastructure (Contextual cohort); at N = 6 one study = 16.7 points; no study combines interrupt modelling with real-time property measurement; NVIC-capability mentions in S16 and S19 were removed by verification as inference-based.

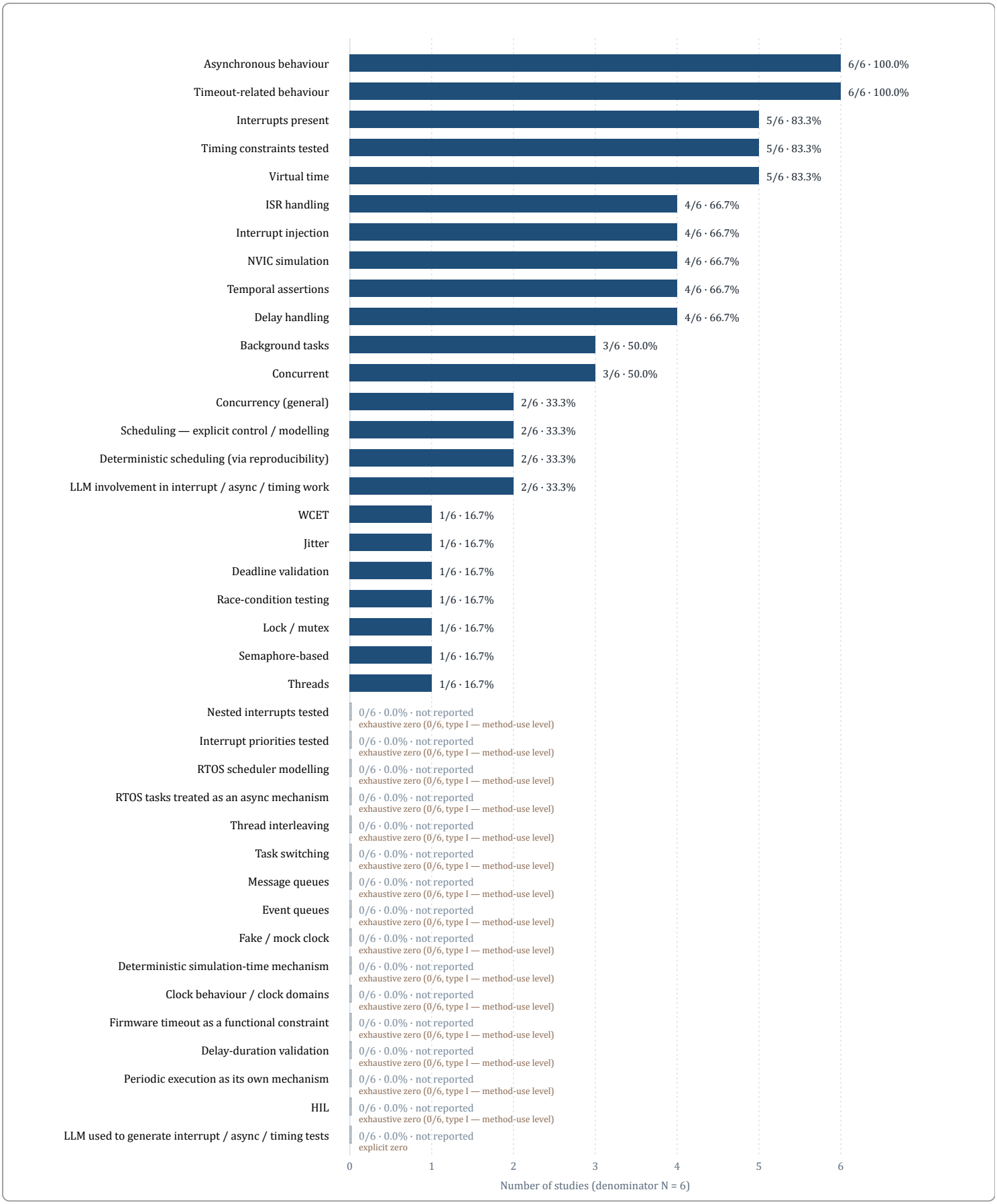


Figure 8. RQ5 — interrupts, asynchronous behaviour and timing (N = 6 studies, 24 verified study×mechanism combinations). Non-zero rows appear above the divider and the exhaustive type-I zero block below it, each 0/6. Per correction D-10 the two blocks must be read together: OS-level scheduling semantics are unmodelled, but concurrency itself is not unhandled (concurrency 2/6; explicit scheduling control 2/6, one of which is input-delivery scheduling rather than OS scheduling). Real-time properties — WCET, jitter, deadline validation and race-condition testing

— are each 1/6 and all four are contributed by the same single study. Mechanism categories overlap, so row totals exceed 24. Source: `fig07_rq5_counts.csv` .

4.6 RQ6 — In what environments (emulators, simulators, virtual platforms, real hardware, HIL) are generated tests executed and validated?

Direct answer. In the RQ6 dataset (N = 6 studies, S13, S17, S20–S23; 11 environment configurations; Figure 9), **Emulator 6/6 (100%)**, **Host / native 6/6**, **Hybrid 6/6** and **Software-only validation 6/6**: execution and validation are universally emulator-based and host-run. **QEMU (dominant engine) 5/6 (83.3%)** (S13, S17, S20, S21, S23). **Real hardware (secondary role only) 2/6 (33.3%)** — P2IM (S20, bug confirmation by re-execution) and HALucinator (S21, Avatar baseline comparison) — and **HIL (baseline only) 1/6 (16.7%)**. Zero categories: **Real hardware as PRIMARY execution environment 0/6**, mirroring the source sentence that no study executes its generated test suite primarily on physical hardware; **Simulator (strict term) 0/6**; **Virtual platform 0/6**; the **Other / Not reported** environment category is **0/6**. The emulator/simulator terminology is preserved as classified: QEMU, AFL-Unicorn, Unicorn Engine and the PIC32 emulator are recorded as emulators.

Exceptions and not-reported cases. For Abtahi & Azim (S13), real hardware is recorded as **"Not reported" — explicitly distinct from "No"** — with physical-platform validation appearing in that study's own future-work statement (quoted here as an observation, not a recommendation). Host machine is not reported for S13/S17/S21. **Overlaps.** Host, emulator and hybrid categories co-apply to all six studies by construction. **Limitations.** Only 2/6 studies are LLM-based (S13, S17); the other four are non-LLM firmware-testing baselines retained as environment context (DP-1A). A truncated RQ5 row reads "Simulator 6/6" and conflicts with RQ6's 0/6 (source conflict D-2); RQ6's authoritative classification table is used here and the counts are not merged.

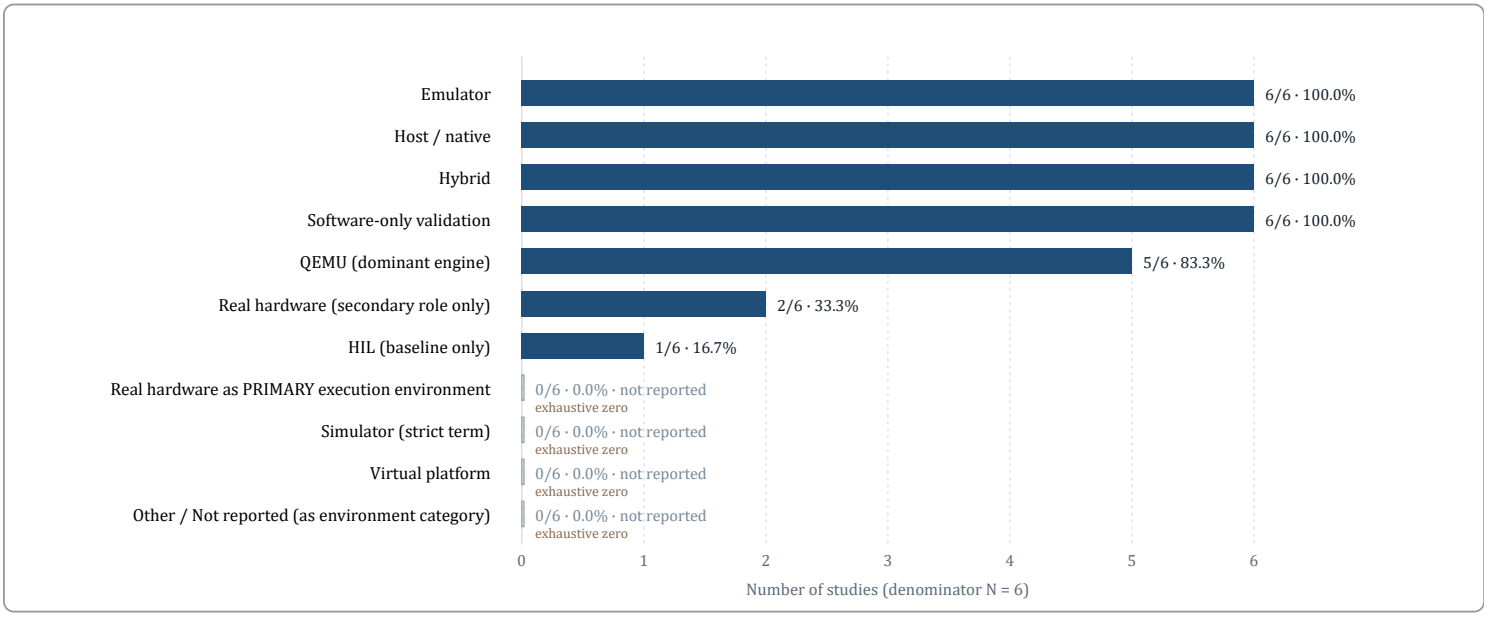


Figure 9. RQ6 — execution and validation environments (N = 6 studies, 11 environment configurations). Emulator 6/6, host/native 6/6, hybrid 6/6 and software-only validation 6/6; QEMU is the dominant engine in 5/6 (83.3%). Real hardware appears in 2/6 (33.3%) but only in secondary roles — bug confirmation by re-execution and baseline comparison — and HIL in 1/6 as a baseline only. Real hardware as the *primary* execution environment is 0/6, as are simulator (strict term), virtual platform and the Other/Not reported category. Only 2 of the 6 studies are LLM-based; the other four are non-LLM baselines retained as environment context under DP-1A. A truncated RQ5 row reads “Simulator 6/6” and conflicts with RQ6’s 0/6 (D-2); RQ6’s authoritative classification is used here and the counts are not merged. Source: `fig08_rq6_counts.csv` .

4.7 RQ7 — What feedback loops (compiler errors, coverage, runtime traces, execution results) are used to refine generated tests?

Direct answer. In the RQ7 dataset (N = 5 studies, S13, S17, S24–S26; 25 feedback-mechanism records; Figure 10), **Automatic closed loop 5/5 (100%)**; the loop-type categories **One-shot / manual / semi-automatic / mixed / unclear / not-reported loops** are each

0/5. Feedback sources: **Execution results 5/5 (100%)**; **LLM self-critique 5/5** (the *term* "self-critique" appears in none of the papers); **Test failures 4/5 (80.0%)**; **Compiler errors 3/5 (60.0%)** and **Coverage 3/5**; **Runtime traces 2/5 (40.0%)** and **Static analysis 2/5**; **Hardware / peripheral feedback 1/5 (20.0%)** — only ADFEmu's DMA input-channel emulation; **Mutation / fault information 1/5**; **Human feedback 1/5**. Zero categories: **Sensor outputs 0/5** (not applicable in dataset) and **Compilation warnings 0/5** (papers use compilation errors, not warnings).

Exceptions and scope. Embedded firmware is the target in only **2/5 (40.0%)** studies (S13, S17); CoverUp (S24) targets Python and Panta (S25)/LLMLOOP (S26) target Java, so the source qualifies these prevalences as spanning embedded *and* general-purpose test generation. **Overlaps.** Sources co-occur within loops; sums exceed 100%. **Limitations.** At N = 5 one study = 20 points; loop frequency is not evidence of loop effectiveness, and no effectiveness ranking is reported here.

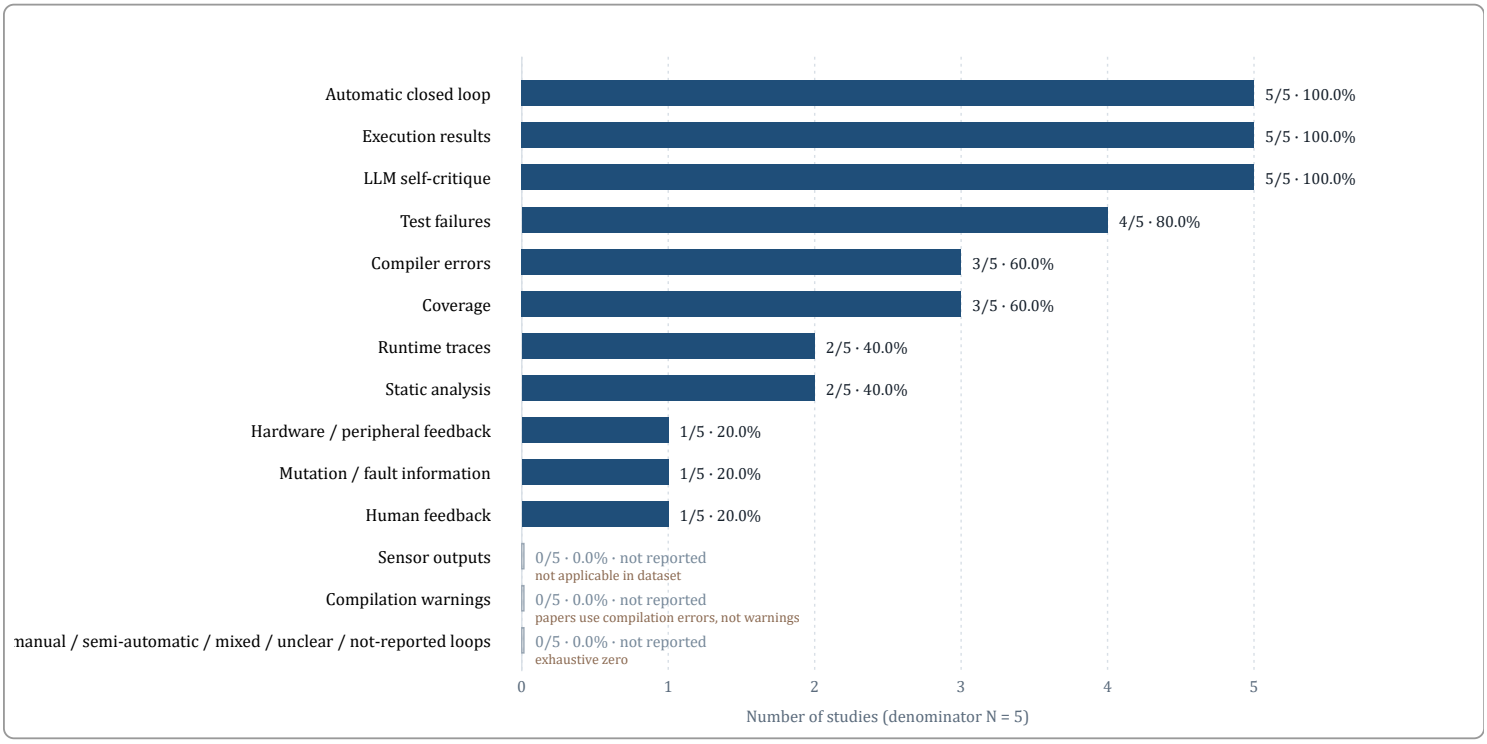


Figure 10. RQ7 — feedback loops and feedback sources (N = 5 studies, 25 feedback-mechanism records). Automatic closed loop 5/5 (100%), with one-shot, manual, semi-automatic, mixed and unclear loops each 0/5. Sources: execution results 5/5; LLM self-critique 5/5 (the term itself appears in none of the papers); test failures 4/5 (80.0%); compiler errors and coverage 3/5 each (60.0%); runtime traces and static analysis 2/5 each; hardware/peripheral feedback 1/5 (20.0%); sensor outputs and compilation warnings 0/5. Embedded firmware is the target in only 2/5 studies, so these prevalences span embedded *and* general-purpose test generation and must not be read as embedded-C only. Sources co-occur, so sums exceed 100%. Source: `fig09_rq7_counts.csv` .

4.8 RQ8 — What metrics and validation methods (compilability, pass rate, coverage, mutation score, fault detection, human evaluation) are used to evaluate generated tests?

Direct answer. In the RQ8 dataset (**4 studies / 5 documents**, S17 ADFEmu, S26 LLMLOOP, S27 MuTAP, S28 Garcia-Carrasco; 46 metric-level records; Figure 11), verification methods are: **Execution 4/4 (100%)**, **Benchmark comparison 4/4**; **Compilation 2/4 (50.0%)**, **Mutation testing 2/4**, **Simulation / emulation 2/4**, **Coverage analysis 2/4**; **Qualitative analysis 1/4 (25.0%)**, **Static analysis 1/4**. Headline metric names reported per study: basic-block/code coverage (S17), mutation score (S27), pass@1 (S26), pass rate (S28). Zero categories: **Real hardware 0/4**, **HIL 0/4**, **Fault seeding 0/4**, **Fault exposure 0/4**, **Human evaluation / expert review 0/4**, and **Statement / Branch / Function / Condition coverage 0/4** each. Comparability: **common benchmark 0/4**, **common baseline 0/4**, **common quantified metric 0/4**, **identical evaluation environment 0/4**, and **coverage AND mutation score together 0/4**.

Exceptions and not-reported cases. Where a metric name recurs across studies, definitions and units differ, so the source bars pooling ("not comparable"; fault detection appears only as fuzzing crash detection, not seeded-fault exposure). **Overlaps.** Method categories

co-apply. **Limitations.** At N = 4 the source explicitly prohibits prevalence claims; RQ8's own summary contains nine internal count conflicts on non-zero magnitudes (source conflict D-3) — the verified classification tables (all values above) are unaffected and are the only source used here; percentages use the 4-study denominator with the 5-document structure disclosed (note N4, §4.11).

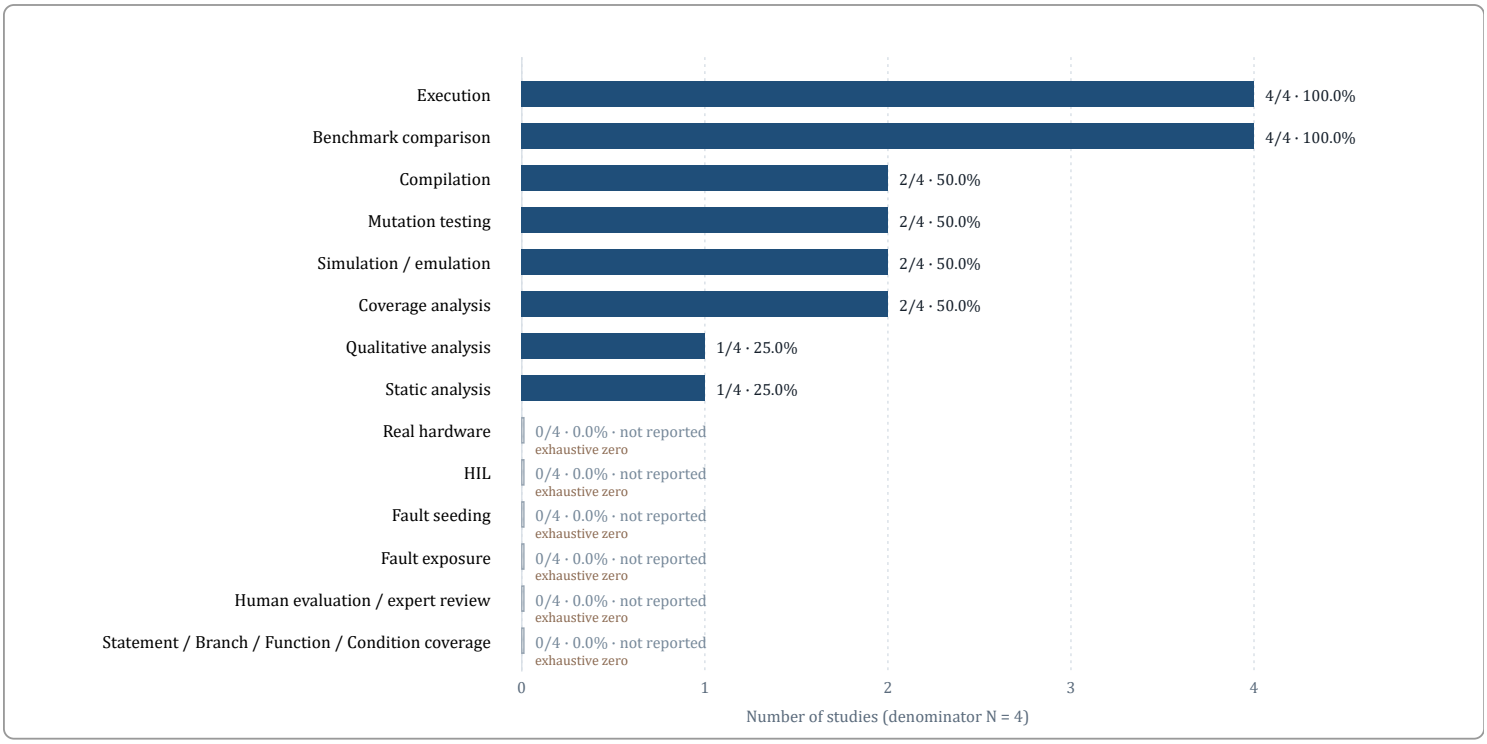


Figure 11. RQ8 — verification methods and evaluation metrics (4 studies / 5 documents, 46 metric-level records). Execution 4/4 and benchmark comparison 4/4; compilation, mutation testing, simulation/emulation and coverage analysis 2/4 each; qualitative and static analysis 1/4 each. Real hardware, HIL, fault seeding, fault exposure, human evaluation and expert review, and statement/branch/function/condition coverage are each 0/4, as are common benchmark, common baseline, common quantified metric and identical evaluation environment. At N = 4 the source explicitly prohibits prevalence claims. Nine internal count conflicts affect non-zero magnitudes only (D-3); the zero rows relied on here are conflict-free. Source: `fig10_rq8_counts.csv` .

4.9 RQ9 — What limitations and research gaps remain in hardware-aware LLM-based test generation for embedded C?

Scope of this subsection. RQ9's verified question asks for *remaining limitations and research gaps*, so this is a descriptive evidence-gap synthesis: zero-count categories, low-frequency categories, not-reported/unclear categories, under-represented contexts/methods/outcomes, and absence of evidence across included studies. No future-work proposals, priorities, policy statements or causal explanations appear here; the RQ9 source's collection of author-stated future directions is reserved for the Discussion by this section's scope.

Denominators and counting rules. The gap synthesis rests on **6 studies at study level** (ADFEmu = S17; Abtahi & Azim = S13; Asmita et al.; a general LLM-testing survey, RQ9-P4; HardSecBench, RQ9-P5; P2IM-lineage, RQ9-P6). Extracted limitation items: **79 → 4 rejected in verification → 75 retained, of which 23 (30.7%) were downgraded by the verification files themselves**. Instance counts are never substituted for study counts (ADFEmu contributes **13/75 (17.3%)** of retained instances but counts once per gap category). Gap categories overlap across studies, so study-support counts do not sum to 6. All percentages are out of 6 (Figure 14).

4.9.1 Gaps supported by author-acknowledged limitations (explicit tier), study support out of 6. Execution validation is present but never hardware-based 5/6 (83.3%); **LLM capability real but narrow and merely advisory 5/6**; **Peripheral / interface dependencies 4/6 (66.7%)** — the most consistently verified hardware gap; **Test oracle problem unresolved 3/6 (50.0%)**; **Timing / temporal behaviour unmodelled at key points 3/6**; **Interrupt semantics handled incompletely 3/6**; **Register / configuration layer handled weakly 3/6**; **DMA support missing or limited 3/6**; **Human involvement not eliminated 3/6**; **Platform and generalisability ceiling**

3/6; **RTOS / concurrency handling limited 2/6 (33.3%)**; **Drivers / HAL complexity 2/6**; **Functional correctness does not imply security correctness 1/6 (16.7%)**; **Reproducibility: closed-source model dependency 1/6**. Composition matters as much as totals: the register 3/6 mixes 1 explicitly stated author limitation with 2 synthesis-inferred adjacent items; the DMA 3/6 includes one item coded *from silence* (its study's own field reads "Not reported"); the interrupt-semantics 3/6 carries an attribution warning — the source's per-study assignment is partly truncated (note N6, §4.11). The two 1/6 gaps are flagged by the source itself as single-study observations, not established literature-wide gaps.

4.9.2 Under-represented and zero-count categories inside the verified RQ1–RQ8 datasets (type I: the method was simply not reported in-sample). Interrupt/async/timing *tests generated by LLMs* 0/6; nested interrupts and priorities 0/6; RTOS scheduler modelling, thread interleaving, task switching, message/event queues 0/6; fake/mock clock and simulation-time mechanisms 0/6; delay-duration and timeout validation 0/6; periodic execution 0/6 (RQ5, §4.5). LLM-built simulator / emulator models, peripheral models, device drivers, HAL and behaviourally faithful doubles: 0 of 27 configurations; SPI, Ethernet, DMA, interrupt controllers and Flash/EEPROM as assessed dependencies 0/6 (RQ4, §4.4). Register map, MCU / processor specification, clock / memory-map / timer / DMA / interrupt specs, board / electrical-interface / sensor-actuator / protocol docs and HAL / driver / BSP / RTOS at inference time 0/8; datasheet+RAG 0; peripheral specification 1/8 (RQ3, §4.3). Integration testing 0/4 and Regression testing 0/4; C-language test code 1/4 (RQ1, §4.1). Chain-of-thought (explicit technique) 0 configurations; fine-tuning of the generation LLM 1/7 (RQ2, §4.2). Real hardware as primary execution 0/6; Simulator (strict term) 0/6; Virtual platform 0/6 (RQ6, §4.6). Sensor outputs 0/5; Compilation warnings 0/5 (RQ7, §4.7). Real hardware, HIL, fault seeding, fault exposure, human evaluation / expert review and Statement / Branch / Function / Condition coverage 0/4 each; common benchmark / baseline / metric / environment 0/4 each (RQ8, §4.8). Reporting-completeness gaps: LLM identity and Prompting Not reported in ≥1 configuration for 5/7 (RQ2); specific peripheral types unreported in 3/6 (RQ4); Not recorded year in 9/28 studies (Figure 2).

4.9.3 Zero counts that do NOT establish gaps (type II — silence in the limitation discussion). The verified files record **0/6 studies discussing as limitations**: HIL, RTOS as such, HAL as a distinct limitation, hallucination, incorrect / invented APIs, named reasoning-limitation categories, formal threats-to-validity sections, statistical analysis, and "lack of execution validation" as such (no study claims validation is missing — the documented issue is its non-hardware basis, §4.9.1). Each is retained strictly as a reporting observation: **silence is not absence**, and none supports a literature-gap claim. Table 5 summarises the type I / type II split for recurring constructs.

Table 5. Constructs whose zero counts are type I (method-use, in-sample) versus type II (limitation-discussion, silence).

Construct	Type I status (dataset, N)	Type II status (RQ9, N = 6)
HIL	0/6 RQ5; 1/6 baseline-only RQ6; 0/4 RQ8	0/6 discussed as limitation → not a gap claim
RTOS / scheduling semantics	0/6 modelling, 0/6 interleaving (RQ5)	0/6 as limitation; 2/6 acknowledge platform scope
HAL	0/8 inference-time context (RQ3); 0/6 generated (RQ4)	0/6; "not established"
Interrupt tests generated by LLMs	0/6 (RQ5)	3/6 acknowledge incomplete interrupt handling
Execution validation basis	0/6 primary real hardware (RQ6)	5/6 acknowledge hardware-free validation
Oracle determination	0/4 human / standard-granularity oracles (RQ8)	3/6 acknowledge oracle problem

4.9.4 Cross-dataset structural observations (descriptive; no pairing claim). Of the 28 RQ1–RQ8 study-set pairs, **14 have zero overlap, 12 overlap 1–3 studies, and 2 overlap ≥4 studies** (RQ2×RQ3 = 6; RQ2×RQ4 = 4); within the low band, RQ5×RQ6 = 2, RQ5×RQ7 = 2, RQ6×RQ7 = 2, RQ7×RQ8 = 2, RQ5×RQ8 = 1, RQ6×RQ8 = 1, RQ1×RQ2 = 1, RQ1×RQ3 = 1, RQ3×RQ4 = 3 (+1 feature-level) (Figure 13). The strategy/context datasets (RQ2, RQ3) share **no studies** with the interrupt/environment/feedback/metric datasets (RQ5–RQ8); the only documented bridges between the two clusters run through RQ4 and depend on the feature-level identity match M2 (R4_p4 ≟ Abtahi & Azim; if rejected, the RQ4×RQ5/RQ6/RQ7 cells drop to 0). The verified corpus contains **no single study that is simultaneously LLM-based, test-generating, embedded-C-source-level and hardware-aware**; the four elements are distributed across the datasets. This is reported as the observable structure of the included studies, not as a causal or evaluative claim.



Figure 14. RQ9 gap-evidence matrix (denominator: 6 studies). Rendered as a matrix and never as a bar chart, because limitation-instance counts must not be read as prevalence: 75 instances were retained of 79 extracted (4 rejected in verification), and one study alone contributes 13 of the 75 (17.3%) while counting once per category. Cells carry evidence grades — EA author-acknowledged, EA-q acknowledged but downgraded in verification, SI synthesis-inferred from adjacent evidence, IFS inferred from silence, NE not established, NR outside the study's scope. G14 (HIL) and G16 (self-reflection) are 0/6 **type-II** counts, marked not established; they may not be converted into literature-gap claims. Only two of the six evidence studies (S17 ADFEmu and S13 Abtahi & Azim) are corpus members; the other four are RQ9-side works and are never counted toward the 28. Source: `fig13_rq9_gap_matrix.csv` .

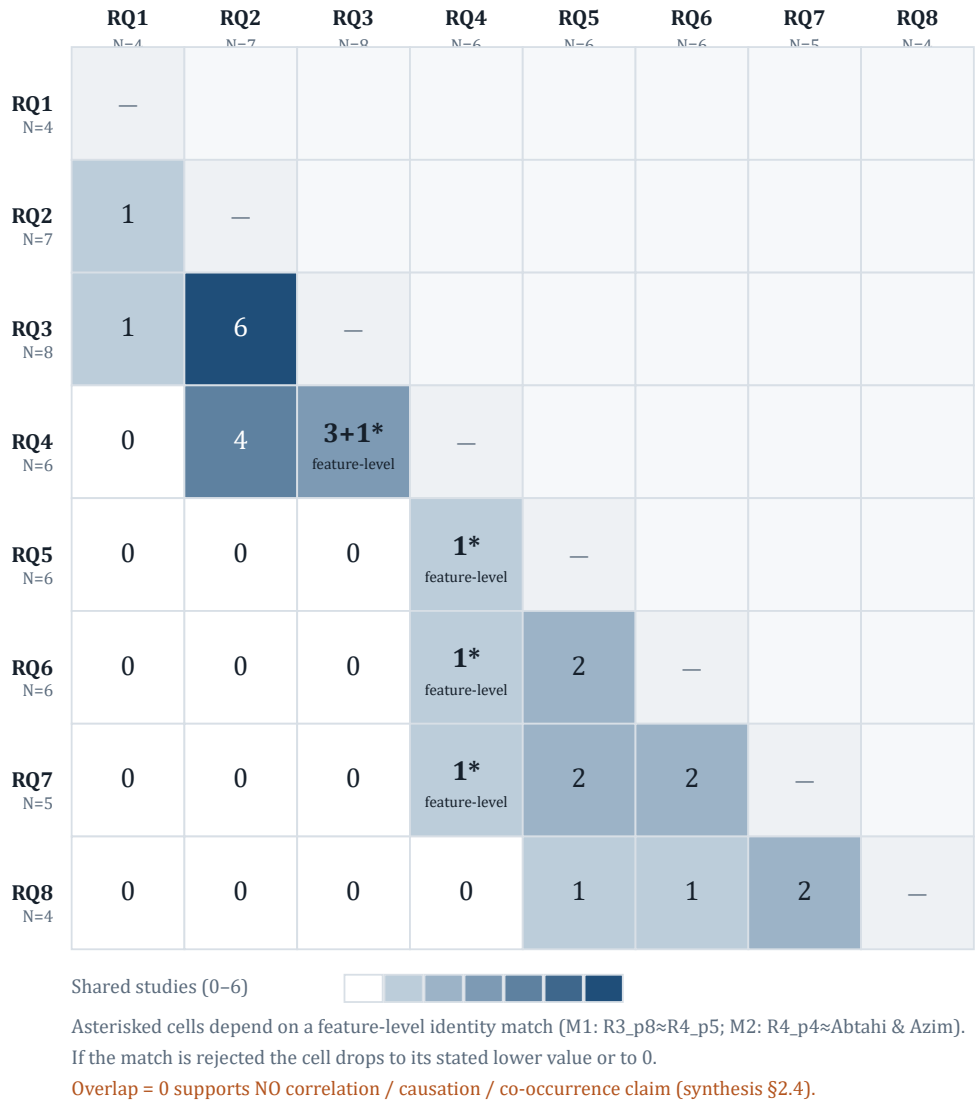


Figure 13. Cross-RQ study-set overlap matrix over the 28 RQ1–RQ8 pairs. Cell values are counts of studies shared by the two datasets. Fourteen pairs share zero studies, 12 share 1–3, and 2 share four or more (RQ2×RQ3 = 6; RQ2×RQ4 = 4). Asterisked cells depend on the pending feature-level identity match M2 (record R4_p4 ≟ Abtahi & Azim); if M2 is rejected those cells drop to 0 and the two corpus clusters become fully disjoint. RQ9 is not included as a peer dimension: it is a derived synthesis over a different six-study evidence set, not an empirical dataset. Because 14 pairs are zero, any statement linking two dimensions is a structural contrast between disjoint populations rather than a paired finding, and no association, correlation or causal claim is made. The matrix value 14 is adopted manuscript-wide; the cross-synthesis narrative's 15 is not reproducible from this table (Appendix E, U10). Source: `fig12_cross_rq_overlap_matrix.csv` .

4.10 Cross-cutting figure summary

Figures 4–11 answer RQ1–RQ8 at their own denominators; Figure 12 carries the RQ4 artefact × status matrix; Figure 13 renders the cross-dataset overlap matrix of §4.9.4; Figure 14 renders the RQ9 evidence matrix behind §4.9.1; Figure 15 renders the screening-side exclusion reasons of §4.0. No figure pools counts across datasets.

4.11 Data notes and unresolved source discrepancies (consolidated)

- **N1 — Denominators.** Every percentage is computed within its own RQ study denominator (Table 4); cross-RQ pooling is never performed. RQ9 percentages use 6 studies, never 75 or 79 instances.

- **N2 — Meaning of zeros.** All 0 counts mean "not reported within the verified dataset"; type II silences (Table 5, §4.9.3) are additionally barred from gap interpretation.
- **N3 — Full-text provenance.** 36 of 46 retained records' full-text evidence is author-verification documents (page-level quoted extractions) rather than publisher PDFs; all 28 studies nonetheless have consultation evidence. 18 studies lack publication originals on file.
- **N4 — RQ8 internal conflicts.** Nine non-zero count conflicts exist in RQ8's own summary (D-3); this section uses RQ8's verified classification tables, whose values above are conflict-free; conflicting magnitudes are not reported here.
- **N5 — Cross-document coding conflicts kept unmerged:** RAG 5/7 (RQ2) vs 3/8 (RQ3) — D-1; truncated RQ5 "Simulator" row (reads 6/6) vs RQ6 Simulator 0/6 — D-2; S09 HIL label ambiguity — D-9.
- **N6 — Attribution pendings inside the verified syntheses:** feature-level matches M1/M2; study identity of grey-zone S12; per-study assignment of the interrupt-semantics gap; the RQ9 reproducibility-gap row does not name its study. None affects any count in §4.1–§4.8; they qualify §4.9 composition only.
- **N7 — Narrative-vs-matrix zero-overlap count.** The cross-synthesis narrative and its own matrix differ by one pair (15 vs 14); this section uses the matrix values throughout (14 zero-overlap, 12 low-band), which are the same values plotted in Figure 13 and exported in `fig12_cross_rq_overlap_matrix.csv`.
- **N8 — Year rule at study level.** A study is year-stated only if at least one record states a four-digit year; parenthetical mentions ("references 2026") are treated as not stated. The single adjudication (S26 version pair plotted at its peer-reviewed year) is disclosed in Figure 2's panel.

---

5. Discussion

Reading conventions. Three rules from Section 4 carry over verbatim to every statement below. First, observations and interpretations are kept distinct: reported counts, verbatim source statements and per-study attributions are observations; analytical readings are flagged as interpretations and are never attributed to the primary studies. Second, every percentage holds only within its own dataset denominator (RQ1 = 4, RQ2 = 7, RQ3 = 8, RQ4 = 6, RQ5 = 6, RQ6 = 6, RQ7 = 5, RQ8 = 4; RQ9 = 6 studies at study level, never the 75 retained limitation instances); no percentage is pooled, averaged or summed across datasets. Third, a zero count means "not reported within the verified dataset under our classification lens", never "absent from the literature"; and zeros at the method-use level (type I) are distinguished throughout from zeros at the limitation-discussion level (type II), which may not be converted into gap claims. Because 14 of the 28 pairwise study-set overlaps are zero (§4.9.4), any statement linking two RQ dimensions for which no study carries both measurements is a *structural contrast between disjoint populations*, not a paired finding; no association, correlation or causal claim is made anywhere in this section unless it rests on within-study co-occurrence.

5.1 The organising finding: a research question without a fully instantiated study

The most consequential observation in this review is about the shape of the evidence itself. The verified dataset contains **no study that is simultaneously LLM-based, test-generating, embedded-C-source-level and hardware-aware** (RQ9 construct-validity statement, confirmed independently by all six RQ9 verification files). The 27 identifiable studies across the eight empirical datasets split into two almost-disconnected clusters (revised cross-RQ synthesis §1.4): cluster A (test/code generation and automotive system-level work, 12 studies, populating RQ1–RQ4) and cluster B (firmware fuzzing, re-hosting and general-purpose test generation, 15 studies, populating RQ4–RQ8). Strategy and context evidence (RQ2, RQ3) shares **no study** with interrupt, environment, feedback and metric evidence (RQ5–RQ8); the only documented bridge between the clusters runs through Abtahi & Azim (S13) via a feature-level identity match (M2) that remains pending author verification (§4.11, note N6). If that match is rejected, the RQ4×RQ5/RQ6/RQ7 overlaps drop to zero and the two clusters become fully disjoint bodies of evidence.

Interpretation. We read this structure — following the cross-RQ synthesis (I9) — as characteristic of a pre-paradigm stage of a cross-cutting topic, not as a failure of the literature: the four elements of the review's core construct are being worked on, productively but

separately, by two research communities that publish in different venues and evaluate against different criteria (the domain split recorded in RQ7, where embedded-firmware studies emphasise runtime/hardware-adjacent feedback while general-purpose test-generation studies emphasise coverage, compilation and mutation feedback, is consistent with this reading). The immediate analytic consequence is that every "relationship among dimensions" discussed below is constrained by the evidence structure: the strongest statements available are (a) exhaustive within-dataset counts, (b) directionally convergent patterns across disjoint datasets, and (c) author-acknowledged limitations at the study level. None of these licenses a causal claim.

5.2 Test types, LLM strategies and context: what the paired evidence shows

The one well-paired relationship. RQ2 × RQ3 (6 studies, title-level matches; S02 EmbC-Test, S05 openSIL, S06 EmbedAgent, S07 Smarter-not-Bigger, S08 H2LooP, S09 EmbedGenius) is the only candidate relationship with adequate same-study pairing (observed). Within it, the pattern is consistent and specific. Retrieval-augmented generation is common (5/7 in RQ2; 3/8 on RQ3's narrower inference-time criterion — the two codings are kept unmerged per §4.11 note N5), but what is retrieved is exclusively software artefacts: legacy tests, repository structure, build logs, dependency lists, library-usage rules, known-good `#include` directives; **no study retrieves a datasheet, register map or hardware specification document** (RQ3, verbatim). Register maps, MCU/processor specifications, board documentation, clock/memory-map/timer/DMA/interrupt/protocol/sensor-actuator specifications and inference-time HAL/driver/BSP/RTOS artefacts are each 0/8; datasheet+RAG, register map+HAL and HAL+driver+peripheral-specification combinations are each 0. Among the studies whose LLM role is genuinely test generation (S02, S05, S12), confirmed hardware context is **0/3**. Context richness does not track strategy complexity: the richest hardware context in the dataset (S06: the only confirmed peripheral specification, plus board, pinout, schematic) coexists with a mid-complexity stack, while the only study supplying datasheets at inference time (S08 H2LooP) does so via training-time weight consolidation — the source records the 117-vendor pretraining corpus as "training-time, not inference-time context" — and performs code completion, not test generation.

Interpretation. In this dataset, "hardware awareness" is operationalised almost entirely as software artefacts plus execution feedback — RQ3's own phrase is that execution feedback (compiler errors, build/flash logs, coverage, HIL action results, present as context in 4/8 studies) is treated as a **stand-in for hardware awareness**. The strategic machinery of the field (RAG, agentic loops, tool pipelines) is being spent on the software side of that boundary. Whether the substitution works is exactly what the validation and metric datasets cannot answer (5.6–5.8), because no study carrying context evidence also carries execution-environment or metric evidence.

Test types × strategies: not assessable. RQ1 × RQ2 shares a single study (S02, which is a unit-test generator using RAG-grounded structured prompting on host-side artefacts). The RQ1 distribution is strictly bimodal — Unit 2/4, HIL 2/4, Integration 0/4, Regression 0/4 — and the two HIL studies (2511.19132v1; AutoHIL, 3832155) carry no strategy records at all, while the only HIL-adjacent study in RQ2 (S07) was coded as test *support*, not generation, and RQ1 records LLM support activity as "never directed at the authoring of a non-unit, non-HIL test type". **No mapping between test type and strategy can be claimed** (observation about the evidence structure). The two 0/4 test types must also be read differently (per §4.1): Integration's zero is an *evidence void* — two explicit scope exclusions that ceded integration to established SIL/MIL/PIL toolchains — whereas Regression's zero is *total silence* (no explicit "No" anywhere), supporting neither capability nor incapability.

5.3 Dependency handling meets the execution environment: an empty intersection

Three datasets describe the same boundary from disjoint vantage points (all observations).

- **RQ4 (dependencies, n = 6).** In 5/6 studies the LLM produces *no* dependency-handling artefact; handling is delegated outward to existing platforms (Wokwi, QEMU plus the official FreeRTOS port), an industrial virtual-vehicle system, an author-built HIL testbench or physical boards. The one unambiguous positive cell is LLM-generated tests plus stubs (S05, host execution); mocks are "Partially / Not established". Every combination of LLM-generated artefacts with hardware-relevant execution — tests with simulators, emulators, HIL or real hardware; LLM-built peripheral models, device drivers, HALs, wrappers, configured execution environments; test doubles combined with simulation/emulation/physical execution — is **0 studies**. Behaviourally faithful doubles (register, timing, interrupt, protocol semantics) are 0/27 configurations, and quantitative fidelity assessment of virtual environments against real hardware is likewise 0 ("No study measures how closely its virtual environment matches real hardware"). The replacement and provision families "never intersect"; hybrid/layered strategies are 0/6.

- **RQ6 (execution, n = 6).** Execution and validation are universally emulator-based and host-run (6/6; QEMU in 5/6); real hardware appears only in secondary roles — bug confirmation by re-execution (S20 P2IM) and an Avatar baseline (S21 HALucinator); HIL appears once, only as a baseline; five studies "explicitly avoid HIL and market hardware-independence as a design goal". No study executes its generated test suite primarily on physical hardware, no "Generation → Compilation → Real Hardware" flow exists, and the compilation of generated tests is reported nowhere.
- **RQ1 (test types, n = 4)** supplies the complementary negative: no study separately qualifies its environment as real target hardware, simulator or emulator, and none reports a complete generation-to-pass funnel. The unit peak (S02, and S04 tran-mt25 — the only study emitting C test code) executes on host toolchains or reports no platform at all; the HIL peak executes on real industrial benches, but there the LLM-generated artefacts are system-level test steps, not C-level test code. S04's own record is the dataset's sharpest negative evidence on the core question: on Zephyr/RTOS code its generated tests were "often unusable without extensive manual modification", the authors "decided to stop further efforts", and concluded that LLMs are "not yet capable of fully autonomous test generation for large, framework-driven, or hardware-dependent software".

Interpretation, bounded. The conjunction "hardware specification in context × LLM interrupt/timing-test generation × HIL validation" (gap B10/U1) and "C-level test generation × hardware-relevant execution" (B14/U5) were never instantiated in any single verified study. This must be read as the *conjunction of independent within-dataset zeros across non-overlapping datasets* — the structure of the literature makes these combinations unobserved; it does not show them to be ineffective. Alternative explanations are retained per the gap analysis: hardware independence is an explicitly marketed *design goal* in cluster B, not a shortfall; delegation to mature infrastructure (S05's strongest configuration deliberately *retrieves* pre-existing doubles rather than synthesising them) may be an engineering conclusion rather than an omission; and RQ1's HIL peak shows LLM activity does occur on real benches, one abstraction level above C test code. What the evidence does support is the existence claim: **no verified study has yet connected the two sides**, and the missing middle layer — behaviourally faithful doubles with a quantified fidelity measure — is named by RQ4 as the most consistently reported yet least consistently measured dimension.

5.4 Interrupts, asynchronous behaviour and timing: modelling without LLMs, inherited time without control

Within RQ5 (n = 6; observations): asynchronous and timeout-related behaviour are reported by 6/6; interrupts are present in 5/6 targets, with ISR handling, interrupt injection, NVIC simulation, temporal assertions and delay handling at 4/6 each. Real-time *property* measurement (WCET, jitter, deadline validation, race-condition testing) is 1/6 for all four — concentrated in the single study S13 (Abtahi & Azim), whose own interrupt handling is "Not reported"; **no study combines interrupt modelling with real-time property measurement**. LLM involvement is limited to 2/6 (S17 path selection; S13 firmware/patch generation) and **LLM-driven interrupt/async/timing test generation is 0/6**; likewise nested interrupts and interrupt priorities were tested by 0/6 (RQ5-P1 explicitly excludes nested/tail interrupts; NVIC-capability mentions in S16 and S19 were removed by verification as inference). Time is inherited, never controlled: virtual/simulation time appears as a side effect of QEMU/Unicorn/Icicle/angr in 5/6, while fake/mock clocks, deterministic simulation-time mechanisms and clock-domain modelling are 0/6 — "temporal control in this literature is event-ordering control, not clock control" (Trend 4, verbatim). "Timeout" is universal as an engineering artefact (build budgets, path thresholds, failure modes) and absent as a tested property (0/6; Trend 5). OS-level RTOS semantics are 0/6 (scheduler modelling, task switching, thread interleaving, message/event queues), but this must be read together with the non-zero concurrency counts (Concurrency 2/6; explicit scheduling control 2/6, one of these being input-delivery scheduling, not OS scheduling; lock/mutex, semaphore and thread counts 1/6 each) — the accurate statement is *OS-level scheduling semantics unmodelled*, not *concurrency unhandled*. S13's virtualised environment was "configured" with interrupt latencies but these were not tested ("configured but not tested"), and the study itself states that virtualisation "cannot fully capture timing behavior, peripheral interactions, and resource constraints", deferring board-level validation (STM32/ESP32/Cortex-M) to future work; the only real-hardware interrupt testing (S16 IEmu, hardware as standard of truth) has no entry in the execution-environment dataset.

Convergence with the context side, causality barred. RQ5 records that all four interrupt-modelling studies obtain the interrupt model *from the artefact under test, not from documentation* (Trend 3), with S16 explicitly avoiding device-manual dependence; RQ3 independently records interrupt/timer/DMA specifications as LLM inputs at 0/8. The two populations do not overlap, so the tempting causal reading — "hardware documentation is missing from context, therefore models must be extracted from binaries" — is **not testable** in this evidence base. What is supported: in two independent datasets, hardware specification documents were

exhaustively verified as absent — once from model input, once from model source. What is also supported as observation: extraction-from-artefact is a deliberate design choice in cluster B, an alternative explanation that weakens any deficit framing of the same zeros.

5.5 Feedback loops: closed everywhere, on the same signals

RQ7 (n = 5) reports loop closure as universal — automatic closed-loop refinement 5/5, with one-shot, manual, semi-automatic and mixed strategies at 0/5 — and quantified benefits where reported: iteration contributes roughly 40% of S24 CoverUp's successful test generations, and S26 LLMLOOP's loops raise pass@1 from 71.65% to 80.85% (pass@10 76.22% → 90.24%). The feedback sources that close the loops are execution results (5/5), LLM-generated corrections (5/5 — a mechanism label the source explicitly notes no paper terms "self-critique"), test failures (4/5), compiler errors (3/5) and coverage (3/5). Hardware-adjacent signals are thin: hardware/peripheral feedback 1/5 (S17's DMA input-channel emulation), runtime traces 2/5, sensor outputs 0/5, compilation warnings 0/5. The loop-saturated studies are mostly not embedded (embedded firmware in only 2/5; S24 Python, S25 Panta and S26 LLMLOOP Java), and the two embedded studies report no iteration budget (0/2; S13's stopping rule is qualitative).

Interpretation. The loops, as built, refine what tooling can observe: compilation, execution, coverage and static findings. The semantic dimensions that define hardware-aware embedded testing — registers, interrupts, timing, protocol behaviour — are exactly the signals the loop ingests least (1/5 hardware feedback, against context and dependency zeros above). This is again a convergence across datasets rather than a measured mediation: RQ7 shares only S13/S17 with the dependency/environment datasets, so we cannot claim that loop design causes the hardware gap — only that within the verified evidence, closed-loop machinery and hardware semantics are strongest where the other is weakest (compare S13: the only study with WCET/jitter/deadline feedback closing a patch loop, and simultaneously the study whose hardware validation is explicitly future work).

5.6 Metrics and validation: decoupled proxies and absent comparability

Within the RQ8 dataset (n = 4; observations): every study validates by execution (4/4) and benchmark comparison (4/4); compilation, mutation, simulation/emulation and coverage analysis appear in 2/4 each; real hardware, HIL, fault seeding, fault exposure, human evaluation, expert review and statement/branch/function/condition coverage are each 0/4; there is no common benchmark, no common baseline, no common quantified metric and no identical evaluation environment, and no study quantifies coverage and mutation score together. Where a metric name recurs (pass rate, execution success, fault detection), definitions and denominators differ across studies — the source bars pooling on exactly that ground. Two studies supply the dataset's decoupling evidence: MuTAP (S27) *dropped* coverage because "coverage is weakly correlated with fault detection" (even while reporting mutation score 93.57% ± 11.18 against Pynguin's 65.94%), and within RQ1, the AutoHIL ablation separates executability (89.15%) from functional correctness (50.51%) and accuracy (46.10%), while S02's generated tests reach an 84.5% runtime-validation rate yet sit below the human baseline on branch (43% vs 76%) and line (67% vs 93%) coverage. Because RQ1 × RQ8 overlap is zero, these are **two disjoint datasets independently recording the same dissociation** (observation + interpretation flagged as such).

Consequences (evidence-supported). (i) Quantitative synthesis is impossible: RQ1 states meta-analysis/effect-size synthesis "impossible" on its own evidence, RQ8 forbids prevalence claims at n = 4 — and this review's own execution of Section 4 follows that rule (no pooled rates anywhere). (ii) Quality claims about generated tests built on a single proxy — compilability, pass rate, or coverage — are unsupported by this dataset; the verified evidence actively contradicts the proxies' agreement. (iii) The metric that would matter most for hardware-aware C tests — fault detection against hardware-relevant behaviour (fault seeding/exposure) — has no verified instance. (iv) The funnel is never complete: no study reports generated → compilable → executed → passed on the same artefacts, and RQ6 adds that "compilation of generated tests is not reported in any paper" in its dataset; comparison across studies therefore compares different populations of artefacts.

5.7 Context reporting, reproducibility, and the reliability of this synthesis

Incomplete context reporting (type I observations). Model identity is unreported in at least one configuration for 5/7 RQ2 studies; prompting is unreported in at least one configuration for 5/7; specific peripheral types are entirely unreported in 3/6 RQ4 studies; 9/28 studies carry no recorded publication year; the corpus is preprint-heavy (6/7 of RQ2 are arXiv preprints with only one peer-reviewed venue; DOIs are largely absent across RQ1/RQ2/RQ7; one included study is a master's thesis admitted under the grey-

literature decision). **Interpretation, stated plainly:** these are limits on *auditability of this review's own conclusions* — for instance, no future cross-study test of "which strategy yields which validation outcome" can even be assembled retrospectively because 5/7 studies leave the strategy field partially blank. They are not evidence that the wider literature fails to report such information.

Reproducibility. The only reproducibility-related limitation verified at the study level in RQ9 is closed-source model dependency (1/6 — S13's GPT-4 reliance); per the single-study rule this is recorded as a single-study gap, not a literature-wide one. Reporting absences for prompts, datasets, source code or hardware settings were *not* verified as limitations (0/6, type II) and are therefore not converted into a reproducibility-gap claim.

Counter-intuitive silences (must be stated with their limits). The RQ9 verification layer found hallucination, incorrect/invented APIs, named reasoning-limitation categories, formal threats-to-validity sections, statistical analyses, HIL, HAL and RTOS-as-limitation discussed by **0/6 studies** (type II: silence, not absence). Meanwhile the LLM-side weaknesses that *were* author-acknowledged are concrete and different: missing context causing ~60% and model logical inconsistency causing ~40% of S13's compilation failures; LLM-as-a-judge verdicts "unreliable and inconsistent across runs"; independently generated solutions "converging to functionally correct designs with similar security weaknesses"; the LLM functioning "as a heuristic guide" whose bad suggestions do not block exploration (S17). One observation from the other side of the corpus complicates the silence: S10 (Englhardt et al., RQ2 dataset, not among the six RQ9 studies) records a register-level fabrication failure — GPT-3.5 compiled code that hallucinated at least one register address or configuration value in 100% of programs, and GPT-4 produced verifiable working IMU code in 16% of cases (14% and 66% variants of the same study's figures are recorded in different sections; §4.11 N4-type multi-version caution applies to every number quoted from this study). The honest composite: fabrication-type failures *are observed* in extracted data where the study looked for them, but were never *verified as an acknowledged limitation* within the RQ9 population; research programmes targeting "hallucination reduction" as the field's bottleneck therefore lack support in this dataset's verified failure profile, while the dataset's actual verified failure modes (context loss, inconsistency, judge instability) point elsewhere.

Reliability of the synthesis itself. The RQ9 evidence was downgraded in verification for 23 of 75 retained instances (30.7%) and 4 instances rejected; instance counts were never substituted for study counts (S17 alone contributes 13/75 = 17.3% of instances but one study per category). RQ8's summary contains nine internal count conflicts on *non-zero* magnitudes — its zero rows used above are unaffected — and several source papers carry unreconciled internal numbers (§4.11 N4, N5). Eight identity/terminology matters remain pending author adjudication (V1–V8), of which M2 (S13's identity as R4_p4) is structural: it decides whether the clusters are connected at all. **Interpretation:** these are limitations of *this review's evidence base*, not findings about the literature; Section 7 (Threats to Validity) is the proper venue for their procedural treatment.

5.8 Synthesis: what the combined evidence does and does not establish about hardware-aware embedded C test generation

Across the nine RQs, one composite picture survives the overlap constraints (each element observation-anchored; the composite reading flagged as interpretation):

1. **The generation side is real but software-anchored.** LLMs generate unit-level tests, stubs and hardware-adjacent production code (19/27 RQ4 configurations), with retrieval and agentic repair loops demonstrably raising compile/execution success in general-purpose settings (S05 96.1% compile-and-link; S09 95.7%/86.5% retrieval+agentic vs 69.5%/56.4% direct; the loop gains in 5.5). What the loops and retrievals act on is code, build and coverage signals — not hardware semantics (5.2, 5.5).
2. **The hardware side is handled by non-LLM infrastructure.** Dependencies are delegated (5/6), interrupts are modelled from binaries (4/4 modelling studies), time is inherited from emulators (5/6), execution is emulator-bound (6/6) and physical hardware appears only to confirm or baseline (2/6, secondary). Where LLMs touch real-time properties (S13), validation is virtualised by the study's own admission, and its interrupt latencies were configured but not tested.
3. **The connective tissue is missing, and the studies say so.** Behaviourally faithful doubles: 0/27; fidelity measurement: 0; LLM-generated tests executed on hardware-relevant substrates: 0; LLM generation of interrupt/timing tests: 0/6; oracle techniques beyond pass/fail and hardware-behaviour references: absent with one verified empirical leak (S20-lineage's PLC bug that the simple memory-error detector missed, recorded in RQ9-P6 L8). The field's own future-work statements cluster on exactly these connections — extending validation to physical boards (S13), peripheral abstraction beyond DMA (S17), input-duration-aware

fuzzing (P2IM lineage), more powerful LLMs (S13 and S17, the *only* future direction proposed by two studies) — while metamorphic testing as an oracle alternative comes from the generic survey (RQ9-P4).

4. **Evaluation cannot yet arbitrate between these positions** (5.6): incomparable metrics, incomplete funnels and decoupled proxies mean the current evidence can describe where effort is spent but not what it buys in test validity.
5. **The construct itself has no complete referent yet** (5.1): "hardware-aware LLM-based embedded C test generation" names a conjunction that no verified study realises end-to-end; the four elements exist in separate literatures whose bridge (M2) is still unconfirmed.

5.9 Practical implications (grounded in the verified evidence)

- **Do not deploy for autonomous hardware-in-the-loop test authoring; deploy where the evidence sits.** The verified positives are host-level unit-test generation with LLM-generated stubs on software-artefact context (S05 — the single unambiguous case), retrieval grounding over repository artefacts, and closed-loop repair against compiler/test/coverage signals (S24, S25, S26). Expect LLM output on framework-driven, macro-heavy, hardware-dependent C (Zephyr-class) to require substantial manual work — this is a direct, single-study negative result (S04), stated at its evidential weight.
- **Treat timing verdicts from emulator-grade environments as provisional.** The verified pattern is event-ordering without clock control (0/6 fake/mock clocks, 0/6 deterministic time mechanisms), timeouts as budgets rather than tested properties, and one study's own admission that virtualisation cannot capture timing/peripheral/resource behaviour. Real-time claims about embedded C tests need physical confirmation in the P2IM sense (re-execution to confirm) or the HALucinator sense (baseline comparison) — both currently one-off practices, or the RQ1 HIL-bench sense — with the funnel actually reported.
- **Judge generated tests by multi-proxy evidence, never by one number.** Executability, coverage and fault detection dissociate within two independent datasets (5.6). Practically: report compilable/executed/passed as separate stages against a fixed denominator, and when validating embedded tests, include fault-relevant checks (seeded or scenario-level) rather than pass/fail alone.
- **Keep human gatekeeping where the datasets place it.** In the industrial HIL/generation studies, humans accepted 38.9%, modified 55.6%, rewrote 5.6% of EmbC-Test artefacts (multi-version caution: the triple sums to 100.1% in the source), 25.79% of scripts needed manual edits, and equivalence and defect validity rested on expert review. RQ7's opposite signal (human involvement 1/5, "shrinking to a gatekeeping role") comes from a mostly non-embedded population; the divergence is unresolved (§4.11 N5-adjacent, D-6) and should be read as domain composition, not as a trend to plan against.
- **Make selection of dependency strategy an explicit engineering decision.** The two verified families — synthesise doubles host-side vs delegate to existing platforms — have never been combined in one study; teams needing both isolation and hardware realism must currently integrate across the boundary themselves, because no verified recipe exists.
- **Require reporting that would let you re-use the work.** Given model-identity and prompting gaps (5/7 each), practitioners adopting published pipelines should expect to re-derive prompts, toolchain pins and board configurations rather than replay them.

5.10 Research implications (directions traceable to the evidence, with status labels)

Author-side, before new data collection (derived; lowest cost). Adjudicate the eight pending items V1–V8; V1 (M2) in particular determines whether any cross-cluster statement in this review survives in its current strength. Until resolved, published text should not describe RQ4 and RQ5–RQ7 findings as evidence about the same study population.

The highest-leverage single study (derived from C1/C2). Produce and report one study that instantiates all four elements simultaneously — LLM generation of embedded-C tests, hardware-specification-informed context, execution on a hardware-relevant substrate (emulator with modelled peripherals, HIL, or board), and a complete funnel — reusing verified artefacts to create the first same-study pairing (e.g., S17's ten firmware samples, P2IM's device set, S04's C-level test material, S05's stub pipeline). Under the current overlap structure, only such studies can convert cross-dataset contrasts into paired evidence.

Context and grounding (author-stated need: peripheral abstraction beyond DMA, S17; derived elsewhere). Test whether register maps / peripheral specifications supplied at inference time — retrieved or structured — change generated-test behaviour on register-access and interrupt paths, against two verified baselines: the software-artefact-only RAG pattern (0/8 hardware-document retrieval) and the extraction-from-artefact pattern (4/4 studies; S16's deliberate manual-avoidance). The alternative explanation that artefact extraction is *superior* must be preserved as a live hypothesis, not dismissed.

Interrupts and timing (author-stated: input duration as "an open challenge for future research", P2IM-lineage; derived: LLM-side). Build LLM-facing representations on top of the interrupt models the cluster-B artefacts already extract (IRQ models, Interrupt Model Tables, rule extraction), targeting the verified zeros: nested interrupts, priorities, ISR-sequence tests, timeout-as-tested-property. Two preconditions from the data: first unify terminology (timeout referents differ 6/6; simulator/emulator classification D-2 pending), and introduce controllable virtual clocks so duration/periodicity become observable quantities; note the scope caveat that some 0/6 counts (e.g., RQ5-P1's nested-interrupt exclusion) bound current studies' claims, not the construct's feasibility.

Doubles, fidelity and oracles (mixed provenance). Construct behaviourally faithful doubles (register/timing/interrupt/protocol semantics) and — critically — a fidelity measure, since the dataset shows the dimension "most consistently reported, least consistently measured" (RQ4). In parallel, develop hardware-behaviour test oracles: the verified leak (S20-lineage PLC bug missed by a memory-error detector) and the acknowledged oracle problem (3/6, strongest at RQ9-P4/P5/P6) define the case; metamorphic testing is author-proposed but has no in-dataset evaluation, and LLM-as-a-judge is a verified unreliable option — neither should be adopted on this evidence alone.

Evaluation infrastructure (author-stated at two studies: specialised/diversified benchmarks, RQ9-P4; security-specific evaluation, RQ9-P5). A shared embedded-C test-generation benchmark must fix MCU family, compiler/toolchain versions, dependency infrastructure, and the four-stage funnel definition, on code unseen during pretraining (leakage: verified concern, RQ9-P4 L4); it should pair coverage with mutation/fault-seeding in one instrument, because the verified dissociation claims (5.6) are otherwise unresolvable. Cost is real: the one large-scale benchmark effort in the RQ9-derived evidence set (HardSecBench lineage, RQ9-P5 — outside the 28-study corpus) documents expensive construction — but it also demonstrates feasibility.

Layered dependency handling (derived, flagged as structurally weakest). Combine LLM doubles at unit level with emulator/HIL execution at integration level — the strategy RQ4 records as never instantiated — and report it inside one study to break the zero-overwall between the replacement and provision families.

What this evidence does *not* support as a research priority (derived prohibitions, kept for balance): framing the bottleneck as hallucination or API invention (0/6 verified as limitation in RQ9; the verified failure modes are context loss, inconsistency, judge instability, convergent weaknesses — the type-II rule means this is "not supported here", not "not a problem"); claiming the literature "ignores" HIL, HAL or RTOS (all type-II silences); and pursuing model-capacity work justified by prevalence statistics this corpus cannot lawfully produce (denominator bounds, §4.11 N1).

5.11 Scope of the interpretive claims in this Discussion

Every statement above is bounded three ways, and these bounds should travel with any citation of this review. First, dataset-relativity: each count describes its own verified sample (sizes 4–8 for RQ1–RQ8; 6 studies at study level for RQ9), and all percentage-extrapolation bans quoted in §4 remain in force. Second, classification-dependence: the zeros inherit our taxonomic lens, and the data record that the lens moves counts — the same QEMU/Unicorn tools sit on opposite sides of a simulator/emulator boundary in two datasets (D-2, pending), "mock" names both a unit double and a system-level simulation verb, "feedback" covers both evaluation-only measurement and refinement input, and "coverage" in one study denotes fault-scenario coverage. Third, search-scope dependence: a different terminological net (re-hosting, cyber-physical test generation, model-in-the-loop, digital-twin testing and similar vocabulary — none of which structured the present extraction) could populate cells currently at zero; accordingly this Discussion asserts that the combinations named in 5.3–5.10 are **unobserved within the verified corpus**, and no stronger claim anywhere in this section should be quoted.

---

6. Research Gaps and Future Directions

6.0 How this section reads the evidence

Five rules from Section 4 govern every claim below, and they are what keep a gap section from becoming a wish list.

1. **Direction of derivation.** Each theme here was named only after inspecting the verified matrix; none was assumed first and matched to evidence afterwards.
2. **Type I versus type II zeros.** A zero at the level of *method use* ("did any verified study do X?" — no) supports an *underexplored-within-this-dataset* claim. A zero at the level of *limitation discussion* ("did any study call Y a limitation?" — no) is silence, not absence, and licenses no gap claim. Both appear below and are never interchanged.
3. **Evidence grades.** EA = author-acknowledged limitation, confirmed by the RQ9 verification files; EA-q = acknowledged but downgraded in verification; SI = synthesis-inferred from adjacent evidence; IFS = inferred from silence; NE = not established (type II); NR = not in the study's scope. Study counts, never limitation-instance counts (the 75 retained instances, 30.7% of which were downgraded in verification), carry the prevalence reading.
4. **The composite ceiling.** No study in the verified dataset is simultaneously LLM-based, test-generating, embedded-C-source-level and hardware-aware [RQ9 §0.5], and 14 of the 28 pairwise overlaps among the RQ1–RQ8 study sets are zero (§4.9.4). Every cross-dimensional gap below is therefore a *conjunction of independently verified zeros across disjoint datasets* — it shows that a combination is unobserved, never that the combination fails.
5. **Provenance of directions.** Each proposed direction is labelled **author-stated** (verbatim future work from a verified study) or **review-derived** (our inference from the evidence, with its preconditions and a stated reason it might not pay off). Single-study items (1/6 in RQ9, or 1/4, 1/7 etc. elsewhere) are marked as such and are not inflated into field-level gaps.

The gap analysis supports all seven candidate themes, but not equally: themes 6.1–6.4 rest primarily on author-acknowledged limitations plus exhaustive type-I zeros; 6.5 is a *composition* gap rather than a method gap (closed-loop feedback is saturated as a technique); 6.6 rests on two disjoint datasets independently negating comparability plus two author-stated benchmark gaps; and 6.7 (reproducibility) is the weakest, where most of the expected evidence is type-II silence and must not be claimed.

6.1 Peripheral-aware test generation

Observed evidence. Peripheral dependence is *the most consistently verified hardware gap* in the RQ9 matrix: 4/6 studies acknowledge it (EA), each in its own words — ADFEmu (S17) abstracts DMA only and admits that identifying how any peripheral "can be abstracted for dynamic analysis purposes, in a hardware-independent and firmware-compatible way" is "an interesting research topic that we would like to pursue in the future" [L11]; Abtahi & Azim (S13) generates networking-protocol code (MQTT) "with only minimal detection and parsing" [L9]; HardSecBench (RQ9-P5) finds open-source models show "limited gains on categories involving peripherals and interface/I-O" [L11, qualified]; and the P2IM-lineage study (RQ9-P6) describes peripheral heterogeneity, differing access conventions and incompatible fuzzer input interfaces as challenges "unique to MCU firmware fuzzing" [L2, narrowed]. On the type-I side: no specific peripheral class — SPI, Ethernet, DMA, interrupt controllers, Flash/EEPROM, device drivers, HAL APIs — was assessed as a dependency in any of the six RQ4 studies (each 0/6, though specific peripheral *types* are simply unreported in 3/6, S05/S13/S14); a formal peripheral specification reached model context in only 1/8 RQ3 studies (S06, EmbedAgent); and among the studies whose LLM role is genuinely test generation, confirmed hardware context is 0/3 (S02, S05, S12) (§4.3). The correct matrix reading matters: 4/6 does **not** mean "four of six have a peripheral problem and two do not" — it means four acknowledge it, one (Asmita et al.) explicitly reports *no* hardware-aware limitations at all, and one (RQ9-P4, the generic survey) is out of scope.

Scarcity versus reporting absence. The core of this theme is genuine method scarcity: dependency handling was assessed as absent-by-design in five of six RQ4 studies (delegated outward), and LLM-built peripheral models or device drivers number zero. The peripheral-*type* breakdown, by contrast, is partly reporting absence — 3/6 studies leave specific types unrecorded, which undercounts rather than overcounts the true handling landscape.

Why it matters, bounded. Peripheral interaction is what distinguishes embedded-C testing from general-purpose test generation; without any abstraction layer beyond the one DMA channel S17 models, generated tests cannot reach device behaviour at all, and the validation claims in Sections 4.4–4.6 remain claims about software-executing-on-software. This statement rests on within-dataset exhaustions (RQ3, RQ4) plus RQ9's EA tier; it does not depend on any cross-dataset pairing.

Directions. (a) **Author-stated:** peripheral abstraction beyond DMA — counters, comparators and similar classes — which S17 names as future work while noting it offers "no roadmap" [L11]. (b) **Review-derived:** extend S06's verified context stack (the only confirmed peripheral specification, plus board, pinout and schematic, with 13 pre-registered configurations) from hardware-adjacent *production-code* generation toward *test generation with explicit peripheral semantics*, reusing its own Wokwi-based rig — the lowest-risk starting point in the corpus because the context pipeline already exists. Precondition: the fidelity measure of 6.2, or the new tests repeat the verified pattern of "handled but never validated as faithful" (quantitative fidelity assessment: 0/27 configurations).

6.2 Register and datasheet grounding of model context

Observed evidence. Eleven categories of hardware specification — register map, MCU/processor specification, board documentation, clock configuration, memory map, electrical/interface, sensor/actuator, communication-protocol, interrupt, timer and DMA specifications — are each **0/8** as confirmed LLM inputs (RQ3), and the three retrieval combinations (datasheet+RAG, register map+HAL, HAL+driver+peripheral specification) are all zero: whatever the five RAG-using studies retrieved, it was exclusively software artefacts (legacy tests, repository structure, build logs, dependency lists, known-good `#include` directives). The two studies that did expose datasheets are instructive exceptions that prove the rule: S08 (H2LooP) puts them into the model through *training-time* weight consolidation over a 117-vendor corpus, not inference-time context, and does not generate tests; S11 (SpecMap) uses datasheets for specification traceability rather than test authoring. On the acknowledgment side, registers are an author-stated limitation in exactly **one** study (P2IM-lineage [L3], "a limitation of our register categorization method"); ADFEmu's RAM-stored DMA-descriptor gap and Abtahi & Azim's MMIO abstraction are SI adjacent items, not author statements (§4.9 composition notes; V6 pending for the adjacent gap row G5).

Scarcity versus reporting absence. RQ3 itself draws the line: three of eight studies carry *no hardware-context record at all* despite working on hardware-dependent code, so the honest wording is "no evidence that context was provided", not "confirmed not provided". The 0/8 figures are therefore exhaustive over what the dataset can show — strong as type-I under-exploration, weak as claims about what models could do.

Why it matters, bounded. Registers are the substrate on which peripherals, interrupts and timing are actually expressed in C; their absence from context means the generation machinery operates one level above the hardware semantics the review's title asks about. The direction of convergence noted in §5.2–5.4 — hardware documents absent from context (RQ3) and, in a disjoint population, interrupt models deliberately extracted from binaries rather than manuals (RQ5 Trend 3, 4/4) — is a *structural contrast*, and we do not upgrade it to a causal story.

Directions. (a) **Review-derived, with live counter-hypothesis:** run the comparison the dataset never instantiates — datasheet/register-map material supplied as *inference-time retrievable context* versus the artefact-extraction route versus training-time consolidation, on the same target board and same test suite. The counter-hypothesis must be preserved, not waved away: extraction-from-artefact is an explicit design choice in cluster B (S16 "explicitly avoiding device-manual dependence"), and if manuals are unavailable, version-mismatched or unparseable, grounding may not pay — which is exactly what no verified study has measured. (b) **Precondition (review-derived):** define a quantitative virtual-to-physical fidelity measure first; RQ4 records that no fidelity measure appears anywhere in the dataset and that this dimension is "most consistently reported … and the least consistently measured" — without a metric, "faithful grounding" is unfalsifiable. Terminology caution retained: the RAG-coding conflict between RQ2 (5/7) and RQ3 (3/8) is unresolved (V4), so stability of the "datasheet+RAG = 0" cell depends on that adjudication.

6.3 Interrupt, asynchronous and timing behaviour

Observed evidence. Within RQ5 (n = 6), the modelling machinery exists without the generator: interrupts are present in 5/6 targets; ISR handling, interrupt injection, NVIC simulation and temporal assertions at 4/6 each — but **LLM-driven interrupt/async/timing test generation is 0/6**, nested interrupts and interrupt priorities were tested by **0/6** (the priority mentions in S16/S19 were struck

by verification as inference, making this a strict zero), and time itself is inherited from emulators (5/6 virtual-time-as-side-effect) while fake/mock clocks, deterministic simulation-time mechanisms and clock-domain models are each **0/6**. "Timeout" is reported 6/6 with six different referents and validated as a firmware property **0/6** (Trend 5). Real-time property measurement is 1/6 for WCET, jitter, deadline validation and race-condition testing alike, concentrated in S13 — whose own interrupt latencies are "configured but not tested" [EA, L10] and which concedes virtualisation "cannot fully capture timing behavior" [L1]. S13's semantics gap is acknowledged at study level (G4/G5 at 3/6 each, EA tier — with the G5 attribution warning V6), and the only real-hardware interrupt testing in the corpus (S16, hardware as standard of truth) sits outside the environment dataset entirely.

Scarcity versus reporting absence. Three sub-cases must be read differently: (i) genuine method scarcity (no LLM has been pointed at interrupt-test generation in any verified study); (ii) *scope-setting*, not incapability (RQ5-P1/S15 explicitly excludes nested and tail interrupts — a boundary of that study, not evidence against the construct; the verification files even bar reading P2IM's out-of-scope items as future work, "'Out of scope' does not automatically equal future work"); and (iii) *terminology artefact* — the timeout numbers are hostage to six incompatible definitions, so the zero partly reflects our classification lens. Against over-cleansing: the concurrency/scheduling zeros are **not** absences of concurrency handling — RQ5 records Concurrency 2/6, explicit scheduling control 2/6 (one of them input-delivery scheduling, not OS scheduling), and semaphore/lock/race/mutex at 1/6 each; the accurate claim is *OS-level scheduling semantics unmodelled*, and the RTOS/concurrency literature-silence (0/6 discussed as limitation) remains type II and is not claimed as a gap.

Why it matters, bounded. Interrupts and timing are the named constructs of this review's subtitle. The verified picture is that the community can *represent* interrupt models (four ways, from binaries) but has never asked an LLM to *test* them, and can *measure* real-time behaviour in exactly one virtualised study that says the virtualisation is insufficient. If the field's ambition is hardware-aware test authoring, this is where the machinery and the generator currently fail to meet.

Directions. (a) **Author-stated, strongest in the dataset:** input-duration-aware testing — the P2IM-lineage study calls the fact that "neither P2IM nor existing fuzzers consider input duration" "an open challenge for future research" [L6, fully supported], which converts one of the trend-4/5 zeros into a named target. (b) **Review-derived:** use the interrupt artefacts cluster B already produces (S15's symbolic-execution IRQ model, S16's binary-extracted interrupt-triggering rules, S19's Interrupt Model Table) as *structured inputs to an LLM test generator*, targeting ISR sequences, then the verified zeros of nesting and priorities; start on the four studies that already have NVIC simulation. Precondition: unify the timeout and time-control vocabulary before new counts are published, or the next synthesis inherits the same six-way referent split. (c) **Review-derived:** controllable virtual clocks (fake/mock clock, deterministic simulation time) as mechanisms, so delay-duration and periodic execution become *tested properties* rather than emulator by-products — with the honest caveat from Trend 4: if a firmware's correctness depends only on event ordering, clock control's marginal value is limited, which is precisely an open empirical question no verified study answers.

6.4 Realistic execution validation (hardware-in-the-loop and physical targets)

Observed evidence. This theme carries the highest study-count in the RQ9 matrix, with a double count that must travel together: **5/6** studies acknowledge that execution validation is present but never hardware-grounded (EA tier), while only **2/6** (S17, S13) lack real-device evaluation as a *verified* gap — the P2IM-lineage study explicitly *did* re-execute crash- and hang-inducing inputs on physical MCUs to confirm bugs, and our verification files bar listing "lack of real hardware evaluation" against it. The type-I evidence converges from three disjoint directions: LLM-generated tests combined with a simulator, emulator, HIL rig or real hardware number **zero studies** in RQ4; real hardware is a *primary* execution environment in **0/6** RQ6 studies, appearing at all in only 2/6 and then only for confirmation or baseline, with five studies "explicitly avoid[ing] HIL and market[ing] hardware-independence as a design goal"; and **0/4** RQ8 studies validate with real hardware or HIL (§4.4, §4.6, §4.8). The one reverse datapoint in the corpus sharpens rather than softens the gap: the two RQ1 HIL studies execute LLM-generated artefacts on genuine industrial benches at 100% execution rate (dSPACE SCALEXIO, RTICANMM, real ECUs) — but the generated objects there are system-level test steps, not C-level test code, and S04, the only study emitting C test code, abandoned Zephyr-level attempts as "often unusable without extensive manual modification".

Scarcity versus reporting absence. The scarcity is real (type I, exhaustively verified across RQ4/RQ6/RQ8), but two framings must be quarantined. First, HIL-as-limitation is a **type-II** zero (0/6; "the single cleanest void… 'Not reported' in all six files") and cannot be converted into "the literature ignores HIL" — especially given RQ1's live HIL usage. Second, hardware-independence is *marketed as a*

design goal in cluster B; the missing HIL papers may partly reflect deliberate positioning (and venue economics), not collective oversight. A third, narrower absence is reporting-level: RQ6 records that compilation of generated tests is reported **nowhere** in its dataset — a pure reporting gap sitting underneath the validation gap.

Why it matters, bounded. Everything the review calls "hardware-aware" ultimately needs a hardware-behavioural verdict: today's verified verdicts come from emulators whose fidelity nobody measured (6.2) and whose time is inherited (6.3). The gap matters because it leaves the review's own construct without its final element — not because we may assert that emulator-only validation is wrong, but because no verified study has yet shown what changes when the same tests run closer to the metal.

Directions. (a) **Author-stated:** physical-platform validation on STM32, ESP32 and ARM Cortex-M boards — Abtahi & Azim's named plan [L12] — and P2IM-style real-device bug confirmation as an existing, replicated template (S20) for how to close at least part of the loop. (b) **Review-derived, and the section's top structural priority:** produce the first study that instantiates all four construct elements at once — LLM generation of embedded-C tests, hardware-informed context, execution on a hardware-relevant substrate (peripheral-modelled emulator, HIL, or board), and a complete generated → compilable → executed → passed funnel. Under the current overlap structure (14 of 28 pairs at zero), only such a study can create within-study pairing; single-dimension additions cannot. Reusing verified artefacts — S17's ten firmware samples, S20's device set, S04's C-level test material, S05's stub pipeline — is the lowest-variance route, and the negative evidence from S04 should size expectations (assisted, human-gated authoring, not autonomy).

6.5 Closed-loop feedback: a composition gap, not a method gap

Observed evidence and retention judgement. Checked against the dataset, "closed-loop feedback" fails the criteria for a classical gap and is retained in a narrower form: automatic closed-loop refinement is *saturated* (5/5 in RQ7; one-shot, manual, semi-automatic and mixed strategies each 0/5), and where iteration was quantified it helped — ~40% of S24's successful generations come from refinement; S26's loops lift pass@1 from 71.65% to 80.85%. The verified shortfalls are three, and each is distinct: **(i) composition** — only 2/5 feedback studies target embedded firmware at all (S24 is Python; S25/S26 Java), so the saturation statement mostly describes general-purpose test generation; **(ii) signal breadth** — the loops close on execution results (5/5), LLM corrections (5/5), test failures (4/5), compiler errors (3/5) and coverage (3/5), while hardware/peripheral feedback is 1/5 (S17's DMA channel emulation), runtime traces 2/5, sensor outputs 0/5 and compilation *warnings* 0/5; and **(iii) downstream reporting** — the two embedded studies report no iteration budget (0/2), one of them stopping on a qualitative criterion ("until a stable, secure, and real-time-compliant firmware iteration is achieved"). The oracle problem sits directly under this theme in the RQ9 matrix: 3/6 acknowledge it (EA; strongest at RQ9-P4's "persistent test-oracle problem"), and it is the only gap in the RQ9-derived evidence set with a demonstrated leak — a real PLC bug that fired under test "went undetected by the simple memory-error detector" (P2IM-lineage [L8]). All three of its supporting studies (RQ9-P4, RQ9-P5, RQ9-P6) lie outside the 28-study corpus.

Scarcity versus reporting absence. Missing hardware-adjacent feedback is method scarcity; missing loop budgets is reporting absence (the loops ran; their stop conditions were not written down); and the effectiveness of loops *on embedded targets specifically* is neither — it is simply untested, since loop frequency is not loop validity and the quantified gains come from the non-embedded majority of the dataset.

Directions. (a) **Author-stated:** additional coverage instrumentation as explicit future work (S13 [L8]); metamorphic and other additional test oracles as the survey's named remedy (RQ9-P4 [L2], coded "one of the strongest RQ9 findings"). (b) **Review-derived:** extend verified loop architectures to hardware-adjacent signals the emulators already emit — MMIO/NVIC/register-state traces, DMA events, WCET/jitter/deadline measurements — treating oracles at three explicit layers (syntactic, functional, hardware-behavioural) rather than pass/fail. Two evidence-based cautions: LLM-as-a-judge is a *verified-unreliable* oracle candidate ("inconsistent across runs", RQ9-P5 [L7, Part]), and metamorphic testing, though author-proposed, has no in-dataset evaluation — the first study adopting it should be designed to produce that evaluation, not assume it.

6.6 Standardised benchmarks and metrics

Observed evidence. Two disjoint datasets independently negate comparability. RQ8 (n = 4): no common benchmark, no common baseline, no common quantified metric, no identical evaluation environment (each 0/4); fault seeding, fault exposure, human

evaluation and expert review each 0/4; no study quantifies coverage *and* mutation score together; and where a metric name recurs across studies, definitions and units differ, which the source treats as barring any pooling. RQ1 (n = 4): four mutually incompatible execution-accounting regimes, no complete funnel anywhere, and its verdict that quantitative meta-analysis "is impossible". The proxy-decoupling evidence — the AutoHIL ablation separating executability (89.15%) from functional correctness (50.51%) and accuracy (46.10%); S02's 84.5% runtime validation against branch/line coverage (43%/67%) below the human baseline (76%/93%); MuTAP dropping coverage for "weakly correlated with fault detection" — comes from these same two populations and is reported with the multi-version caveat where the source figures conflict (§4.11). Author acknowledgment exists at study level too: no widely recognised benchmark for unit-test generation and pretraining leakage inflating results (RQ9-P4 [L3, L4]); lack of a standardised, comprehensive benchmark and the need for security-specific evaluation (HardSecBench [L2, L16]).

Scarcity versus reporting absence. All three coexist and must be told apart: method scarcity (nobody has built a shared embedded-C test-generation benchmark in the corpus); reporting absence (funnel stages and denominator definitions left unstated across datasets); and *internal inconsistency* — RQ8's own summary carries nine count conflicts on non-zero magnitudes (its zero rows, which this section relies on, are unaffected) — which is a property of this review's extraction, not of the literature. One overclaim guard applies: "benchmarks are missing" is scoped to *embedded-C test generation*; HardSecBench itself (924 validated tasks, 4,425 test cases) proves that large benchmark construction is feasible and expensive in this adjacency.

Why it matters, bounded. Metrics are the arbitration layer for 6.1–6.5: until execution success, coverage and fault detection are defined and reported compatibly, the field cannot say which of the missing combinations above would actually help, and this review could not lawfully compute any pooled rate.

Directions. (a) **Review-derived, consolidating four author-stated asks:** a shared benchmark for embedded-C test generation that pins target MCU family, compiler/toolchain versions, dependency infrastructure, and a complete four-stage funnel definition, uses code unseen during pretraining [RQ9-P4 L4], and instruments coverage *together with* mutation/fault seeding so the decoupling claims become testable rather than anecdotal. (b) **Author-stated:** security-specific evaluation as a mandatory layer alongside functional refinement loops [RQ9-P5 L16], motivated by the corpus's verified finding that functional refinement yields little security gain and independent generations converge on similar weaknesses.

6.7 Reproducibility (retained, with the weakest evidential footing)

Observed evidence and retention judgement. This theme is retained only in a disciplined form. The *only* reproducibility-related limitation verified at study level in RQ9 is closed-source model dependency (1/6; S13's GPT-4 reliance [A14]) — under the single-study rule, an observation, not a field-level gap. The expected evidence for broader reproducibility claims is type-II silence: "missing prompts/datasets/source code/hardware settings/benchmark details" are each discussed as limitations by 0/6 files, and P2IM-lineage's "manual architecture-specific model" item was *removed* in verification because the paper states the design extends to AVR, MIPS and RISC-V — so this section does not claim the literature fails to report its artefacts. What can be stated are type-I reporting-completeness facts about our own corpus: LLM identity and prompting are each unreported in ≥1 configuration for 5/7 RQ2 studies; 6/7 are arXiv preprints with one peer-reviewed venue; DOIs are absent across most datasets; year is unrecorded for 9/28 studies; and for 18 of the 28 studies the full-text evidence in our files is per-record verification documents rather than publisher PDFs (§4.11 N3).

Why it matters, bounded. These completeness gaps limit what *anyone* can later verify: retrospective cross-study tests of "which strategy, which validation outcome" cannot even be assembled when the strategy field is partly blank, and independent re-runs of the reviewed pipelines are frequently impossible as published. That is a statement about auditability, not about authors' lab hygiene.

Directions. (a) **Review-derived, folded into 6.6:** the shared benchmark should require artefact disclosure as an entry condition — model and version pinned per configuration, prompts or their hashes, toolchain versions, and the execution substrate — turning completeness into a publication incentive rather than an exhortation. (b) **Author-side, requiring no new literature:** adjudicate the eight pending verification items V1–V8 before extending the corpus [review-derived from C3/C5/C6]. V1 (whether R4_p4 and Abtahi & Azim are one study) decides whether the two research clusters are connected at all; its failure would downgrade every cross-cluster statement in Sections 5–6 by one strength level, which is why this zero-cost step is placed ahead of most new data collection.

6.8 Limitations of this review's own search scope, attached to the gaps above

Each gap above inherits the same boundary conditions, and they should travel with any citation of this section:

- **Every zero is lens-bound.** Each 0-count means "not observed within the verified dataset under our classification scheme", never "no such study exists". The dataset itself shows the lens moves counts: the same QEMU/Unicorn tooling is coded "simulator 6/6" in one extraction and "simulator 0/6" in another (D-2, pending V3); "mock" names both a unit double (S05) and a system-level simulation verb (S14); "coverage" denotes fault-scenario coverage in one RQ1 study; and "feedback" covers both evaluation-only measurement and refinement input (the reason S13's coverage use is coded Not reported, not Yes, in RQ7).
- **Terminology reach.** The extraction organised studies around the review's RQ vocabulary. Adjacent literatures the records do not show as covered — re-hosting as a named term, cyber-physical or model-in-the-loop test generation, digital-twin testing — could populate cells currently at zero; no claim of exhaustive coverage of those vocabularies is made, and none of the gap statements above should be read as one.
- **Small denominators.** RQ study sets run 4–8; RQ9's study denominator is 6 with instance counts (75 retained, 30.7% downgraded) never substituting for studies; a single study moves RQ1 percentages by 25 points and RQ2 percentages by 14.3. All prevalence language in this section is bounded to its own denominator, and no cross-dataset rate is computed anywhere.
- **Unresolved conflicts, unadjudicated.** The nine RQ8 internal count conflicts, the RAG-coding difference (V4), the simulator terminology split (V3), the interrupt-gap attribution (V6), the HIL-label sensitivity in RQ4 (D-9: one study's classification moves HIL popularity 33.3%→16.7%), and the preserved multi-version source figures (D-8) are reported as conflicts, not resolved; directions keyed to them (6.2, 6.3, 6.7) carry that conditionality explicitly.
- **No causal claims.** Because 14 of 28 study-set pairs share zero studies, statements linking dimensions across RQs remain structural contrasts throughout Sections 5–6; the gap inventory says what has never been *jointly observed*, not what would fail if jointly attempted.

6.9 Closing orientation

Ranking below is our editorial judgement (review-derived), ordered by what each repair adds to the *structure* of the evidence rather than by the size of any single deficiency: (1) the four-element instantiation study of 6.4, the only design that creates within-study pairing; (2) the zero-cost V1–V8 adjudication of 6.7; (3) the benchmark-and-funnel instrument of 6.6, which turns proxy decoupling into a testable question; (4) the fidelity measure and layered oracles of 6.1–6.2 and 6.5, without which faithfulness and validity claims in this domain cannot be stated at all; then the domain-specific build-outs — hardware-spec grounding trials (6.2), interrupt-aware generation on existing models and controllable clocks (6.3), hardware-adjacent loop signals (6.5), and beyond-DMA peripheral abstraction (6.1). Each item is traceable, in the subsections above, to a specific verified count, a named study and an evidence grade; none is offered as established necessity.

---

7. Threats to Validity

This section assesses the threats to the validity of this systematic review across four dimensions: internal validity (search and selection), construct validity (classification and extraction), external validity (generalizability), and conclusion validity (synthesis and metrics). The assessment is based strictly on the documented audit trail of the review process.

7.1 Search coverage and publication bias

Threat. The primary threat to internal validity is the lack of systematic documentation for the original search strategy. As documented in the SLR Search Audit Report (§A.2), the only confirmed search activity with a verifiable timestamp is a DeepSeek export from 2026-10-12 containing 59 excluded records. No logs exist for database queries (e.g., Scopus, IEEE Xplore, Web of Science), search strings,

result counts, or date filters for the initial identification phase. Consequently, the "upper half" of the PRISMA 2020 flow diagram (Identification → Screening → Retrieval) is reconstructed from author working files rather than observed from search-engine exports.

Effect. The review's coverage of the literature cannot be independently verified or reproduced. It is unknown whether relevant studies published in venues not surfaced by the specific (but undocumented) discovery route were missed. The potential for publication bias is substantial given the corpus's preprint skew (§7.8). The audit explicitly bars the claim that traditional databases were systematically searched: publication of a study in a database-indexed venue is not evidence that the database was searched.

Mitigation.

- **Completed:** The Search Audit Report was produced to transparently map the available evidence and to explicitly flag every missing search-metadata item (search strings, per-database counts, dates, filters — audit List B, ten items). A Screening Register was retrospectively constructed assigning every record a unique ID (REC-001…REC-105), an exclusion code where applicable (EC-1…EC-6, covering all 59 excluded records) and a population tag, yielding an auditable list of what was screened even where the per-paper "Discovery route" column reads "NOT RECORDED".
- **Not performed:** The supplementary search across Scopus, Web of Science, IEEE Xplore, ACM DL and arXiv proposed in the audit (§5.1, "Priority 🔴") has **not been executed** as of this draft. The review therefore relies on the corpus available at audit time; coverage-related findings should be read as descriptive of this specific, documentation-limited dataset.

7.2 Selection and screening bias

Threat. The screening process was not documented prospectively. The register records that reviewer identity, screening dates and the adjudication stage (title/abstract vs full text) were not recorded for the included records; there is no inter-rater agreement data. The 59 excluded and 46 included records derive from the author's workbooks, and for RQ1/RQ2 in particular the audit (§B.3) finds no exclusion records at all, with file naming (`rq_1 find N.pdf`) hinting at a discovery process whose method is unconfirmed.

Effect. Selection bias cannot be quantified. It is unknown whether inclusion reflects systematic screening or a corpus of "known" papers; if the latter, the evidence base over-represents the authors' immediate research circle and search-engine surface behaviour.

Mitigation.

- **Completed:** The Screening Register and Study-Level Final Determinations table were constructed retrospectively and then adjudicated: the final determinations carry the column "FINAL full-text determination (author-accepted DP defaults 1A-6A, 2026-10-12)", meaning each record's status was re-checked against the author-approved default policy (six decision points covering grey literature, non-LLM infrastructure, non-embedded comparators, indirect-testing relevance, and cohort demarcation) rather than inherited silently. Zero records were excluded at the full-text stage; instead, 12 non-LLM studies were retained as *demarcated* Contextual/Comparator cohorts and 3 as Gray-zone, each with its counting rule recorded — this prevents scope creep while preserving transparency about the origin of every study.
- **Not performed:** Independent double screening, reviewer calibration and inter-rater reliability statistics (e.g., Cohen's Kappa) were not performed and are not claimed.

7.3 Terminology variation and classification ambiguity

Threat. The included studies and the per-RQ extractions use inconsistent terminology for core constructs, threatening construct validity:

- **Simulator vs emulator:** the RQ5 extraction codes "Simulator" 6/6 while RQ6 codes Simulator 0/6 for the same tooling (QEMU, Unicorn), classifying them strictly as emulators (cross-synthesis D-2; the RQ5 row label was truncated in parsing, so the discrepancy cannot be resolved from the files).
- **Mock:** in RQ4, "mock" denotes a unit-test double in S05 (openSIL) but is the source paper's own verb for system-level vehicle-state simulation in S14 (P26).

- **Coverage:** in RQ1, "test coverage" in study 2511.19132v1 means fault-scenario coverage, not code coverage.
- **Feedback:** RQ7 separates evaluation-only measurement from refinement-loop input (S13's coverage is used as an evaluation index, not a loop, hence "Not reported" rather than "Yes").
- **HIL label ambiguity:** one RQ4 study's HIL classification is ambiguous (D-9); excluding it would move RQ4's HIL and real-hardware prevalence from 33.3% to 16.7%.

Effect. Cross-study category counts are sensitive to classification rules; a zero count may reflect a strict definition rather than an empirical absence. Terminology-driven counts cannot be merged across RQs (e.g., no simulator figure may be pooled between RQ5 and RQ6 pending adjudication).

Mitigation.

- **Completed:** The revised cross-RQ synthesis adopts a "preserve and report, never silently harmonize" policy: an eleven-entry Discrepancy Log (D-1…D-11) records every unresolved conflict with both sides and their source lines; Results, Discussion and Gaps sections quote each count within its own dataset's classification and flag terminology-dependence wherever a number is load-bearing; the type I / type II zero distinction (method-use zeros vs limitation-discussion zeros) constrains what any zero may be used to claim.
- **Not performed:** No adjudication of these conflicts by a second coder; eight items remain formally pending author verification (V1–V8).

7.4 Missing or non-original full text

Threat. A significant portion of the "full text" evidence is derivative. The PRISMA audit concluded that for 22 of the 46 included records (covering all 14 studies of the RQ5–RQ8 cluster) the local PDFs are author-generated verification documents created 2026-10-08/09 via print-to-PDF pipelines — they share embedded-font signatures with known verification artifacts, and the "original PDF filename" values recorded for these rows do not exist on disk. Twenty-four records (RQ1–RQ4 folders) are confirmed publication originals.

Effect. Extraction for the 22 affected records relies on secondary artifacts rather than publisher-verified originals. If a verification document itself contains an error, downstream extraction inherits it without an independent anchor. Page references in these sources may be approximate.

Mitigation.

- **Completed:** The distinction is recorded in the register ("Local full text resolved path"), disclosed in the Results data notes (N3) and in the PRISMA reporting ("consultation evidence" per study; 28/28 studies were demonstrably read at page level via quoted page/section citations). The verification documents contain per-item quotes, page/section positions and verification verdicts, so every extracted claim traces to a specific location in the artifact used.
- **Not performed:** Re-retrieval of the publisher originals for the 22 derivative records, and re-verification of extracted claims against them.

7.5 Inconsistent study reporting and metric incompatibility

Threat. Primary studies report data inconsistently, and metrics are often incomparable:

- **Internal contradictions in sources:** R2_p4's Top-1 appears as 60.73% and 60.75%; its query count as 3,208 and 1,700; R2_p7's success rates as 14%, 16% and 66% in different places; RQ1's source reports 88.65% vs 94.98%, and an accept/modify/rewrite split summing to 100.1% (D-8).
- **Metric heterogeneity:** RQ8 records no common benchmark, baseline, quantified metric or evaluation environment (each 0/4), and states that studies sharing a metric *name* differ in definition and denominator, barring pooling; RQ1 records four mutually incompatible execution-accounting regimes and declares effect-size synthesis "impossible".

- **Extraction-side conflicts:** RQ8's own summary table contains nine internal count conflicts on non-zero magnitudes (D-3), and some source tables were damaged in parsing (corrupted repeated values; a truncated RQ9 gap table), limiting per-study attribution (D-11).

Effect. Quantitative synthesis is impossible for most dimensions; prevalence claims are constrained by small denominators; load-bearing source figures carry multi-version uncertainty.

Mitigation.

- **Completed:** Conflicts are preserved, logged and side-reported (D-3, D-8, D-11), never resolved by selection; the review's own Results use RQ8's verified classification tables (whose zero rows — the ones cited downstream — are unaffected by the nine non-zero conflicts); every percentage is bounded to its own denominator (Registry Table 4) and no cross-dataset pooling is performed anywhere; multi-version figures are quoted only with the preserved-contradiction caveat.
- **Not performed:** Manual re-adjudication of the nine RQ8 conflicts and the damaged-table attributions against the source PDFs (scheduled as author action V5/V6, not yet done).

7.6 Multiple reports of the same study and deduplication

Threat. The 46 included records collapse to 28 unique studies: 18 records are cross-RQ re-loggings of studies already present in another RQ dataset, and one study (S26 LLMLOOP) has two reports (conference version plus preprint). The register annotated these clusters retrospectively; no prospective de-duplication log exists, and the PRISMA-side count "duplicate records removed: 0" reflects annotated re-loggings rather than a dedup run.

Effect. Without study-level collapsing, prevalence statistics would double-count studies appearing in several RQ datasets, inflating apparent evidence for overlapping dimensions.

Mitigation.

- **Completed:** The Study-Level Final Determinations table assigns a unique Study ID (S01–S28) to every record cluster, with Records(n) summing exactly to 46; all study-level counts in Results de-duplicate by study, configuration-level counts retain all records, and the two levels are never mixed; the cross-RQ overlap matrix and the "reports = 29" audit (28 studies + 1 documented version pair) are built on this registry.
- **Not performed:** Bibliographic fuzzy-duplicate detection across the screened 105 records (dedup was by normalized-title clustering within the register, recorded as such).

7.7 LLM-assisted extraction and synthesis errors

Threat. Extraction and synthesis for the verified datasets were performed with LLM assistance. LLMs can hallucinate, misread technical details, or collapse the distinction between "Not reported" and "No" — the exact failure that would fabricate gap claims from silence.

Effect. If uncorrected, extraction errors would propagate into every RQ count and therefore into the Discussion and Gaps sections.

Mitigation.

- **Completed:** Each RQ report carries per-study verification tables and explicit coding rules ("Not reported", "Unclear", "No" kept distinct throughout); the RQ9 layer adds a second-order verification in which 4 of 75+4 extracted items were rejected and 23 of 75 retained items (30.7%) were downgraded by the verification files themselves — direct evidence that the check layer overrode first-pass coding; inference-based codings were struck (e.g., NVIC-capability mentions "removed as inference"); and prohibition rules from the files ("absence cannot be converted into a research gap", "'Out of scope' ≠ future work") are enforced at the claim level.

- **Not performed:** Independent re-extraction of any record by a non-LLM second extractor, or a formal error-rate study of the extraction pipeline.

7.8 Limited generalizability

Threat. The corpus is small (28 studies; the LLM-construct population on which the review's core claims rest numbers 13), heavily preprinted (RQ2: 6/7 arXiv, 1 peer-reviewed venue; DOIs absent across most datasets; one included grey-literature thesis), and concentrated in two sub-communities (firmware fuzzing/re-hosting; automotive/HIL test generation). The review's headline construct — hardware-aware LLM test generation for embedded C — is instantiated end-to-end by **no** study in the corpus (RQ9 construct-validity finding).

Effect. Findings describe this verified dataset, not the field at large; generalizing to "how the literature does X" would be unsupported, and the preprint-heavy composition further weakens stability (unreviewed, potentially superseded results).

Mitigation.

- **Completed:** Every RQ report's self-qualification against prevalence claims is quoted and obeyed (e.g., RQ5: "licenses no prevalence claim about the broader SLR population"); cohort demarcation (LLM-construct vs Contextual vs Comparator vs Gray-zone) fixes which population each statistic describes, and LLM-population statements use N = 13, never 28; all gap statements are worded as "unobserved within the verified dataset" with the search-scope caveat carried in the Gaps section (§6.8); the grey-literature thesis is used under a descriptive-only restriction.
- **Not performed:** Snowballing or citation-chasing to test corpus completeness (proposed in the audit's logging templates; not executed).

7.9 Scope statement

The four threat classes above bound every claim in this review: search documentation gaps constrain coverage claims (§7.1–7.2); classification and artifact issues constrain construct claims (§7.3–7.4); reporting heterogeneity constrains conclusion claims (§7.5–7.6); extraction-assurance limits constrain error bounds on all counts (§7.7); and the dataset's size and composition constrain all generalization (§7.8). Where the records are silent on a procedural safeguard (registration, double screening, inter-rater agreement, sensitivity analysis), this section states that silence rather than implying the safeguard existed.

---

8. Conclusion

8.1 Purpose and method

This review set out to characterise, from verified primary evidence, how large language models are being used for hardware-aware test generation in embedded C — specifically what test types they produce, which strategies and contexts they use, how hardware and peripheral dependencies are handled, how interrupts, asynchronous behaviour and timing are addressed, where generated tests are executed and validated, which feedback loops refine them, how results are measured, and what limitations the literature itself acknowledges (RQ1–RQ9).

The review proceeded from a documented corpus of **105 screened records**, of which **59 were excluded** (56.2%) under six recorded exclusion codes and **46 were included** (43.8%); the included records collapse to **28 unique studies** corresponding to **29 reports**. Cohorts were demarcated rather than merged: **13 LLM-construct studies (46.4%)**, 8 Contextual non-LLM (28.6%), 4 general-SE comparators (14.3%) and 3 Gray-zone (10.7%), with LLM-population statements always drawn from the 13-study cohort. Each research question was answered from its own independently verified extraction dataset (denominators 4, 7, 8, 6, 6, 6, 5 and 4 studies for RQ1–RQ8; 6 studies for the derived RQ9 analysis), and every percentage is valid only inside its own denominator. Where the

primary sources conflict, the conflicts are reported side by side and adjudicated by neither this review nor its readers' convenience; where a category is empty, the emptiness is reported as "not observed within the verified dataset", never as absence from the literature. The identification stage of the search was reconstructed from author working files rather than observed from database exports, and this constraint is disclosed throughout.

8.2 Principal findings across RQ1–RQ8

Test types are bimodal, with the middle levels empty. Reported activity concentrates at unit level (2/4, 50.0%) and at HIL/system level (2/4, 50.0%); integration (0/4) and regression (0/4) are absent, and the two absences differ in kind — integration was explicitly ceded to mature SIL/MIL/PIL toolchains, while regression is total silence that supports no conclusion in either direction. Only one of four studies emits C-language test code (25.0%); the rest produce Python or JSON artefacts.

Strategy work is real but aimed at software artefacts. Zero-shot prompting is near-universal (6/7, 85.7%), RAG is common (5/7, 71.4%), agentic workflows appear in 4/7 (57.1%), generation-LLM fine-tuning in only 1/7 (14.3%), and chain-of-thought as an explicit technique in no configuration. All seven studies are multi-model; no single model is used by more than two studies. Reporting completeness is the strategy side's own weak point: model identity and prompting technique are each unreported in at least one configuration for 5/7 studies (71.4%).

"Hardware awareness" is currently a software-artefact concept. Software context reaches the model in 8/8 studies (100%), while any embedded-hardware context appears in 5/8 (62.5%), low-level hardware documentation in 3/8 (37.5%), datasheets at inference time in 2/8 (25.0%) and a formal peripheral specification in 1/8 (12.5%). Register maps, MCU specifications, board documentation, clock and memory-map configuration, electrical/interface, sensor/actuator, communication-protocol, interrupt, timer and DMA specifications are each confirmed as model input in **0/8** studies; HAL/driver/BSP/RTOS artefacts reach inference-time context in 0/8; and among the three studies whose LLM role is genuinely test generation, confirmed hardware context is **0/3 (0%)**. Execution feedback (compiler errors, build/flash logs, coverage, HIL action results) is present as context in 4/8 (50.0%) and functions, in the source's own terms, as a stand-in for hardware awareness.

Dependency handling is delegated, not generated. In 5/6 studies (83.3%) the LLM produces no dependency-handling artefact at all; instead 19/27 configurations (70.4%) have it generate hardware-related *production* code. The two handling families — provision (4/6, 66.7%) and replacement (2/6, 33.3%) — never intersect, and hybrid strategies number 0/6. Exactly one unambiguous positive cell exists: LLM-generated tests with stubs, executed on host. Behaviourally faithful doubles (register, timing, interrupt or protocol semantics) appear in **0 of 27 configurations**, and quantitative simulator/emulator fidelity assessment appears nowhere in the dataset.

Interrupt semantics are modelled without LLMs; time is inherited, never controlled. Asynchronous and timeout-related behaviour are reported in 6/6 studies (100%), interrupts are present in 5/6 targets (83.3%), and ISR handling, interrupt injection, NVIC simulation, temporal assertions and delay handling each reach 4/6 (66.7%). Real-time property measurement is concentrated in a single study: WCET, jitter, deadline validation and race-condition testing are each 1/6 (16.7%), contributed by the same study, whose own interrupt handling is unreported. LLM-driven interrupt/asynchronous/timing test generation is **0/6**; nested interrupts and interrupt priorities were tested by 0/6; fake or mock clocks, deterministic simulation-time mechanisms and clock-domain modelling are each 0/6. Time is a side effect of running an emulator in 5/6 studies, and although all six report timeout-related behaviour, firmware timeout is tested as a functional constraint in none. OS-level scheduling semantics are unmodelled (0/6 each for scheduler modelling, task switching, thread interleaving and message/event queues), which must be read alongside the non-zero concurrency counts (concurrency 2/6, explicit scheduling control 2/6, semaphore/lock/race each 1/6).

Execution is emulator-bound. Emulator, host, hybrid and software-only validation each apply to 6/6 studies (100%), with QEMU dominant in 5/6 (83.3%). Real hardware appears in 2/6 (33.3%) and only in secondary roles — bug confirmation and baseline comparison; HIL appears in 1/6 (16.7%), again only as a baseline, while five studies explicitly avoid HIL and market hardware-independence as a design goal. Real hardware is the *primary* execution environment in 0/6 studies; simulators (strict term) and virtual platforms are 0/6; and no study in the dataset reports the compilation of its generated tests.

Feedback loops are closed, but on software signals. Automatic closed-loop refinement is universal (5/5, 100%), with one-shot, manual, semi-automatic and mixed strategies at 0/5. Loops close on execution results (5/5), LLM-generated corrections (5/5), test failures (4/5, 80.0%), compiler errors (3/5, 60.0%) and coverage (3/5), while hardware/peripheral feedback reaches only 1/5 (20.0%) and sensor outputs none. Embedded firmware is the target in only 2/5 studies (40.0%), and neither embedded study reports an iteration budget.

Evaluation cannot arbitrate between these positions. Execution (4/4) and benchmark comparison (4/4) are universal; compilation, mutation testing, simulation/emulation and coverage analysis each appear in 2/4 (50.0%). Real hardware, HIL, fault seeding, fault exposure, human evaluation, expert review and statement/branch/function/condition coverage are each **0/4**, as are common benchmarks, common baselines, common quantified metrics and identical evaluation environments; no study quantifies coverage and mutation score together. Two disjoint datasets independently record that the usual proxies dissociate: RQ1 finds four mutually incompatible execution-accounting regimes and no complete funnel, and RQ8 records that where a metric name recurs across studies its definition and denominator differ, which is why the source bars pooling. Within RQ1 the evidence also separates executability from test validity, and within RQ8 one study dropped coverage on the grounds that it is weakly correlated with fault detection. No study anywhere in the corpus reports a complete generated → compilable → executed → passed funnel, and quantitative meta-analysis is therefore impossible rather than merely unattempted.

The corpus is two clusters that do not meet. Of the 28 pairwise study-set overlaps, **14 are zero**, 12 share 1–3 studies, and only 2 share four or more (RQ2×RQ3 = 6; RQ2×RQ4 = 4). The datasets carrying all strategy and context evidence share no study with those carrying all interrupt, environment, feedback and metric evidence, and the single documented bridge between the clusters rests on a feature-level identity match that remains pending verification. Consequently no study in the verified corpus is simultaneously LLM-based, test-generating, embedded-C-source-level and hardware-aware.

What the evidence does and does not license about LLM effectiveness. No overall verdict is supported, and this review offers none. The verified record is specific and bounded on both sides. On the positive side, all four RQ1 studies report executing their generated artefacts (4/4), RQ7 finds automatic closed-loop refinement universal (5/5, 100%) with quantified gains reported wherever iteration was measured, and RQ8 records execution-based validation in every study (4/4). On the negative side, the RQ1 sources use four mutually incompatible execution-accounting regimes and none reports a complete generated → compilable → executed → passed funnel; the single study that produced C test code for an RTOS codebase abandoned the effort, its output described as often unusable without extensive manual modification; and RQ8 finds no common benchmark, baseline, quantified metric or evaluation environment (each 0/4), so no study's result can be checked against another's. These statements describe particular configurations in particular datasets; neither the positives nor the negatives scale to a field-level claim that LLMs are, overall, effective or ineffective for hardware-aware embedded-C test generation.

8.3 Most important verified limitations and gaps (RQ9)

At study level (denominator 6; the 75 retained limitation instances are never used as prevalence, and one study alone contributes 13 of them, 17.3%, while counting once per category), the highest-support author-acknowledged limitations are: **execution validation is present but never hardware-grounded, 5/6 (83.3%)**, reported alongside the narrower verified count that only 2/6 lack real-device evaluation as a confirmed gap, because one study did re-execute crash-inducing inputs on physical devices; and **LLM capability is real but narrow and merely advisory, 5/6 (83.3%)**. Next is **peripheral/interface dependence, 4/6 (66.7%)** — the most consistently verified hardware gap across the RQ9-derived six-study limitation set (of which only two are corpus members, S17 and S13; the other four are RQ9-side works). At 3/6 (50.0%) each: the unresolved test-oracle problem, the only gap with a demonstrated empirical leak, in which a real bug fired under test and went undetected by the simple memory-error detector; timing behaviour unmodelled at key points, of which the strongest is the finding that neither the study nor existing fuzzers consider input duration, named by its authors as an open challenge; incomplete interrupt semantics; weak register/configuration-layer handling; missing or limited DMA support; human involvement not eliminated; and a platform and generalisability ceiling. At 2/6 (33.3%): limited RTOS/concurrency handling, and driver complexity — with HAL explicitly *not established* as a distinct verified limitation. Two single-study observations (1/6 each) are recorded as such and not generalised: functional correctness not implying security correctness, and closed-source model dependency.

Three features of this evidence limit how strongly any of it may be stated. Verification downgraded 23 of the 75 retained instances (30.7%) and rejected 4 outright; several apparently hardware-related items are inferences from silence rather than author statements; and a set of commonly assumed LLM weaknesses — hallucination, incorrect or invented APIs, named reasoning limitations — is discussed as a limitation by **0/6** studies, as are formal threats-to-validity sections, statistical analysis, HIL and HAL. Because these are silences in limitation discussion rather than absences of method use, they support no gap claim; what they do establish is that the verified LLM-side weaknesses in this literature are concrete and different from the field's usual assumptions — compilation failures attributed largely to missing context and to model logical inconsistency rather than to fabrication, LLM-as-a-judge verdicts unreliable across runs, independent generations converging on functionally correct designs with similar security weaknesses, and an LLM used only as a heuristic guide whose poor suggestions do not block exploration.

On the method-use side, the corpus exhaustively verifies as absent the combinations that a hardware-aware pipeline would need: LLM-generated tests executed on a simulator, emulator, HIL rig or real hardware (0 studies); LLM-built peripheral models, device drivers, HALs, wrappers or configured execution environments (0); behaviourally faithful doubles and quantitative fidelity assessment (0/27 and 0 respectively); LLM generation of interrupt, asynchronous or timing tests (0/6); and hardware specifications in model context (0/8 across eleven categories). Each of these is a statement about the verified dataset under this review's classification lens, and each carries the alternative explanations recorded in §6 — hardware independence as a marketed design goal, delegation to mature infrastructure as a deliberate engineering choice, artefact-based interrupt-model extraction as a preferred route rather than a fallback, and scope boundaries that bound a study's claims without bounding the construct's feasibility.

8.4 Implications for future research

The evidence points to a small number of directions, each traceable to a specific verified finding and each carrying its precondition.

First, produce studies that instantiate the whole construct. Because 14 of 28 pairwise overlaps are zero, only a study that simultaneously generates embedded-C tests with an LLM, supplies hardware-informed context, executes on a hardware-relevant substrate and reports a complete funnel can convert today's cross-dataset contrasts into within-study evidence. Reusing verified artefacts and rigs already in the corpus is the lowest-variance route, and the one documented C-level failure should size expectations toward assisted, human-gated authoring rather than autonomy.

Second, close the measurement gaps before the method gaps. A quantitative virtual-to-physical fidelity measure does not exist anywhere in the dataset, although fidelity is the dimension most consistently reported as a limitation and least consistently measured; without it, "hardware-aware" and "faithful" remain unfalsifiable. Likewise, a shared benchmark for embedded-C test generation must pin MCU family, toolchain versions, dependency infrastructure and the four-stage funnel definition, use code unseen during pretraining, and instrument coverage together with mutation or fault seeding so that the proxy dissociation recorded in two disjoint datasets becomes testable rather than anecdotal.

Third, put hardware semantics inside the loop. The verified machinery for interrupt modelling already exists in four studies and has never been pointed at LLM test generation; nested interrupts, priorities and ISR sequences are the named zeros. Time needs to become controllable — fake or mock clocks and deterministic simulation time — so that delay duration, periodic execution and timeout become tested properties instead of tool budgets, with terminology unified first, since six studies currently report "timeout" with six different referents. Context grounding should be tested as an explicit comparison: hardware specifications supplied at inference time versus extraction from the artefact versus training-time consolidation, with the possibility preserved that artefact extraction is the better route. Feedback loops, already saturated as a technique, need hardware-adjacent signals (register, MMIO/NVIC state, DMA events, timing measurements) and oracles distinguished at three layers — syntactic, functional and hardware-behavioural — while treating LLM-as-a-judge as a verified-unreliable candidate.

Fourth, do the zero-cost work first. Eight verification items remain pending, and one of them decides whether the two research clusters are connected at all; its failure would downgrade every cross-cluster statement in this review by one strength level. Resolving these before extending the corpus is the only improvement in evidential strength available without new literature.

Finally, three framings the evidence does not support and that future work should avoid: treating hallucination reduction as this field's bottleneck (the verified failure profile is context loss, logical inconsistency, judge instability and convergent weaknesses);

claiming that the literature ignores HIL, HAL or RTOS (all type-II silences, with live HIL usage recorded elsewhere in the corpus); and justifying model-capacity work by prevalence statistics this corpus cannot lawfully produce.

8.5 Scope of these conclusions

Everything above is bounded by the review's own conditions of evidence. Counts describe datasets of 4 to 8 studies and an RQ9 denominator of 6, so single studies move percentages by 12.5 to 25 points and no prevalence claim about the wider literature is licensed. Every zero means "not observed within the verified dataset under this review's classification scheme"; the records show that the lens moves counts, with the same tooling coded as simulator in one dataset and emulator in another, "mock" naming both a unit double and a system-level simulation verb, "coverage" denoting fault-scenario coverage in one study, and "feedback" covering both evaluation-only measurement and refinement input. Search documentation is incomplete at the identification stage, terminology reach is bounded by the review's own vocabulary, 22 of 46 records rest on author-generated verification documents rather than publisher originals, and nine internal count conflicts in one dataset's summary remain unadjudicated. Within those bounds, the verified picture is coherent and specific: a field with mature generation machinery on the software side, mature hardware-modelling machinery on the infrastructure side, and no verified study yet connecting the two — a structure consistent with a pre-paradigm intersection rather than with a failing literature.

References

Citation style. Numbered, in order of first citation. Entries [1]–[22] preserve the order established in Section 1; [23]–[36] were added during the citation audit. A numbered style is used deliberately: author lists are recorded in the project files for only 6 of the 36 entries, so an author–year style cannot be produced from the verified records without fabrication.

Transcription rule. Every bibliographic field below is transcribed from a named project record. `[NOT RECORDED]` means no supplied source states that field; such fields were **not** filled from general knowledge, and no external lookup (DOI resolver, arXiv API, web search) was performed. `[CONFLICT – Cn]` points to the metadata conflict register in Appendix C, where both recorded values are printed side by side and neither is adjudicated. Study identifiers (S01–S28) are those of Table 4 and Appendix C; cohort labels are those of Table 3.

Corpus boundary. Part A entries are the 28 included studies and no others. Part B entries are works cited in this manuscript that are *not* members of the reviewed corpus; they must never be counted toward the 28 studies, the 46 records or any cohort denominator. Part C entries are the reporting standards consulted for this review's format.

Part A — The 28 included studies (S01–S28)

[1] (S02, LLM-construct) EmbC-Test: How to Speed Up Embedded Software Testing Using LLMs and RAG. arXiv:2603.09497. Authors (per the 119-row reference workbook): M. Harnot, S. Komarnicki, M. Polok, T. Oksanen. Year: [CONFLICT — C1]. Venue: arXiv preprint.

[2] (S05, LLM-construct) Library-Aware Doubles and Iterative Repair for Large Language Model-Generated Unit Tests in OpenSIL Firmware. Authors (per the 119-row reference workbook): M. T. Bach, Y. Zheng, H. Razafindranto, T. Alam, A. Leather, R. Sandhu, J. Arora. Year 2026 (accepted 2026-04-16). Venue: [NOT RECORDED in the supplied document]. Identifier: [CONFLICT — C2].

[3] (S03, LLM-construct) AutoHIL: LLM-Based ECU Functional Test Generation through Domain Knowledge Augmentation. *Proc. ACM Softw. Eng.* **3**, ISSTA, Article ISSTA064, 2026. DOI 10.1145/3832155. Authors: [NOT RECORDED].

[4] (S04, LLM-construct, grey literature) Execution-path-aware prompting of Large Language Models for test-case generation of embedded system code. Master's thesis, Hamburg University of Technology (TUHH), 22 December 2025. No DOI stated. The author

surname "Tran" is [NOT RECORDED in any data field — derivable only from the local file name `tran-mt25.pdf`]. Admitted under decision default DP-5A with a descriptive-only synthesis restriction.

[5] (S07, LLM-construct) Smarter, not Bigger: Fine-Tuned RAG-Enhanced LLMs for Automotive HIL Testing. Year: [CONFLICT — C3]. Venue: [NOT RECORDED in the supplied document]. Identifier: [CONFLICT — C3]. Authors: [NOT RECORDED].

[6] (S08, grey zone) H2LooP Spark Preview: Continual Pretraining of Large Language Models for Low-Level Embedded Systems Code. Year: [CONFLICT — C4]. Venue: [NOT RECORDED in the supplied document]. Identifier: [CONFLICT — C4]. Authors: [NOT RECORDED]. Admitted under DP-4A (IC-6, indirect relevance).

[7] (S09, LLM-construct) EmbedGenius: Towards Automated Software Development for Generic Embedded IoT Systems. Year: [CONFLICT — C5]. Venue: [NOT RECORDED in the supplied document]. Identifier: [CONFLICT — C5]. Authors: [NOT RECORDED].

[8] (S13, LLM-construct) Securing LLM-Generated Embedded Firmware through AI Agent-Driven Validation and Patching. arXiv:2509.09970, 2025. Venue: arXiv preprint. Authors: [NOT RECORDED in the screening register; the pair "Abtahi & Azim" is attested only in the RQ5/RQ6/RQ7/RQ9 synthesis documents]. Identity versus record R4_p4 is the feature-level match **M2**, pending author verification (Appendix D, U3).

[9] (S15, Contextual non-LLM) AidFuzzer: Adaptive Interrupt-Driven Firmware Fuzzing via Run-Time State Recognition. 34th USENIX Security Symposium, 2025. DOI: [NOT STATED as a DOI in the supplied document]. Authors: [NOT RECORDED].

[10] (S16, Contextual non-LLM) IEmu: Interrupt modeling from the logic hidden in the firmware. *Journal of Systems Architecture* **154**:103237, 2024. DOI 10.1016/j.sysarc.2024.103237. Authors: [NOT RECORDED].

[11] (S10, LLM-construct) Exploring and Characterizing Large Language Models for Embedded System Development and Debugging. Authors (per the register's Details field): Englhardt et al. Year: [CONFLICT — C6]. Identifier: [CONFLICT — C6]. Venue: [NOT RECORDED in the supplied document].

[12] (S19, Contextual non-LLM) AIM: Automatic Interrupt Modeling for Dynamic Firmware Analysis. arXiv:2312.01195, 2023. Venue: arXiv preprint. Authors: [NOT RECORDED].

[13] (S20, Contextual non-LLM) P2IM: Scalable and Hardware-independent Firmware Testing via Automatic Peripheral Interface Modeling. 29th USENIX Security Symposium, 2020. DOI: [NOT STATED]. Authors: [NOT RECORDED]. The extended version is listed separately at [33]; it is a second *report* of this study, not a 29th study.

[14] (S21, Contextual non-LLM) HALucinator: Firmware Re-hosting Through Abstraction Layer Emulation. 29th USENIX Security Symposium, 2020. DOI: [NOT STATED]. Authors: [NOT RECORDED].

[15] (S22, Contextual non-LLM) Fuzzware: Using Precise MMIO Modeling for Effective Firmware Fuzzing. 31st USENIX Security Symposium, 2022. DOI: [NOT STATED]. Authors: [NOT RECORDED].

[16] (S23, Contextual non-LLM) DICE: Automatic Emulation of DMA Input Channels for Dynamic Firmware Analysis. 2020. Venue: [CONFLICT — C11: the recorded cell reads "USENIX Security Symposium paper lineage stated in supplied PDF", which is a provenance note rather than a venue string]. DOI: [NOT STATED]. Authors: [NOT RECORDED].

[17] (S24, General-SE comparator) CoverUp: Effective High Coverage Test Generation for Python. arXiv:2403.16218. Year: [CONFLICT — C7]. Venue: arXiv preprint. Authors: [NOT RECORDED]. Admitted under DP-3A; non-embedded target, therefore not counted as an embedded-C finding.

[18] (S25, General-SE comparator) LLM Test Generation via Iterative Hybrid Program Analysis. arXiv:2503.13580, 2025. Venue: arXiv preprint. Authors: [NOT RECORDED]. The work name "Panta" is attested in the register's Notes column. Admitted under DP-3A.

[19] → moved to Part B (outside the included corpus; see Appendix D, U7).

[20] (S06, LLM-construct) EMBEDAGENT: Benchmarking Large Language Models in Embedded System Development. 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), 12–18 April 2026, Rio de Janeiro, Brazil. DOI 10.1145/3744916.3787788. Authors: [NOT RECORDED].

[21] → moved to Part B (outside the included corpus; see Appendix D, U7).

[22] (S26, General-SE comparator) LLMLOOP: Improving LLM-Generated Code and Tests through Automated Iterative Feedback Loops. 2025 IEEE International Conference on Software Maintenance and Evolution (ICSME), 2025. DOI 10.1109/ICSME64153.2025.00109. Authors: [NOT RECORDED]. **Version pair:** the preprint arXiv:2603.23613 (2026) is a second *report* of this same study. This pair is why reports = 29 while studies = 28. Admitted under DP-3A.

[23] (S27, General-SE comparator) Effective test generation using pre-trained Large Language Models and mutation testing. *Information and Software Technology* **171**:107468, 2024. DOI 10.1016/j.infsof.2024.107468. Authors: [NOT RECORDED]. Admitted under DP-3A. **Name caution:** this study is referred to in the manuscript prose as "MuTAP", a work name that is not attested in any bibliographic field of the project records (Appendix D, U4).

[24] (S01, LLM-construct) LLMs-Powered Real-Time Fault Injection: An Approach Toward Intelligent Fault Test Cases Generation. arXiv:2511.19132, 2025. Venue: arXiv preprint. Authors: [NOT RECORDED].

[25] (S11, grey zone) SpecMap: Hierarchical LLM Agent for Datasheet-to-Code Traceability Link Recovery in Systems Engineering. Year: [NOT RECORDED]. Venue: [NOT RECORDED]. DOI: [NOT RECORDED]. Authors: [NOT RECORDED]. Admitted under DP-4A. **Every bibliographic field for this entry is absent from the project records; it cannot be completed from them.**

[26] (S14, LLM-construct) Automating a Complete Software Test Process Using LLMs: An Automotive Case Study. Year: [NOT RECORDED]. Venue: [NOT RECORDED]. DOI: [NOT RECORDED]. Authors: [NOT RECORDED]. Identity versus record R3_p8 is the feature-level match **M1**, pending author verification (Appendix D, U3).

[27] (S17, LLM-construct) ADFEmu: Enhancing Firmware Fuzzing with Direct Memory Access (DMA) Input Emulation Using Concolic Execution and Large Language Models (LLMs). *Computers, Materials & Continua* **84**(3), 2025. DOI 10.32604/cmc.2025.065672. Authors: [NOT RECORDED]. Appears in the RQ5, RQ6, RQ7, RQ8 and RQ9 datasets.

[28] (S18, Contextual non-LLM) Stop Starving or Stuffing Me: Boosting Firmware Fuzzing Efficiency with On-demand Input Delivery. DOI 10.1145/3545948.3545957. Year: [CONFLICT — C10: not explicitly stated on the supplied first page; recoverable from the DOI resolver, which was not consulted]. Venue: [NOT EXPLICITLY STATED]. Authors: [NOT RECORDED]. **Name caution:** referred to in the prose as "FIDO", a short form not attested in the registry title (Appendix D, U4).

[29] (S28, LLM-construct) Closed-loop evaluation of LLM agents for embedded software development. *Journal of Systems Architecture* **179**:103937, 2026. DOI 10.1016/j.sysarc.2026.103937. Authors: [NOT RECORDED].

[30] (S12, grey zone) **[cannot be cited]** The registry title for this study reads literally "Not reported in provided excerpt"; year, venue, DOI and authors are all [NOT RECORDED]; and the local file is CID-encoded and not machine-readable with the available tooling. This study is one of the 28 included studies and one of the 3 grey-zone studies, so it **is** counted in every relevant denominator, but no reference entry can be constructed for it without fabrication. It is therefore enumerated here as an explicitly uncitable entry and referred to in the prose only as "one grey-zone study (S12, identity unresolved)". Author action required: Appendix D, U1.

Part B — Works cited that are not members of the reviewed corpus

These five entries are cited in the manuscript but are **not** among the 28 included studies. None may be counted toward the 28 studies, the 46 records, the 29 reports or any cohort denominator.

[19] (non-corpus; entered the project via the RQ10 dataset, absent from the 105-record screening register) Guzu et al., Large Language Models for C Test Case Generation: A Comparative Analysis. *Electronics* **14**(11):2284, 2025. DOI 10.3390/electronics14112284. Cited in Section 1 as adjacent comparative work.

[21] (non-corpus; RQ9-P4) Software Testing With Large Language Models: Survey, Landscape, and Vision. *IEEE Transactions on Software Engineering* **50**(4), 2024. DOI: [NOT STATED on the supplied first page]. Authors: [NOT RECORDED]. Cited in Section 1 as the broad survey and used as one of the six RQ9 evidence studies. **Not an RQ1–RQ8 included study.**

[31] (non-corpus; RQ9-P3) Asmita et al., Fuzzing BusyBox: Leveraging LLM and Crash Reuse for Embedded Bug Unearthing. 2024. Venue: [CONFLICT — C12: the recorded cell reads "USENIX Security 2024 (as identified by the supplied final extraction/paper context)", a self-labelled derived attribution rather than a recorded venue]. DOI: [NOT STATED]. One of the six RQ9 evidence studies; contributes 10 of the 75 retained limitation instances.

[32] (non-corpus; RQ9-P5) HardSecBench: Benchmarking the Security Awareness of LLMs for Hardware Code Generation. arXiv:2601.13864, 2026. Venue: arXiv preprint. Authors: [NOT RECORDED]. One of the six RQ9 evidence studies; contributes 18 of the 75 retained limitation instances. **Scope note:** the synthesis records its object as security-aware hardware/firmware *code generation*, not test generation, so it is outside the RQ1–RQ8 test-generation corpus on two counts.

[33] (second report of S20) P2IM: Scalable and Hardware-independent Firmware Testing via Automatic Peripheral Interface Modeling — extended version. arXiv:1909.06472, 2019. Venue: arXiv preprint / extended version. This is the evidence base for RQ9-P6. It is a *report* of study S20, not a 29th study; listing it separately is what makes the report layer traceable, and it must not be added to any study count.

Part C — Reporting standards consulted for this review's format

Consulted for reporting format only, as recorded in Section 3.0. These are methodological sources for the review's own presentation; they are not evidence about the reviewed subject and are not counted in any denominator.

[34] PRISMA 2020 statement and flow-diagram templates. URL as recorded in the project prompt library: `https://www.prisma-statement.org/prisma-2020` and `https://www.prisma-statement.org/prisma-2020-flow-diagram` . Cited in Section 3.0 as (Page et al., 2021). Full author list, article title, journal, volume and pages: [NOT RECORDED in the project files — only the URLs and the in-text short form are recorded].

[35] PRISMA 2020 explanation and elaboration (records, reports and studies; study-selection reporting). URL as recorded in the project prompt library: `https://www.bmj.com/content/372/bmj.n160` . Full author list, article title, volume and pages: [NOT RECORDED in the project files beyond this URL].

[36] PRISMA-S extension for reproducible search reporting. URL as recorded in the project prompt library: `https://www.prisma-statement.org/prisma-search` . Full author list, article title, journal and year: [NOT RECORDED in the project files beyond this URL].

Cross-check

Part A carries 28 distinct study identifiers (S01–S28, with [30] enumerated but uncitable), matching the study-level determinations table exactly. Part B carries 5 entries, none of which is a study identifier. Part C carries 3 entries. **28 studies + 5 non-corpus works + 3 reporting standards = 36 entries.** No entry is duplicated. S20 appears once as a study ([13]) and once as a distinct report ([33]); that is the documented reports = 29 / studies = 28 relationship made explicit, and [33] is never added to a study count.

What this reference list deliberately does not contain. (i) No bibliographic field was filled from general knowledge: for well-known works in this corpus (P2IM, HALucinator, Fuzzware, DICE, AidFuzzer, the IEEE TSE survey, Guzu et al.) authors, page ranges and DOIs could have been supplied from memory and were not, because a memory-supplied DOI is precisely the failure mode that makes a reference list look verified while being wrong. (ii) No external lookup was performed — no DOI resolver, no arXiv API, no web search; conflict C10 in particular is recoverable in one query and was left unrecovered on purpose. (iii) No reference was added because it seemed relevant: the list contains exactly the works the manuscript cites plus the three reporting standards Section 3.0 names. Notably absent, and deliberately so, are standard systematic-review methodological sources that no section of this manuscript actually cites; adding them would require first citing them in Section 3. (iv) No metadata conflict was resolved by choosing a side: all twelve are printed with both values in Appendix C.

Appendix A — Search protocol, search log and reporting-gap disclosure

A.1 What the project records establish about the search

This appendix exists because the review's search documentation is incomplete, and reporting that incompleteness precisely is a scientific obligation rather than a formality. Three statements summarise the position, each traceable to the search-history audit and the screening register:

1. **Exactly one discovery route has retained evidence.** An AI-tool (DeepSeek) export dated **2026-10-12**, held as `deepseek_csv_20261012_f6f391.txt` : 59 records across five columns (Category, Title, Year, Venue/DOI, Reason for Exclusion). The export contains **excluded records only**. No query string, no result count and no per-source provenance were retained. Fifty-nine of the 105 register rows carry this route verbatim in their "Discovery route (evidence)" column.
2. **The discovery route of the 46 retained records is NOT RECORDED.** The register's own qualifier on every one of those rows is "local PDF presence = custody only, not route." File-name patterns (`rq_1 find N.pdf` , `rq_2 find_N.pdf`) hint at a discovery process, but the 11 corresponding files are extraction and verification documents — each opens with a "PAPER IDENTIFICATION" block — and carry no source or "found via" field.
3. **No bibliographic database search is documented.** A full project scan found no `.ris` , `.bib` or database export of any kind. The strings Scopus, Web of Science, IEEE Xplore, ACM Digital Library, DBLP and PubMed occur **zero times across 105 register rows × 22 columns**. IEEE (×17) and ACM (×14) do occur in the venue column, but those denote the publication venues of the included papers, not platforms that were searched; publication in a database-indexed venue is not evidence that the database was searched. No systematic database search is therefore claimed anywhere in this manuscript.

Registers: not applicable. No trial or preprint register is in scope for software-engineering literature, and register-related keywords return zero hits across the 21 project texts examined. The PRISMA registers branch is reported as not applicable rather than as 0 (Figure 1).

Identification total: unknown. The figure 105 counts records *registered and adjudicated*, not records *identified*. It is therefore a lower bound on the identification total, whose upper bound is unknown. No "N records identified" figure is reported for any N.

A.2 Supplementary search: proposed, and not executed

The search-history audit proposed a supplementary search to remedy the missing log. **It was not executed**, and the project contains no execution record for it. The proposal is reproduced below verbatim and in full, so that a reader can see exactly what was proposed and confirm that nothing in this manuscript depends on it having been run. Every query is labelled **PROPOSED QUERY** in the source, and the source's own record templates carry unfilled placeholders reading "[to be completed after execution]".

If the authors did execute any of these searches, Section 3.3 and Figure 1 must be updated together with dates, strings and result counts before submission; until then the manuscript reports only what the records establish.

A.2.1 Sources proposed, with the audit's own priority labels

Source	Coverage as described in the audit	Audit priority
Scopus	Comprehensive; covers mainstream SE conferences and journals	mandatory
Web of Science Core Collection	Comprehensive; high-quality journals	mandatory
IEEE Xplore	Core for SE and embedded systems	mandatory
ACM Digital Library	Top SE venues (ICSE, FSE, ASE and similar)	mandatory

Source	Coverage as described in the audit	Audit priority
SpringerLink	Embedded and real-time systems	recommended
arXiv (cs.SE, cs.AI)	Preprints; most recent work	recommended
DBLP	Computer-science bibliography; supplementary	optional

A.2.2 Concept decomposition recorded in the audit

Concept group	Keywords as recorded
A — technique / method	LLM, large language model, GPT, ChatGPT, Claude, code generation, test generation
B — target domain	embedded, firmware, IoT, microcontroller, MCU, hardware-aware
C — programming language	C, C++, embedded C
D — test type	unit test, integration test, fuzzing, test case generation, test automation
E — hardware feature	peripheral, interrupt, ISR, DMA, MMIO, register, hardware dependency

A.2.3 Proposed query strings (verbatim; never executed)

Scopus — search field TITLE-ABS-KEY (title, abstract, keywords):

```
TITLE-ABS-KEY (
  ( "large language model" OR "LLM" OR "GPT" OR "ChatGPT" OR "code generation" )
  AND ( "embedded" OR "firmware" OR "microcontroller" OR "IoT" )
  AND ( "test generation" OR "unit test" OR "fuzzing" OR "test automation" )
)
AND LIMIT-TO ( DOCTYPE , "ar" OR "cp" )
AND LIMIT-TO ( LANGUAGE , "English" )
```

Web of Science — search field TS (topic = title + abstract + keywords):

```
TS=(
  ("large language model" OR "LLM" OR "GPT" OR "ChatGPT" OR "code generation")
  AND ("embedded" OR "firmware" OR "microcontroller" OR "IoT")
  AND ("test generation" OR "unit test" OR "fuzzing" OR "test automation")
)
AND DOCUMENT TYPES: ( ARTICLE OR PROCEEDINGS PAPER )
AND LANGUAGES: ( ENGLISH )
```

IEEE Xplore — search field All Metadata:

```
("All Metadata":"large language model" OR "All Metadata":"LLM" OR "All Metadata":"GPT")
AND ("All Metadata":"embedded" OR "All Metadata":"firmware")
AND ("All Metadata":"test generation" OR "All Metadata":"fuzzing")
```

ACM Digital Library — search field All fields:

```
( "large language model" OR "LLM" OR "GPT" )
AND ( "embedded" OR "firmware" )
AND ( "test generation" OR "fuzzing" )
```

arXiv — search fields cat + abs:

```
(cat:cs.SE OR cat:cs.AI) AND (
  abs:"large language model" OR abs:"LLM"
) AND (
  abs:"embedded" OR abs:"firmware"
) AND (
  abs:"test" OR abs:"fuzzing"
)
```

The audit's per-search record template — database, platform, date, time, query string verbatim, search field, filters, total results, exported count, export format and export path — remains unfilled for every source. No date restriction, language restriction or publication-type filter is recorded as having been applied to the actual corpus, and this manuscript states neither that one was applied nor that none was.

A.3 Consequences for reproducibility, stated plainly

- PRISMA-S-style per-source query reporting is **not possible** for this review as documented. This is a limitation of the underlying process, not of this write-up (§3.3, §7.1).
- The review's coverage of the literature cannot be independently verified or reproduced from the records. Whether relevant studies in venues not surfaced by the undocumented route were missed is unknown (§7.1).
- The screening register was constructed retrospectively on 2026-10-12 by auditing already-existing per-RQ workbooks and verification documents dated 2026-10-08 to 2026-10-09. Eligibility was adjudicated against criteria finalised after dataset assembly, under author-accepted decision defaults (§3.0, §3.4, §3.6). This is retrospective reconstruction, and it is reported as such rather than as a pre-specified procedure.
- No protocol registration exists. The review is reported as unregistered (§3.0).
- Reviewer identity and screening dates are recorded as NOT RECORDED on all 105 rows; single- versus multiple-reviewer arrangements, consensus procedures and any inter-rater agreement measurement are undocumented, and no reviewer count or κ statistic is reported (§3.6, §7.2).

Appendix B — Eligibility criteria and decision defaults as recorded

B.1 Status of the criteria

The inclusion criteria were formulated as **PROPOSED** criteria during the 2026-10-12 audit — explicitly *not* described as the historical criteria used to assemble the original per-RQ datasets — and were then applied to the already-registered records. The register preserves this status verbatim on all 46 included rows: "not yet re-screened against the PROPOSED criteria — decision PENDING author approval". The final determinations were subsequently executed under **author-accepted decision defaults DP-1A to DP-6A, dated 2026-10-12**; the accepted status is recorded in the determination column header itself, which is the authoritative field (§3.6).

The complete wording of criteria IC-1 through IC-6 is not stored in the project records. Only the three codes below appear in row-level annotations, with their operative role recoverable from those annotations. Reconstructing the full criterion texts requires author input (Appendix E, U11). Likewise, of the six accepted decision defaults, only DP-1A, DP-3A, DP-4A and DP-5A are cited on records (8, 6, 4 and 1 times respectively); the texts of DP-2A and DP-6A are not present in any file.

B.2 Inclusion-criterion codes attested on records

Code	Operative role as evidenced	Where attested
IC-1	Publication type / grey literature. Applied to S04, a master's thesis; grey literature is admissible under the DP-5A restriction (descriptive-only synthesis).	S04 determination and notes
IC-5	LLM-construct eligibility: the study itself uses an LLM for test generation or test support. Every included LLM-construct record carries the tag "LLM-construct candidate (IC-5 met by this record)". Failures of IC-5 were **demarcated rather than excluded**, under DP-1A / DP-3A.	Register rows across the LLM-construct cohort
IC-6	Directness of testing relevance. Studies whose testing relevance is indirect — context mechanisms or model substrates rather than test generation — are grey-zone under DP-4A.	S08, S11, S12 determinations

Full text available on file was the operative retrieval condition: no record was lost to retrieval failure (§3.4, Figure 1).

B.3 Decision defaults actually cited on records

Default	Times cited	Effect as recorded
DP-1A	8	Non-LLM firmware infrastructure is demarcated into the Contextual cohort rather than excluded; used for RQ4–RQ6 environment framing only.
DP-3A	6	General-SE LLM test generation on non-embedded targets is demarcated into the Comparator cohort rather than excluded; calibrates RQ7/RQ8 mechanisms.
DP-4A	4	Indirect testing relevance (IC-6) is retained as Grey zone with flags rather than excluded; evidence kept verbatim.
DP-5A	1	Grey literature (the S04 master's thesis) is admitted with a descriptive-only synthesis restriction.
DP-2A	not cited on any record	Text not present in any project file.
DP-6A	not cited on any record	Text not present in any project file.

Demarcation rather than exclusion is the operative principle: no verified evidence was discarded, and no non-LLM study could inflate an LLM statistic, because cohort membership fixes the denominator for every claim (Table 3).

B.4 Exclusion criteria and reason codes (all 59 excluded records)

All 59 screened-out records carry a substantive scope reason; **zero** exclusions were attributed to retrieval failure (paywall and unobtainable phrasing return zero hits across the register). The six reason codes EC-1 to EC-6, their definitions as recorded in the register, and their counts (30 / 9 / 5 / 5 / 5 / 5, summing exactly to 59) are given in **Table 1 (§3.5)** and are not repeated here. Three points of provenance belong in this appendix.

First, the codes are **retrospective mappings** of the twelve recorded category labels. The register marks them "(PROPOSED code)" and, where the mapping was reconstructed after the fact, "(retroactive code)". The category-to-code mapping is 1:1 in the reverse direction — each category maps to exactly one code and no category splits — so the codes add no information beyond the original category labels.

Second, the twelve underlying category labels and their counts, as rendered in Figure 15, are: non-embedded unit-test generation 8; blockchain/smart contract 5; education 5; hardware verification 5; mobile app development 5; non-embedded fuzzing 5; testing LLMs themselves 5; web development 5; blockchain consensus verification 4; database/text-to-SQL 4; enterprise code review / IaC 4; game development 4. EC-1 covers six of these categories and EC-3 covers two; the remaining four codes map one-to-one.

Third, Figure 15 is framed as **screening-side context only**. These are category labels of records excluded before eligibility, not PRISMA full-text exclusion reasons; the full-text exclusion count is 0 (Figure 1, Appendix B.6 and §3.6), and the two must not be conflated.

B.5 A documented asymmetry between the RQ datasets

RQ1 and RQ2 have no exclusion records at all. Their datasets (4 and 7 included studies respectively) entered the project without a documented rejected-candidate pool, unlike RQ3–RQ8, whose candidates were screened against the reasons in Table 1. The search-history audit explicitly flags that it "cannot confirm whether RQ1/RQ2 papers passed systematic screening". This asymmetry is a selection-bias threat (§7.2) and requires author resolution (Appendix E, U12).

B.6 Quality assessment

No quality assessment was applied. No scoring of any included study exists in the project: there is no QA column in the register or the determinations table, and no scored rubric output anywhere in the project. What exists is a **draft rubric held as a proposal only** — dimensions QA1 to QA7: objective stated; LLM/approach stated; embedded context stated; test procedure stated; execution/validation setup stated; metrics reported; reproducibility detail — generated under an explicit instruction to label a proposed rubric as a proposal rather than as an already-used method. It has not been used to include, exclude, weight or qualify any study, and no quality-assessment results are reported anywhere in this manuscript (§3.7).

Two study-specific treatment facts substitute for quality gating where they matter: the grey-literature thesis (S04) is admitted under a descriptive-only restriction (DP-5A); and RQ8's own synthesis prohibits popularity claims because of its denominator (4 studies) and nine recorded intra-dataset count conflicts on non-zero magnitudes — a data-quality judgement embedded in the synthesis layer rather than in a QA instrument. Reporting completeness was handled per RQ through the "Not reported / Unclear / No" distinction (§2.5) rather than through a scored instrument.

Appendix C — Included-study characteristics (S01–S28)

All 28 included studies, transcribed from the study-level determinations table and the audited study-year detail. **Years** apply rule N8 (§4.11): a study is year-stated only if at least one of its records states a four-digit year, so a parenthetical mention such as "(references 2026)" is recorded as *not recorded*; missing years are never imputed. **Identifier** cells reproduce the recorded value and carry the conflict reference (Cn) where two project records disagree — both sides are printed in Appendix D and neither is adjudicated here. **Records** lists the screening-register rows that collapse into the study; the column sums to 46, which is the record-layer count, not a study count.

Cohort labels are those of Table 3 (§3.6). Reference numbers point to the References section; [30] is enumerated but cannot be cited (Appendix E, U1).

ID	Ref	Title as registered	Year	Cohort	Records (n)	Datasets	Stable identifier(s) recorded	Ruling
S01	[24]	LLMs-Powered Real-Time Fault Injection: An Approach Toward	2025	LLM-construct	1	RQ1	arXiv:2511.19132	FINAL INCLUDED

ID	Ref	Title as registered	Year	Cohort	Records (n)	Datasets	Stable identifier(s) recorded	Ruling
		Intelligent Fault Test Cases Generation						
S02	[1]	EmbC-Test: How to Speed Up Embedded Software Testing Using LLMs and RAG	Not recorded	LLM-construct	2	RQ1 & RQ2, RQ3	arXiv:2603.09497 [C1]	FINAL INCLUDED
S03	[3]	AutoHIL: LLM-Based ECU Functional Test Generation through Domain Knowledge Augmentation	2026	LLM-construct	1	RQ1	DOI 10.1145/3832155	FINAL INCLUDED
S04	[4]	Execution-path-aware prompting of Large Language Models for test-case generation of embedded system code	2025	LLM-construct (gray-lit)	1	RQ1	not recorded	FINAL INCLUDED
S05	[2]	Library-Aware Doubles and Iterative Repair for Large Language Model-Generated Unit Tests in OpenSIL Firmware	2026	LLM-construct	3	RQ2, RQ3, RQ4	not recorded [C2] — NO stable identifier recorded on any copy	FINAL INCLUDED
S06	[20]	EMBEDAGENT: Benchmarking Large Language Models in Embedded System Development	2026	LLM-construct	3	RQ2, RQ3, RQ4	DOI 10.1145/3744916.3787788; https://doi.org/10.1145/3744916.3787788	FINAL INCLUDED
S07	[5]	Smarter, not Bigger: Fine-Tuned RAG-Enhanced LLMs for Automotive HIL Testing	Not recorded	LLM-construct	2	RQ2, RQ3	not recorded [C3] — NO stable identifier recorded on any copy	FINAL INCLUDED
S08	[6]	H2LooP Spark Preview: Continual Pretraining of Large Language Models for Low-Level Embedded Systems Code	Not recorded	Gray-zone (indirect testing relevance)	2	RQ2, RQ3	not recorded [C4] — NO stable identifier recorded on any copy	RETAINED (flagged)
S09	[7]	EmbedGenius: Towards Automated Software Development for	Not recorded	LLM-construct	3	RQ2, RQ3, RQ4	not recorded [C5] — NO stable identifier recorded on any copy	FINAL INCLUDED

ID	Ref	Title as registered	Year	Cohort	Records (n)	Datasets	Stable identifier(s) recorded	Ruling
		Generic Embedded IoT Systems						
S10	[11]	Exploring and Characterizing Large Language Models for Embedded System Development and Debugging	Not recorded	LLM-construct	2	RQ2, RQ4	not recorded [C6] — NO stable identifier recorded on any copy	FINAL INCLUDED
S11	[25]	SpecMap: Hierarchical LLM Agent for Datasheet-to-Code Traceability Link Recovery in Systems Engineering	Not recorded	Gray-zone (indirect testing relevance)	1	RQ3	not recorded	RETAINED (flagged)
S12	[30]	Not reported in provided excerpt	Not recorded	Gray-zone (identity unknown)	1	RQ3	not recorded	RETAINED (flagged)
S13	[8]	Securing LLM-Generated Embedded Firmware through AI Agent-Driven Validation and Patching	2025	LLM-construct	4	RQ4, RQ5, RQ6, RQ7	arXiv:2509.09970 [C8]	FINAL INCLUDED
S14	[26]	Automating a Complete Software Test Process Using LLMs: An Automotive Case Study	Not recorded	LLM-construct	1	RQ4	not recorded	FINAL INCLUDED
S15	[9]	AidFuzzer: Adaptive Interrupt-Driven Firmware Fuzzing via Run-Time State Recognition	2025	Contextual non-LLM	1	RQ5	not recorded	FINAL INCLUDED (demarcated)
S16	[10]	IEmu: Interrupt modeling from the logic hidden in the firmware	2024	Contextual non-LLM	1	RQ5	10.1016/j.sysarc.2024.103237	FINAL INCLUDED (demarcated)
S17	[27]	ADFEmu: Enhancing Firmware Fuzzing with Direct Memory Access (DMA) Input Emulation Using Concolic Execution and Large	2025	LLM-construct	4	RQ5, RQ6, RQ7, RQ8	10.32604/cmc.2025.065672	FINAL INCLUDED

ID	Ref	Title as registered	Year	Cohort	Records (n)	Datasets	Stable identifier(s) recorded	Ruling
		Language Models (LLMs)						
S18	[28]	Stop Starving or Stuffing Me: Boosting Firmware Fuzzing Efficiency with On-demand Input Delivery	Not recorded	Contextual non-LLM	1	RQ5	10.1145/3545948.3545957 [C10]	FINAL INCLUDED (demarcated)
S19	[12]	AIM: Automatic Interrupt Modeling for Dynamic Firmware Analysis	2023	Contextual non-LLM	1	RQ5	arXiv:2312.01195	FINAL INCLUDED (demarcated)
S20	[13]	P2IM: Scalable and Hardware-independent Firmware Testing via Automatic Peripheral Interface Modeling	2020	Contextual non-LLM	1	RQ6	not recorded [C9]	FINAL INCLUDED (demarcated)
S21	[14]	HALucinator: Firmware Re-hosting Through Abstraction Layer Emulation	2020	Contextual non-LLM	1	RQ6	not recorded	FINAL INCLUDED (demarcated)
S22	[15]	Fuzzware: Using Precise MMIO Modeling for Effective Firmware Fuzzing	2022	Contextual non-LLM	1	RQ6	not recorded	FINAL INCLUDED (demarcated)
S23	[16]	DICE: Automatic Emulation of DMA Input Channels for Dynamic Firmware Analysis	2020	Contextual non-LLM	1	RQ6	not recorded [C11]	FINAL INCLUDED (demarcated)
S24	[17]	CoverUp: Effective High Coverage Test Generation for Python	2025	General-SE comparator	1	RQ7	arXiv:2403.16218 [C7]	FINAL INCLUDED (demarcated)
S25	[18]	LLM Test Generation via Iterative Hybrid Program Analysis	2025	General-SE comparator	1	RQ7	arXiv:2503.13580	FINAL INCLUDED (demarcated)
S26	[22]	LLMLOOP: Improving LLM-Generated Code and Tests through Automated Iterative Feedback Loops	2025	General-SE comparator	3	RQ7, RQ8	10.1109/ICSME64153.2025.00109; arXiv:2603.23613	FINAL INCLUDED (demarcated)

ID	Ref	Title as registered	Year	Cohort	Records (n)	Datasets	Stable identifier(s) recorded	Ruling
S27	[23]	Effective test generation using pre-trained Large Language Models and mutation testing	2024	General-SE comparator	1	RQ8	10.1016/j.infsof.2024.107468	FINAL INCLUDED (demarcated)
S28	[29]	Closed-loop evaluation of LLM agents for embedded software development	2026	LLM-construct	1	RQ8	10.1016/j.sysarc.2026.103937	FINAL INCLUDED

Column sums and cross-checks. 28 study rows; Records(n) sums to **46** (verified); datasets span RQ1–RQ8; cohorts sum to 13 LLM-construct (including the S04 grey-literature thesis) + 8 Contextual non-LLM + 4 General-SE comparator + 3 Grey zone = **28**. Rulings: 25 FINAL INCLUDED + 3 RETAINED = 28, with **0** full-text exclusions. Studies holding a publication original: S01–S10 (10); studies whose full-text evidence is derived verification documents rather than publisher PDFs: S11–S28 (18) — see §4.11 note N3 and §7.4.

C.1 Per-study flags carried into the synthesis

ID	Flag as recorded
S02	Register notes two local copies as byte-identical; that note is mis-aimed (the identical pair is the RQ_1/RQ_2 copies of EmbC-Test, not REC-011, which is a derived verification document) — wording fix pending, counts unaffected
S04	Grey literature (DP-5A): master's thesis, descriptive-only synthesis restriction
S08	Grey zone under DP-4A (IC-6, indirect testing relevance); evidence retained verbatim with flag
S11	Grey zone under DP-4A (IC-6, indirect testing relevance); evidence retained verbatim with flag
S12	Identity unresolved in the records; local full text is CID-encoded and not machine-readable — enumerated but uncitable (Appendix E, U1)
S13	Identity versus record R4_p4 is feature-level match **M2**, pending verification (Appendix E, U3); M2 is the only documented bridge between the two corpus clusters
S14	Identity versus record R3_p8 is feature-level match **M1**, pending verification (Appendix E, U3)
S15	Demarcated under DP-1A: non-LLM infrastructure; excluded from LLM-population statistics; used for RQ4–RQ6 environment framing
S16	Demarcated under DP-1A: non-LLM infrastructure; excluded from LLM-population statistics; used for RQ4–RQ6 environment framing
S18	Demarcated under DP-1A: non-LLM infrastructure; excluded from LLM-population statistics; used for RQ4–RQ6 environment framing
S19	Demarcated under DP-1A: non-LLM infrastructure; excluded from LLM-population statistics; used for RQ4–RQ6 environment framing
S20	Demarcated under DP-1A: non-LLM infrastructure; excluded from LLM-population statistics; used for RQ4–RQ6 environment framing
S21	Demarcated under DP-1A: non-LLM infrastructure; excluded from LLM-population statistics; used for RQ4–RQ6 environment framing
S22	Demarcated under DP-1A: non-LLM infrastructure; excluded from LLM-population statistics; used for RQ4–RQ6 environment framing
S23	Demarcated under DP-1A: non-LLM infrastructure; excluded from LLM-population statistics; used for RQ4–RQ6 environment framing
S24	Demarcated under DP-3A: non-embedded target; not counted as an embedded-C finding; used to calibrate RQ7/RQ8 mechanisms
S25	Demarcated under DP-3A: non-embedded target; not counted as an embedded-C finding; used to calibrate RQ7/RQ8 mechanisms

ID	Flag as recorded
S26	Demarcated under DP-3A: non-embedded target; not counted as an embedded-C finding; used to calibrate RQ7/RQ8 mechanisms; Conference-plus-preprint version pair of one study — the reason reports (29) exceed studies (28)
S27	Demarcated under DP-3A: non-embedded target; not counted as an embedded-C finding; used to calibrate RQ7/RQ8 mechanisms

Studies without a flag are FINAL INCLUDED in the LLM-construct cohort with no pending identity or provenance qualification.

C.2 Record-to-study mapping (46 records → 28 studies)

The 18 study-level collapses are cross-RQ re-registrations of the same study, **not** PRISMA duplicate-removal events; duplicate records removed is 0 (§3.6, Figure 1). Ten studies carry more than one record; eighteen carry exactly one.

Study	Records	Register rows
S02	2	REC-002/RQ1 & RQ2 R1_p2 / R2_p1; REC-011/RQ3 R3_p1
S05	3	REC-005/RQ2 R2_p2; REC-017/RQ3 R3_p7; REC-019/RQ4 R4_p1
S06	3	REC-006/RQ2 R2_p3; REC-012/RQ3 R3_p2; REC-020/RQ4 R4_p2
S07	2	REC-007/RQ2 R2_p4; REC-016/RQ3 R3_p6
S08	2	REC-008/RQ2 R2_p5; REC-014/RQ3 R3_p4 / H2LooP Spark Preview
S09	3	REC-009/RQ2 R2_p6; REC-015/RQ3 R3_p5; REC-024/RQ4 R4_p6
S10	2	REC-010/RQ2 R2_p7; REC-021/RQ4 R4_p3
S13	4	REC-022/RQ4 R4_p4; REC-030/RQ5 p6; REC-036/RQ6 p6; REC-041/RQ7 p5
S17	4	REC-027/RQ5 p3; REC-035/RQ6 p5; REC-037/RQ7 p1; REC-042/RQ8 p1
S26	3	REC-040/RQ7 p4; REC-044/RQ8 p3; REC-046/RQ8 p5

10 multi-record studies holding 28 records, plus 18 single-record studies, = 46 records over 28 studies.

Appendix D — Metadata conflict register (unresolved; both sides printed)

Per the project's standing rule, unresolved conflicts are presented side by side and no downstream section picks a side silently. Twelve conflicts are recorded. "Reference workbook" denotes the two bibliographic workbooks that serve as the de facto reference master; "Audit Appendix B" denotes the identifier list reconstructed inside the search-history audit.

Table D1. Bibliographic metadata conflicts C1–C12.

ID	Study	Field	Reference workbook says	Audit Appendix B says	Note
C1	S02 EmbC-Test	Year	"Not stated in supplied PDF"	2026	The identifier `arXiv:2603.09497` agrees in both. Its YYMM prefix (2603) implies March 2026, consistent with the audit — but that is an *inference from the identifier*, not a recorded year.

ID	Study	Field	Reference workbook says	Audit Appendix B says	Note
C2	S05 openSIL	Venue + identifier	both "Not stated in supplied PDF"	`arXiv:2606.19725v2`	The workbook records year 2026 (accepted 2026-04-16); the identifier prefix implies June 2026. **The determinations table records "NO stable identifier recorded on any copy", directly contradicting Appendix B.** This is the sharpest conflict in the set.
C3	S07 Smarter-not-Bigger	Year + venue + identifier	all three "Not stated in supplied PDF"	2025, `arXiv:2511.22584v1`	The prefix implies Nov 2025, internally consistent with the audit's year. The determinations table also says "NO stable identifier recorded on any copy".
C4	S08 H2LooP	Year + venue + identifier	"Not stated in supplied PDF (references 2026)"	2026, `arXiv:2603.11139v2`	The prefix implies Mar 2026. Determinations table: "NO stable identifier recorded on any copy". Per rule N8 the parenthetical is not a stated year, so S08 is counted as *not recorded* in Table 5 and Appendix C.
C5	S09 EmbedGenius	Year + venue + identifier	all three "Not stated in supplied PDF"	**2025**, `arXiv:2412.09058v2`	**Internal contradiction inside Appendix B itself:** the prefix implies Dec **2024** while the stated year is 2025, both values in the same source table. Determinations table: "NO stable identifier recorded on any copy".
C6	S10 Englhardt et al.	Year + venue + identifier	all three "Not stated in supplied PDF"	2023, `arXiv:2307.03817v2`	The prefix implies Jul 2023, consistent. The author surname *is* attested (register Details field). Determinations table: "NO stable identifier recorded on any copy".
C7	S24 CoverUp	Year	**2025** (workbook + register)	—	`arXiv:2403.16218` prefix implies Mar **2024**. A prior companion list printed "arXiv:2403.16218, 2025", carrying both values in one entry without flagging the mismatch.
C8	S13 Abtahi & Azim	Year	RQ5/RQ6/RQ7 rows: **2025**; **RQ4 row: "Not reported in the provided text"**	—	One study, four register rows, two year treatments. Not a contradiction — the RQ4 record simply lacks the field — but it must not be read as "the study has no year".
C9	S20 / RQ9-P6	Year	S20 row: **2020** (29th USENIX Security); RQ9-P6 row: **2019** (`arXiv:1909.06472`)	—	Two reports of one lineage, correctly separated. Citing them as one entry with two dates blurs the report layer, so they are split into [13] and [33].
C10	S18 FIDO	Year + venue	"Not explicitly stated on supplied PDF first page" (both)	—	A DOI *is* recorded (10.1145/3545948.3545957), so year and venue are recoverable from a DOI resolver — but that would be an external lookup, which the project's evidence rules bar. Left as `[NOT RECORDED]` .
C11	S23 DICE	Venue	"USENIX Security Symposium paper lineage stated in supplied PDF"	—	Not a venue string but a provenance note. The real venue must be transcribed from the paper.
C12	RQ9-P3 Asmita	Venue	"USENIX Security 2024 (as identified by the supplied final extraction/paper context)"	—	Self-labelled as a *derived* attribution. Must not be printed as a confirmed venue without checking the paper.

The shared root cause of C2–C6, worth the authors' attention. These five conflicts are not five independent errors. The determinations table states **"NO stable identifier recorded on any copy"** for exactly the five studies (S05, S07, S08, S09, S10) where Appendix B supplies concrete arXiv identifiers. One of these two records is wrong. Because the determinations table is the study-level

authoritative carrier while Appendix B is a reconstruction inside the search audit, the audit recommends treating the table as governing and the Appendix B identifiers as *proposed, unverified* — but this is an author decision, recorded as U2 below. If Appendix B is accepted instead, the table's identifier-poor flag must be corrected, or the two files will keep contradicting each other in any future audit.

Effect on the manuscript if resolved either way. No count, denominator, percentage or finding in Sections 4 to 8 depends on any of C1–C12. The affected surfaces are the reference entries themselves, Appendix C's identifier column, and — for C1, C5 and C7 only — the year distribution in Figure 2 and the "9 of 28 studies with no recorded year (32.1%)" statement in §4.0. Those two must therefore be re-derived *together with* the reference list, not separately.

Appendix E — Unresolved issues requiring author decision

Ordered by how much of the manuscript each one touches. Items U1 to U10 are carried forward from the citation audit; U11 and U12 arise from the criteria and screening records. None is resolved in this manuscript, and none silently alters a count.

Table E1. Open items, what each blocks, and what the authors must supply.

#	Issue	What it blocks	What the authors need to supply
U1	**S12 has no title, no identifier, no author and an unreadable local file.** It is one of the 28 included studies and one of the 3 grey-zone studies, so it *is* counted — but it cannot be cited.	A complete reference list; full citability of the 3-study grey-zone cohort.	The paper's identity. The local file retains only a Figure 1 caption ("GenAI-driven test case generation", with LLMs/VLMs/VSS for software-defined vehicles) and configurations C1–C4; it is CID-encoded and not machine-readable with available tooling. If it cannot be identified, the honest options are (a) keep it as an enumerated-but-uncitable study with a footnote saying so — the course taken here, which preserves every count in §4 — or (b) re-adjudicate its inclusion.
U2	**Identifier conflicts C2–C6:** the determinations table says "NO stable identifier recorded on any copy" for S05, S07, S08, S09 and S10, while audit Appendix B supplies concrete arXiv identifiers for the same five.	Whether entries [2], [5], [6], [7] and [11] may print an identifier at all.	A ruling on which record governs. The audit recommends that the determinations table governs and the Appendix B identifiers be marked *proposed, unverified*. If Appendix B is accepted instead, the table's identifier-poor flag must be corrected.
U3	**Feature-level identity matches M2 (S13 ≟ record R4_p4) and M1 (S14 ≟ record R3_p8)**, both pending. **M2 is the only documented bridge between the two corpus clusters.**	The strength of every cross-cluster statement in §5 and §6; whether entries [8] and [26] are one study or two; the asterisked cells of Figure 13.	A reference-list comparison of the two source records (author, year, title, identifier). Note that DOI comparison is **not possible** for M2, because the RQ9 records list that study's venue and DOI as not reported; the match must rest on `arXiv:2509.09970` or on full-text features. **If M2 is rejected, the RQ4×RQ5/RQ6/RQ7 overlaps drop to 0 and the two clusters become fully disjoint.**
U4	**"MuTAP", "Garcia-Carrasco" and "FIDO" are unattested work names**, used 40, 17 and several times respectively across §4–§8, while appearing in **no** bibliographic field of either workbook, the register or the determinations table.	Whether §4–§8 are citing the papers' own tool and author names or synthesis-layer shorthand. "Garcia-Carrasco" is additionally a *surname*, so it implies an author list no record supports.	Confirmation from the three papers (S27, S28, S18) that these are their own names. If any is not, the prose must switch to the registry title. This is a short check against three documents and it removes the largest remaining fabrication-risk surface in the manuscript. The registry titles are: *Effective test generation using pre-trained Large Language Models and mutation testing*; *Closed-loop evaluation of LLM agents for embedded software development*; *Stop Starving or Stuffing Me: Boosting Firmware Fuzzing Efficiency with On-demand Input Delivery*.

#	Issue	What it blocks	What the authors need to supply
U5	**Year conflicts C1, C5, C7** (S02, S09, S24), including C5's internal contradiction where one source table gives both "2025" and an identifier implying 2024.	A defensible year in entries [1], [7] and [17]; and any statement about the corpus's publication-year distribution.	The publication year from each paper. §4.0 reports 9 of 28 studies with no recorded year (32.1%), so resolving these three would change that count: **Figure 2 and §4.0 must be re-derived together with the reference list**, not separately.
U6	**Venue strings that are provenance notes rather than venues** — C11 (DICE) and C12 (Asmita / Fuzzing BusyBox).	Entries [16] and [31] being publishable.	The actual venue line from each paper.
U7	**Section 1 must mark [19] and [21] as outside the reviewed corpus.** Guzu et al. [19] is absent from the 105-record screening register entirely, having entered the project via the RQ10 dataset; [21] is RQ9-P4.	The claim in §4.0 that 105 records → 46 → 28 is the complete selection chain. If a reader assumes [19] or [21] is an included study, the arithmetic appears not to close.	**Applied during assembly:** both are now marked in §1 and listed in Part B of the References. No count changes. Recorded here for the audit trail.
U8	**Author lists are recorded for only 6 of the 36 reference entries.**	Any author–year citation style.	Author lists from the papers, or a decision to submit with a numbered style. Given that 30 of 36 entries lack authors, **an author–year style (APA, Harvard) cannot be produced from these records at all**; the numbered style used here is also what §1 already uses.
U9	**Citation style and target venue were never supplied.**	Final formatting of every entry.	The venue. The References use a neutral numbered style with journal name, volume:article, year and DOI. Conversion to IEEE, ACM, APA or Elsevier numbered is mechanical once the venue is named.
U10	**Overlap-band count contradiction.** The cross-RQ synthesis narrative asserts **15** zero-overlap pairs and an 11-pair low band, while its own matrix yields **14** and 12.	One figure in the Abstract, §4.9.4, §4.11 note N7, §5, §6.0, §6.4, §6.8, §8.2, Figure 13 and its source table.	A ruling on which value governs. **This manuscript adopts the matrix value 14 throughout**, because the matrix is computed cell by cell from the study-set memberships and is the same set of values plotted in Figure 13 and exported in `fig12_cross_rq_overlap_matrix.csv` ; the narrative's 15 is not reproducible from that table. Adopting 15 instead would require re-deriving the figure and every dependent sentence. §3.6 lists the item as requiring author verification.
U11	**The full text of criteria IC-1 to IC-6 and of defaults DP-2A and DP-6A is not stored in any project file.**	§3.4 reporting a complete, auditable eligibility specification.	The criterion and default texts as the authors applied them, or confirmation that the operative-role descriptions in Appendix B.2–B.3 are the fullest available record.
U12	**RQ1 and RQ2 have no exclusion records at all**, so it cannot be confirmed whether their candidates passed systematic screening.	The claim that one screening procedure produced all eight datasets; the selection-bias assessment in §7.2.	How RQ1 and RQ2 candidates were and were not considered. Until then §3.5 and §7.2 report the asymmetry as a documented limitation.

Items deliberately not listed as open. Two matters that earlier project passes carried as pending were resolved on the evidence and are therefore *not* in Table E1. First, the coexistence of "PENDING" and "FINAL" values on all 46 included rows was determined to be a field-synchronisation artefact rather than a substantive conflict: the determination column whose header carries the dated author decision governs, the 28-row determinations table independently corroborates it, and all downstream RQ documents already use it (DP-1A ×8, DP-3A ×6, DP-4A ×4, DP-5A ×1). Second, the report-layer count was resolved at **29** rather than left as a 28–46 range, on the strength of the S26 version pair, the normalisation of S06's two DOI spellings to one report, and the RQ8 final answer's independent statement of "5 documents, 4 studies".

Appendix F — Supplementary materials, data dictionaries and reproducibility artifacts

F.1 Project records underlying every count in this manuscript

Table F1. Authoritative carriers, what each establishes, and where it is used.

Record	Content	Used for
`includen and exclude for rq1–rq8.xlsx`	Master screening workbook, 105 rows = 46 included + 59 excluded; sheets *Combined List* and *Summary*	§3.6, §4.0, Figure 1, Figure 15, Appendix C
`Screening_Register_RQ1–RQ8.csv`	105 rows × 22 columns, unique IDs REC-001…REC-105; discovery route, duplicate status, exclusion codes, cohort tags, determination fields	§3.2–§3.6, §7.1–§7.2, Appendix A, Appendix B
`Study_Level_Final_Determinations_RQ1–RQ8.csv`	28 study rows S01–S28; `Records(n)` sums to 46; cohorts 13 / 8 / 4 / 3; 0 full-text exclusions	Table 3, Figure 1, Appendix C, §4.0
`deepseek_csv_20261012_f6f391.txt`	The single documented discovery route: 59 excluded records, 5 columns, no query, no result count	§3.2, §3.10, Appendix A.1
`SLR_Search_Audit_Report.md`	Search-history audit; proposed (never executed) supplementary protocol; identifier reconstruction in Appendix B	§3.2–§3.3, §7.1, Appendices A, D
PRISMA count audit and final-status report	Box-by-box evidence status; three-layer counting; file-level forensics on the 46 local documents	Figure 1, §3.6, §7.4, §7.6
RQ1–RQ9 final verified extraction syntheses	Per-RQ datasets, denominators, configuration-level records, zero blocks, self-qualifications	§4.1–§4.9, Figures 4–14
Revised cross-RQ synthesis	28-pair overlap matrix; two-cluster structure; discrepancy log D-1…D-11; uncertainty register V1–V8; gap matrix G1–G16	§4.9.4, §5, §6, Figure 13, Figure 14
RQ9 verification documents (six)	Per-item limitation evidence with L-codes, verbatim quotations, page/section locators and verification rulings	§4.9, §6, Figure 14
Citation audit and corrected reference list	Entry-by-entry transcription, conflict register C1–C12, name-normalisation table, open items U1–U10	References, Appendices D, E

Two same-named PDF exports of the screening register and the determinations table exist in the project. Each is a single page with only a handful of extractable cells and neither can serve as equivalent evidence; **the CSV carriers are authoritative** and are the ones cited throughout. Two further PDFs (`PRISMA without graph.pdf` , `PRISMA2020_CALCULATION!.pdf`) were established by bookmark-level comparison to be print copies of the Markdown audits and contain no additional counts.

F.2 Data dictionaries for the counting layers

Record (n = 46). One row of the screening register. A study entered for several research questions appears as several records. The 18 study-level collapses are cross-RQ re-registrations of the same study, **not** PRISMA duplicate-removal events; duplicate records removed is 0.

Report (n = 29). One published or deposited document. 28 studies contribute one report each, plus the documented S26 conference-and-preprint version pair. S06's two DOI spellings normalise to one report. The report count is **not** the 46 files on disk, because 36 of those are derived verification documents rather than publications.

Study (n = 28). One unit of scientific work, S01–S28. Not the same as "28 LLM studies": only 13 belong to the LLM-construct cohort.

Configuration. An experimental setting, model variant or prompt condition within one study. Configuration-level counts (51 RQ2, 32 RQ3, 27 RQ4, 24 RQ5, 11 RQ6, 25 RQ7, 46 RQ8, 79→75 RQ9 limitation items) are never presented as study prevalences.

Limitation instance (75 retained, of 79 extracted, 4 rejected). An RQ9-only unit. Never used as a denominator: the RQ9 study denominator is 6 throughout. One study (S17, ADFEmu) contributes 13 of the 75 instances (17.3%) but counts once per gap category.

F.3 Figure source data

Every figure in this manuscript is generated exclusively from audited values, and each has an exported source table.

Table F2. Figure-to-source mapping.

Figure	Subject	Source data
1	PRISMA 2020 flow, template 2	Box values from the PRISMA count audit §4, transcribed into `build_prisma_en.py` ; each box carries its evidence status
2	Publication years at study level	`fig01_publication_year_study_level.csv` , `fig01_study_year_detail.csv` (rule N8 applied; missing years never imputed)
3	RQ dataset scale and cohort composition	`fig02_rq_dataset_scale.csv`
4	RQ1 test types	`fig03_rq1_counts.csv`
5	RQ2 models and strategies	`fig04_rq2_counts.csv`
6	RQ3 hardware/software context	`fig05_rq3_counts.csv`
7	RQ4 dependency-handling families	`fig06_rq4_counts.csv`
8	RQ5 interrupts, asynchronous and timing	`fig07_rq5_counts.csv`
9	RQ6 execution environments	`fig08_rq6_counts.csv`
10	RQ7 feedback loops	`fig09_rq7_counts.csv`
11	RQ8 metrics and validation	`fig10_rq8_counts.csv`
12	RQ4 artefact × verification-status matrix	`fig11_rq4_artefact_matrix.csv`
13	Cross-RQ study-set overlap matrix	`fig12_cross_rq_overlap_matrix.csv`
14	RQ9 gap evidence matrix	`fig13_rq9_gap_matrix.csv`
15	Screening-side exclusion reasons	Derived from the register's category labels; framed so it is not read as a PRISMA stage count

F.4 Reproducibility scripts

Script	Function
`derive_data.py`	Recomputes all structural counts from the three authoritative tables. Fails hard on a missing sheet, column or row rather than defaulting.
`fig_content.py`	Holds the verbatim RQ-content values, each row carrying its source anchor.
`render_dashboard.py` , `render_lib.py`	Render the figure dashboard and standalone figures as inline SVG with inline CSS and minimal inline JS. No CDN, no external dependency.

Script	Function
`xlsx_stdlib.py`	Reads the `.xlsx` carriers using only the Python standard library (`zipfile` + `xml.etree`), because no spreadsheet library is available in this environment.
`validate_output.py` , `validate_offline.py`	Structural validation and offline-safety scan (zero external URLs, `@import` , script sources or CDN libraries).
`verify_anchor_trace.py` , `verify_independent.py`	Verify that every anchor-bearing content item locates in its named source, and re-check drawn labels against the source dictionaries.
`build_prisma_en.py`	Generates Figure 1 with computed geometry and eight validation checks: XML well-formedness, box-overlap detection, text-fit, vertical containment, offline safety, presence of every verified count token, one status chip per box, and absence of residual non-English text.
`build_appendix_c.py`	Transcribes Appendix C from the two authoritative carriers and asserts that `Records(n)` sums to 46 and that 28 study rows are present.

Regenerate the figure set with `python derive_data.py` followed by `python render_dashboard.py` ; regenerate Figure 1 with `python build_prisma_en.py` ; regenerate Appendix C with `python build_appendix_c.py` .

F.5 Validation actually performed, and validation not performed

Performed. Static validation of all figure SVGs (XML well-formedness, balanced tags, offline safety, and a numeric cross-check that every drawn label and every tabulated n/N matches its source dictionary — 14 figures, 0 mismatches; 134 of 134 anchor-bearing content items located in their named sources); 16 arithmetic reconciliation relations computed from the registers, of which 15 close exactly and the one non-closing relation is a records-versus-studies unit mismatch by design; three-source agreement on the 105 / 46 / 59 counts; set-equality of the 59 excluded titles across three carriers; zero cross-membership between the included and excluded sets; 46 distinct file hashes across the 46 retained records; and the eight geometry and content checks on Figure 1 reported in §F.4.

Not performed, and not claimed. Live browser rendering of the figure pages — no browser runtime is available in this environment and none was downloaded, so the print stylesheet and pixel layout were not exercised; PNG export of figures, for the same reason, with high-resolution SVG supplied instead; any external bibliographic lookup (DOI resolver, arXiv API, web search); independent re-extraction of any record by a second extractor; any formal error-rate study of the extraction pipeline; and any re-run of the RQ analyses, which the audit chain verified in place rather than recomputed.

Appendix G — Factual claims in Section 3 and their supporting project files

This table is the claim-by-claim provenance record for Section 3. Two of its columns reproduce source material exactly as it exists in the project, and both therefore retain non-English text. **File names** are given verbatim because they are literal paths on disk; translating one would break traceability. **Quoted source strings** in the final column are given verbatim because they are evidence of what the record says; translating one would make it no longer a quotation. The underlying project documentation was written in Chinese, and this appendix is the one place where that is deliberately visible rather than smoothed over. Where such a string carries meaning for a claim in the manuscript body, an English gloss is supplied at that point in the body text.

#	Claim as stated in Section 3	Supporting file(s)	Where in source
A1	105 registered records; 46 retained / 59 excluded	`PRISMA\includen and exclude for rq1-rq8.xlsx` (Combined List, Summary sheet); `includeEXCLUDE\Screening_Register_RQ1-RQ8.csv` ; PRISMA final-status record §2.1	Summary rows "Included 46 / Excluded 59 / Total 105"; register data rows; three-source title-set equality

#	Claim as stated in Section 3	Supporting file(s)	Where in source
A2	No database search documented; 0/105 rows attribute a database; zero hits for Scopus/WoS/IEEE Xplore/ACM DL/DBLP/PubMed in 105×22 cells; no .ris/.bib anywhere	Register col05; search-history audit §B.1–B.2, §6.3; PRISMA final-status record §3 item 1	"Discovery route" distribution; explicit prohibition table
A3	DeepSeek export 2026-10-12 = 59 excluded-only records, 5 columns, no query/result count	`deepseek_csv_20261012_f6f391.txt` ; search-history audit A.2 activity 1	header row; audit table (query ❌ / result count ❌)
A4	46 included records' discovery route NOT RECORDED; "custody only, not route"	Register col05 (46 rows)	verbatim qualifier
A5	11 `rq_N find` files are extraction docs, not search artefacts	PRISMA final-status record §3 item 3	"PAPER IDENTIFICATION" structure
A6	Registers NOT APPLICABLE	PRISMA final-status record §3 item 2	21-text keyword scan = 0
A7	PROPOSED queries never executed; supplementary search only proposed	search-history audit §3.2 (PROPOSED QUERY + unfilled templates), §5.1 action 1	"[执行后填写]" placeholders
A8	No protocol registration	zero hits for registration evidence; only prohibition phrasing in prompt library	PRISMA record §3 item 2 parenthetical
A9	Criteria PROPOSED 2026-10-12 and applied retroactively; DP defaults 1A–6A author-accepted 2026-10-12; 42 INCLUDED + 4 RETAINED = 46; 25 + 3 = 28	Register col08/col19 headers and cells; `Study_Level_Final_Determinations_RQ1–RQ8.csv`	column header string; DP citations (8/6/4/1)
A10	IC-1 (grey lit → DP-5A), IC-5 (LLM construct), IC-6 (indirect) usage	Register rows S01–S14 passim	tag strings; full IC text absent
A11	EC-1..EC-6 definitions, counts 30/9/5/5/5/5, Σ=59, retroactive-code labels	Register col09/col18; PRISMA record §4 note	verbatim "(retroactive code)"
A12	12-category → EC 1:1 reverse mapping	PRISMA record §4 (EC distribution paragraph)	mapping statement
A13	RQ1/RQ2 no exclusion pool; systematic screening of their intake unconfirmed	search-history audit §A.3/§A.4/§B.2 item 5	verbatim "无排除记录" ("no exclusion records"); explicit caveat
A14	Deduplication = annotation; 0 removed; 10 clusters (sizes as listed); 77 "no duplicate… normalized-title clustering"; excluded side scoped out	Register col06/col11; PRISMA record §2.3–2.4	cluster strings verbatim
A15	Automation tools unused (tool-name 0 hits)	PRISMA record §3 item 5	keyword table
A16	Stage of adjudication not recorded; title/abstract for excluded = flagged inference	Register col07/col09/col10	"(stage inferred, see Notes)"
A17	Reviewer identity & screening date NOT RECORDED on all 105 rows	Register col10	verbatim
A18	46 sought, 0 not retrieved; full-text exclusions 0 (this register)	PRISMA record §3 items 6–7, §4 boxes 8–11	file-level evidence + reason scan
A19	10 originals + 36 derived docs; 18 studies (S11–S28) without originals; 22/22 recorded original filenames absent; 8 producer signature groups; font-MD5 link	PRISMA record §5.1–5.3	producer table, corroboration items 1–3
A20	Report layer = 29 (S26 version pair; S06 DOI normalisation; ≠46 files)	`Study_Level_Final_Determinations` col08; PRISMA record §2.4, §5.4	version-pair note
A21	Cohorts 13/8/4/3 with IDs; LLM denominator 13 ≠ 28	PRISMA record §2.5; determinations col07	cohort table

#	Claim as stated in Section 3	Supporting file(s)	Where in source
A22	Arithmetic: 15/16 relations close; R4 unit mismatch by design	PRISMA record §7 (R1–R16)	per-formula table
A23	Open conflicts: REC-002↔REC-011 note mis-aimed; 14-vs-15 zero-overlap wording gap; M2 unresolved; S12 identity unresolved	PRISMA record §2.4, §8, §9; cross-RQ mother doc §2.1 vs §2.3	status rows
A24	No QA applied; QA1–QA7 proposal-only	prompt library Prompt 6; absence in register/determinations	"label it as a proposal"
A25	Extraction granularity 46 studies / per-RQ config counts (51/32/27/24/11/25/46/79→75)	cross-RQ mother doc §1 table; RQ9 doc §1.1	denominators table
A26	Verification structure (Previous/Verified/Evidence/Page/Reason-for-Change) & rules (no silent change, no invented quotes, Not reported ≠ No)	36 verification PDFs sampled in PRISMA record §5.1; prompt library Prompt 7	content-type evidence
A27	No meta-analysis: RQ8 four zeros + 6-dim heterogeneity; RQ1 incomparable funnels; per-RQ no-extrapolation quotes	mother doc §1.1, §3.7 area, §8.1 D/V entries	verbatim quotes
A28	RQ9 discipline: n=6 studies; ADFEmu 13/75=17.3% instances, 1/category; P2IM reference-only; no bar chart	`RQ9_研究缺口与未来方向.md` §1.2/§3.1/§3.2; PRISMA record §6	declared compliance
A29	RQ8 n=4 vs 5-document disclosure with both percentage sets	`RQ_8 FINAL ANSWER!.pdf` §10 (via PRISMA record §6)	self-disclosure
A30	Two-cluster structure; overlap distribution 14/12/2 of 28 pairs; bridge via M2 pending	mother doc §2.1, §2.3, §6.3	matrix + paragraph
A31	Construct-validity: no study meets all four elements	RQ9 doc §Abstract (Chinese-titled section); mother doc RQ9 section	quoted threat
A32	AI mentions 57/60 are evaluated models, 0 discovery evidence	PRISMA record §3 item 3	forensic count
A33	Retrospective character: audits dated 2026-10-12; verification docs 2026-10-08/09; RQ2 report 2026-10-09	audit report header; PRISMA record §5.1 date clusters; audit §A.4	dates
A34	Years 2020–2026 in retained records; 17/46 no usable year; year not a criterion	recomputed from register col15 by `derive_data.py` (bins 2/14/13/17)	figure pipeline audit
A35	S04 master's thesis descriptive-only restriction	determinations col08; DP-5A notes	verbatim restriction

Claims we deliberately do NOT make (per project-internal prohibition list, PRISMA record file §10.3): "we systematically searched Scopus/WoS/IEEE/ACM"; "N records identified" for any N ≠ the registered 105; "deduplication left 28 records"; "18 records removed"; "0 full-text exclusions proves strict/lenient criteria"; "this review complies with PRISMA 2020"; "28 LLM studies".

Prepared 2026-10-13 from: `C:\A_RESEARCH!\PRISMA\SLR_Search_Audit_Report.md` ; `C:\A_RESEARCH!\PRISMA\PRISMA_md file.md` ; `C:\A_RESEARCH!\PRISMA\PRISMA2020_计数复算与一致性审计.md` ; `C:\A_RESEARCH!\PRISMA\prisma_flow_diagram.html` (§10); `C:\A_RESEARCH!\PRISMA\includen and exclude for rq1-rq8.xlsx` ; `C:\A_RESEARCH!\includeEXCLUDE\Screening_Register_RQ1-RQ8.csv` ; `C:\A_RESEARCH!\includeEXCLUDE\Study_Level_Final_Determinations_RQ1-RQ8.csv` ; `C:\A_RESEARCH!\deepseek_csv_20261012_f6f391.txt` ; `C:\A_RESEARCH!\CROSS_RQ_SYNTHESIS!\REVISED FINALMD FILE.md` ; `C:\A_RESEARCH!\Research GAPS AND FUTURE DIRECTIONS!\RQ9_研究缺口与未来方向.md` ; `C:\A_RESEARCH!\SLR_Prompt_Library_RQ1-RQ9_HTML_PRISMA.txt` . The two supplied PDFs (`PRISMA without graph.pdf` , `PRISMA2020_CALCULATION!.pdf`) were established in prior forensic passes to be print copies of the two Markdown audits (PRISMA record §1: bookmark-level comparison, no new counts); no PDF-specific extraction was required for this draft.